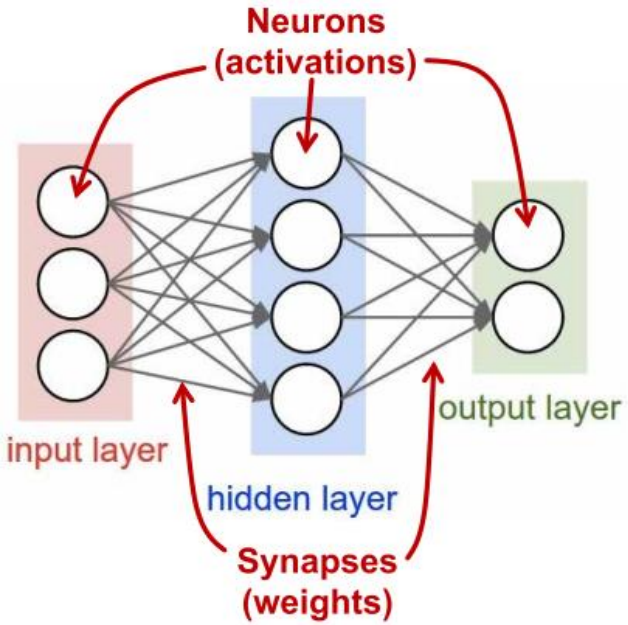
On-Device AI 기본

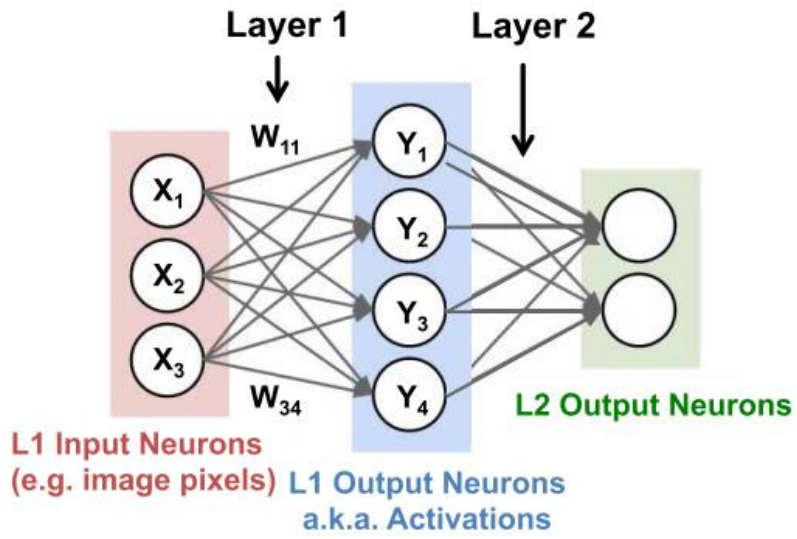
성균관대 신동군 교수

Outline

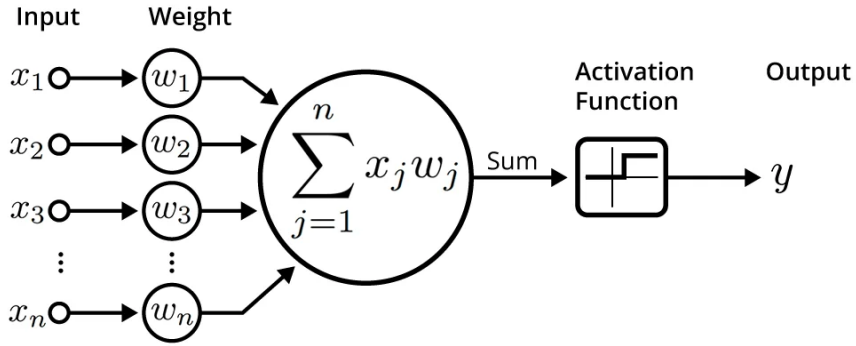
- On-Device AI
- Network Pruning
- Quantization
- Knowledge Distillation

Deep Neural Network





$$y = ReLU(\sum w_i x_i + b)$$



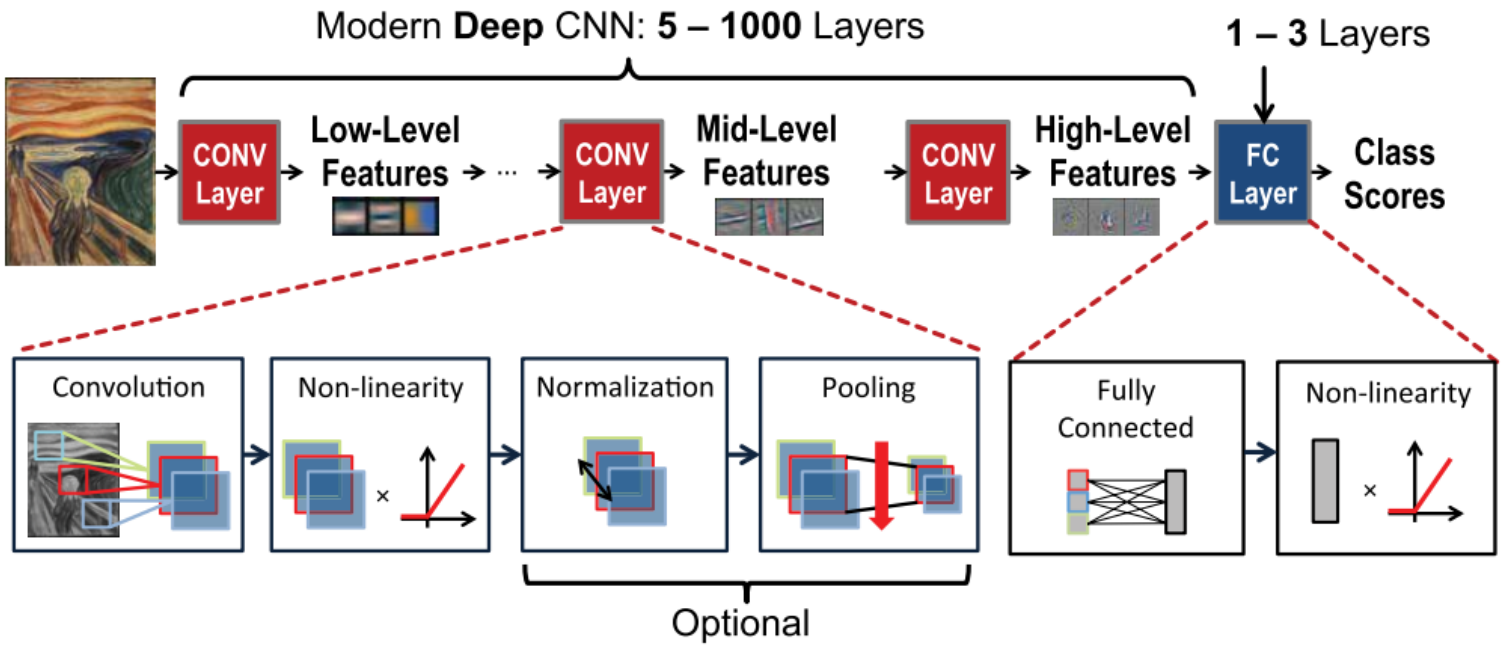
An illustration of an artificial neuron. Source: Becoming Human.

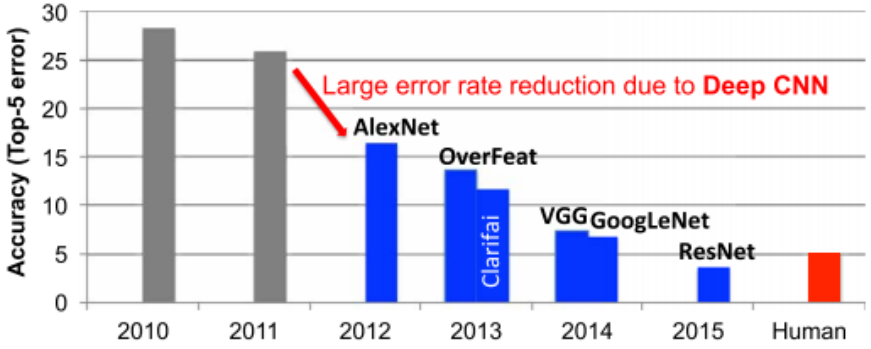
Efficient Processing of Deep Neural Networks: A Tutorial and Survey [Sze et al. 2017]

https://insights.sei.cmu.edu/blog/deep-learning-going-deeper-toward-meaningful-patterns-in-complex-data/

Convolutional Neural Networks (CNNs)

- Processing of 2-D features
- Conv layer, Pooling layer, Fully-Connected (FC) layer
- Each layer generates a successively higher-level abstraction of the input data, called a feature map (fmap)

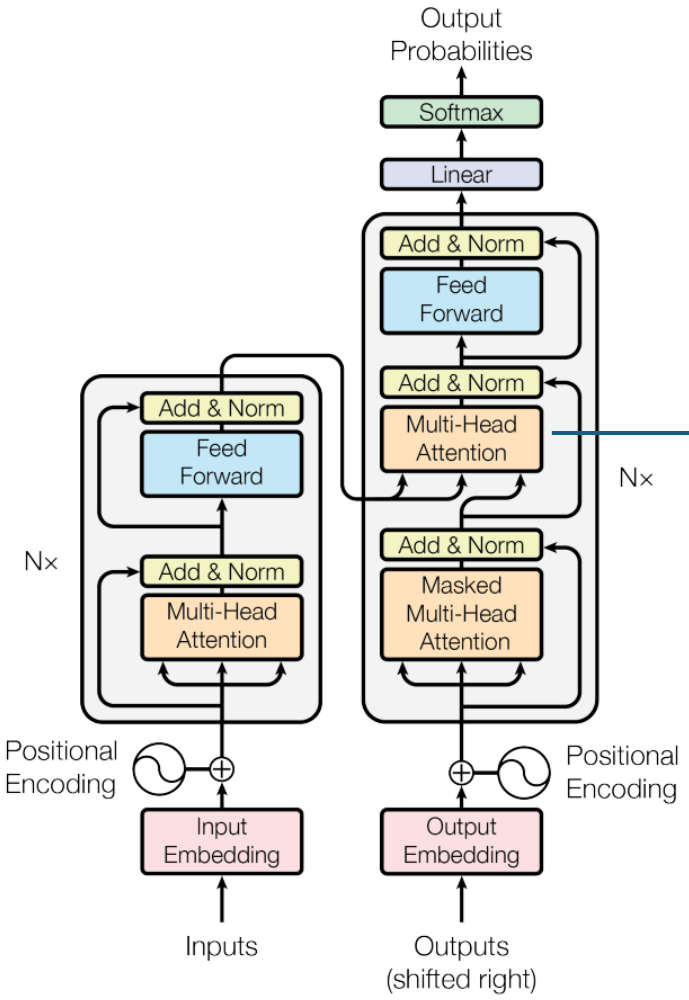


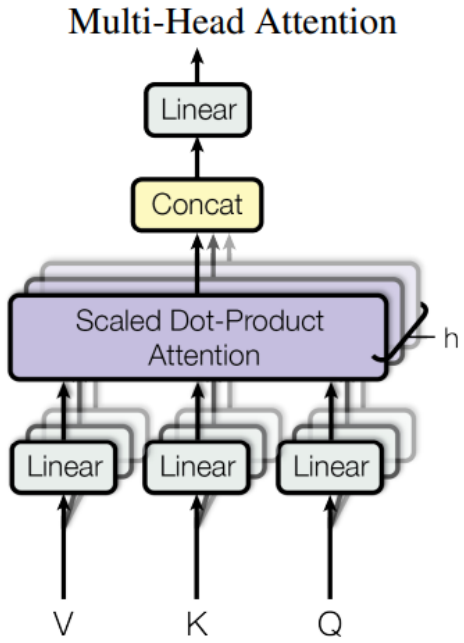


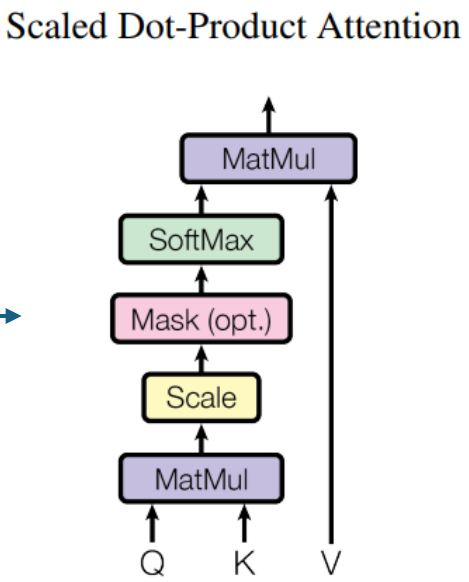
Results from the ImageNet Challenge

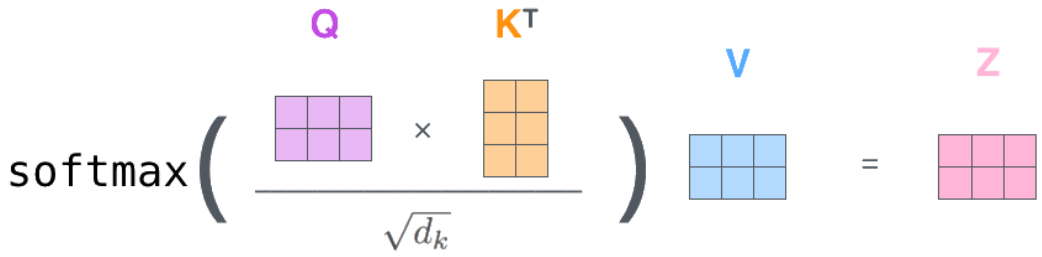
Efficient Processing of Deep Neural Networks: A Tutorial and Survey [Sze et al. 2017]

Transformer







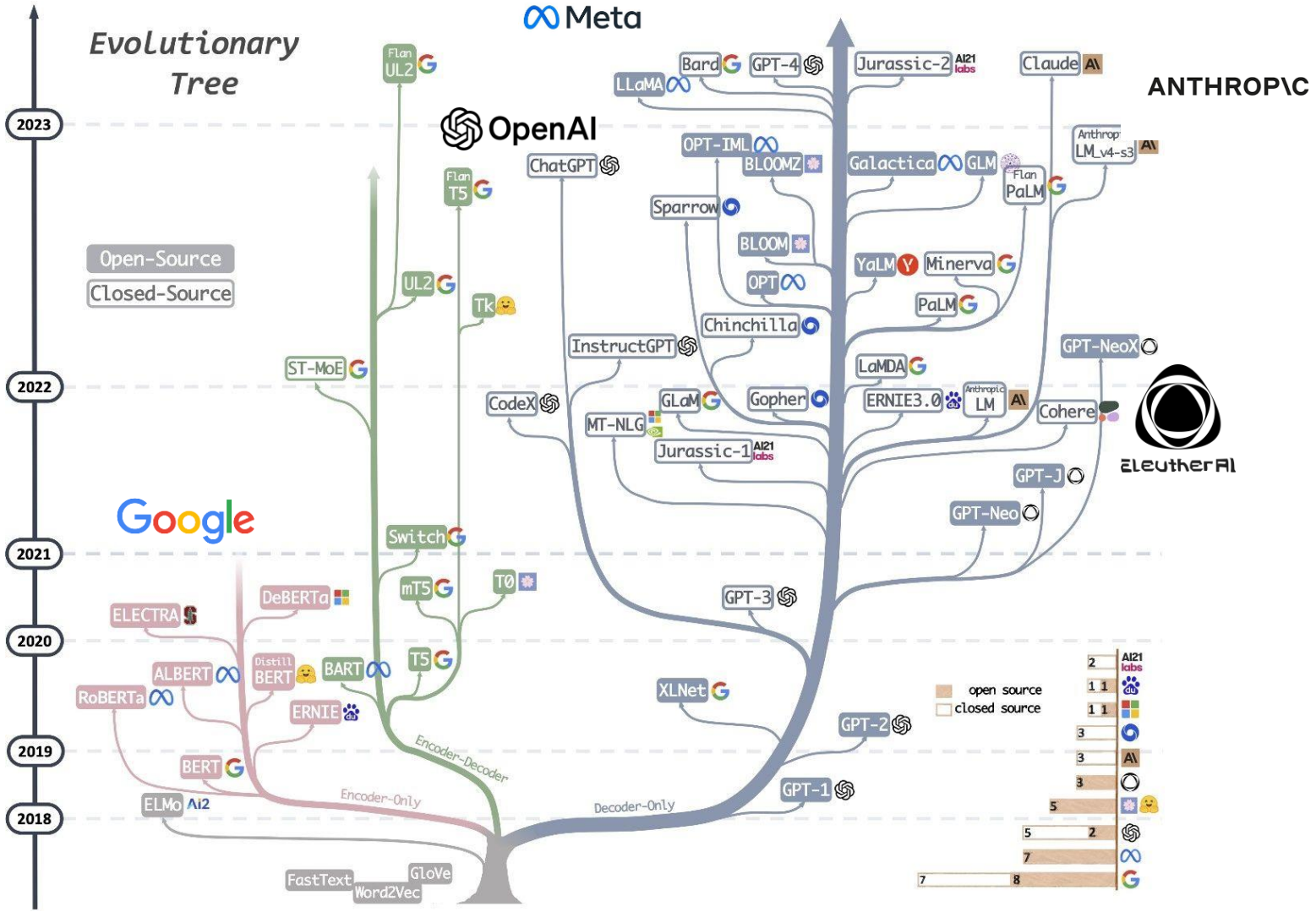


https://benjaminwarner.dev/2023/07/01/attention-mechanism

Encoder Decoder

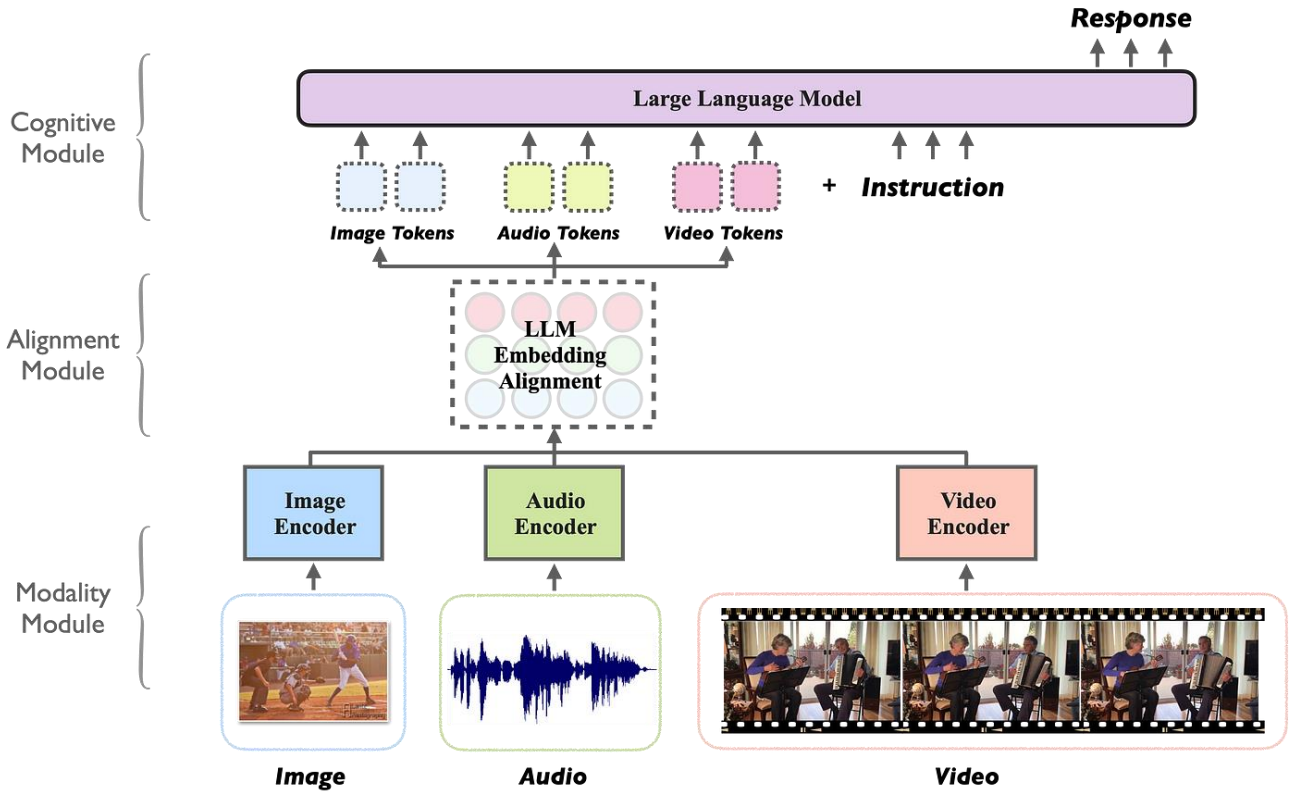
Attention Is All You Need [Vaswani et al. 2017]

Large Language Models

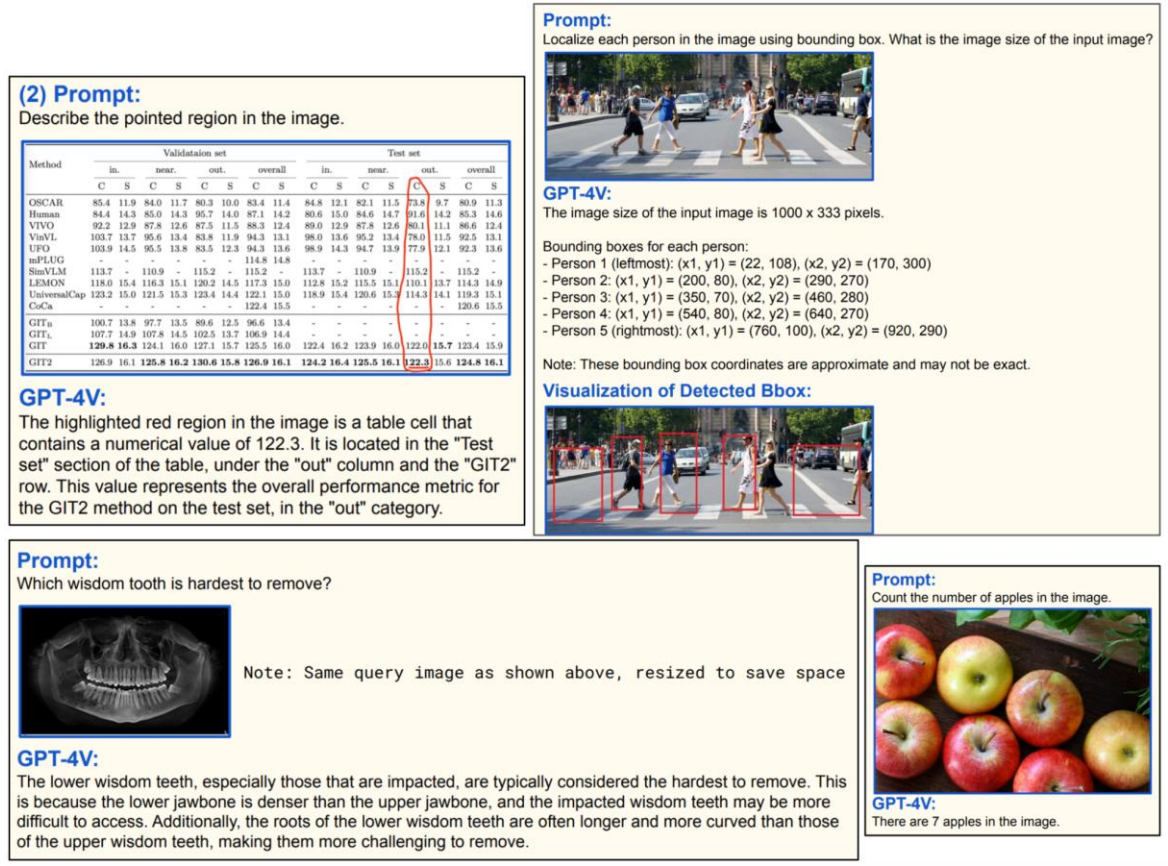


Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond [Yang, 2023]

Multi-modal LLM



https://medium.com/@cout.shubham/exploring-multimodal-large-language-models-a-step-forward-in-ai-626918c6a3ec

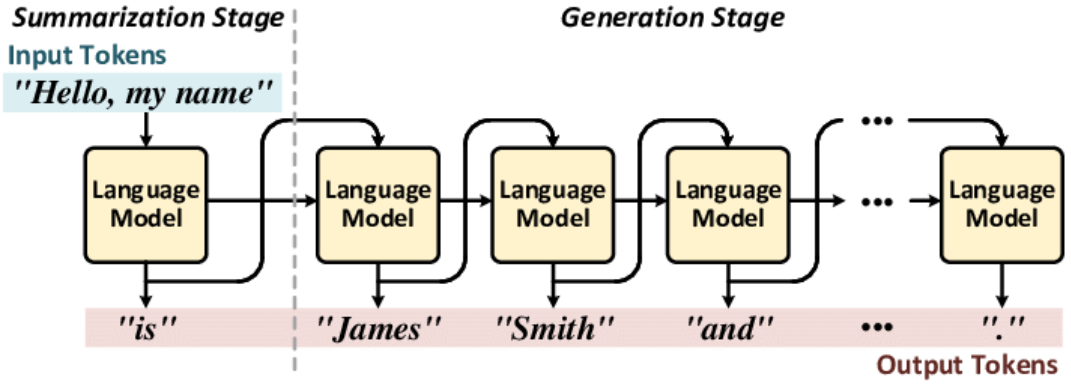


GPT-4V(ision) can take visual inputs

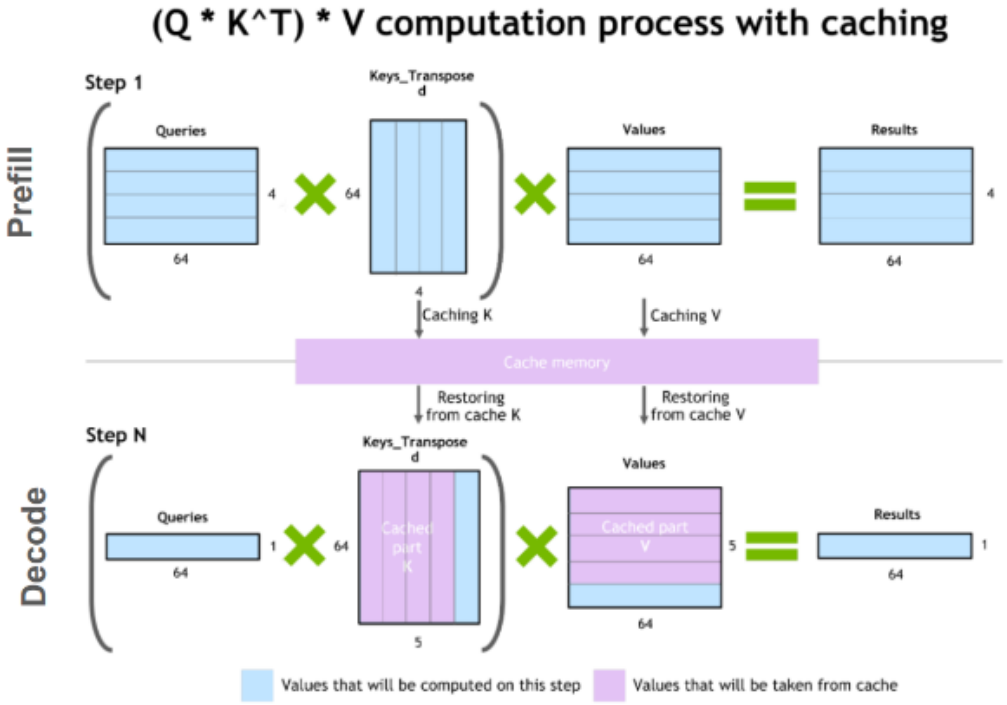
https://huyenchip.com/2023/10/10/multimodal.html

LLM Operations

- **Prefill phase**
  - Process the input tokens to compute the intermediate states (keys and values)
  - Masked GEMM
- **Decode phase**
  - Generate output tokens autoregressively one at a time.
  - Each sequential output token needs to know all the previous iterations' output states (KV Cache).
  - GEMV (underutilize GPU)
  - Memory-bound: data (weights, keys, values, activations) are transferred to the GPU from memory
  - Can improve GPU utilization by request batching



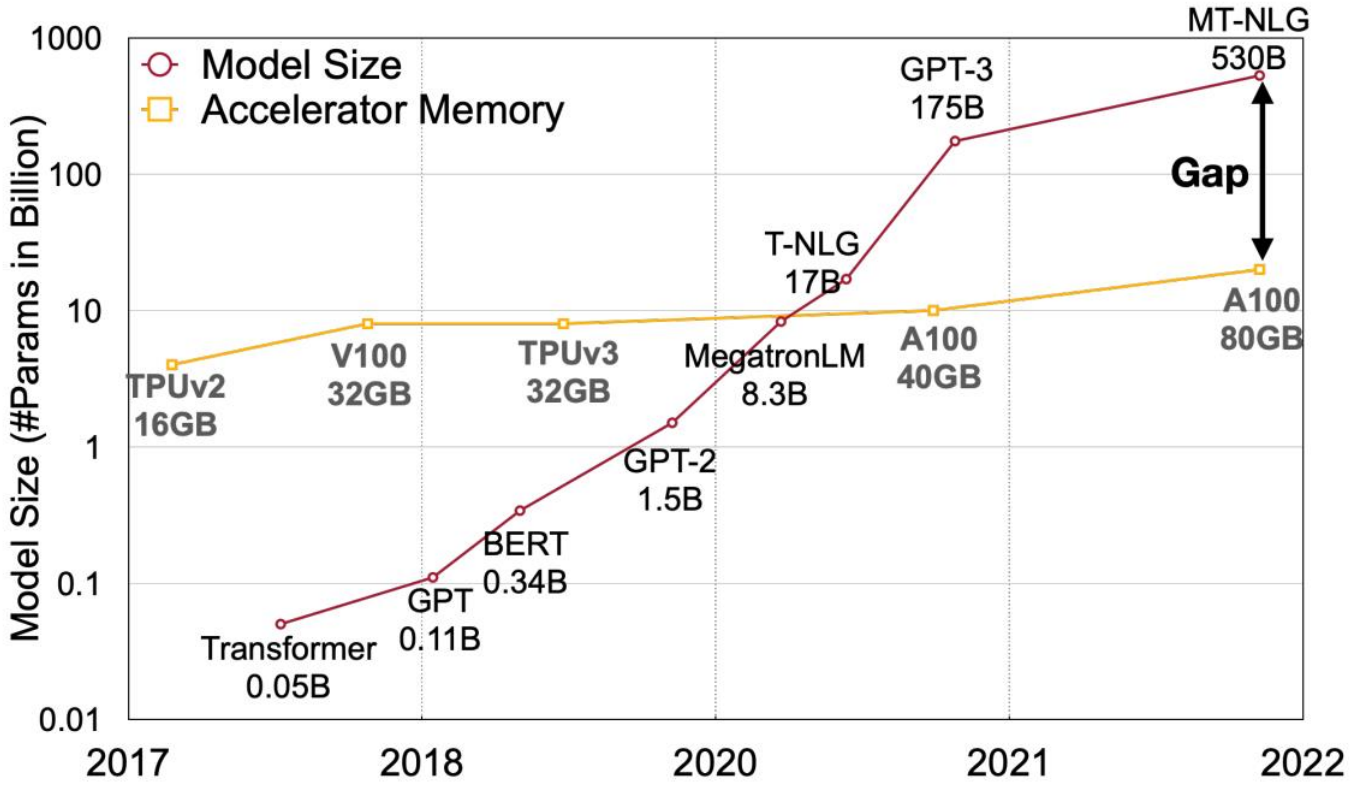
https://arxiv.org/pdf/2209.10797



https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/

Today's AI is too BIG!

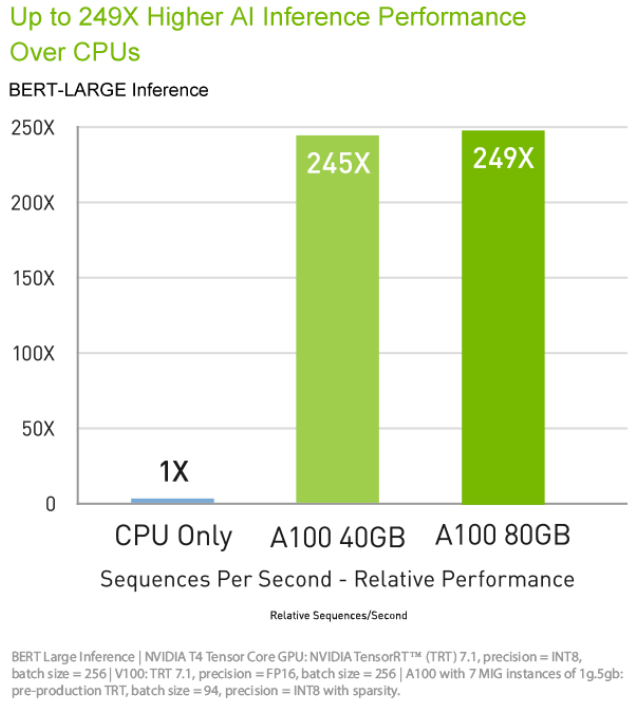
- The model size of LLMs is developing at a faster pace than GPU memory.
- Big gap between the supply and demand for memory.
- **Quantization and model compression** techniques can help bridge the gap.



SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

Computing Power

- Traditional microprocessors are not designed to deal with Machine Learning.
- GPUs with thousands of cores can speed up the ML training process.
- NVIDIA Selene Supercomputer
  - 560 DGX A100 = 4,480 GPUs
  - DGX A100 = 8 x A100



NVIDIA DGX A100 https://www.nvidia.com/ko-kr/data-center/a100/



GPUs	**8x NVIDIA A100**
GPU Memory	320 GB total
Peak performance	5 petaFLOPS AI | 10 petaOPS INT8
NVSwitches	6
System Power Usage	**6.5kW max**
CPU	**Dual AMD Rome 7742** 128 cores total, 2.25 GHz(base), 3.4GHz (max boost)
System Memory	**1TB**
Networking	**8x Single-Port Mellanox ConnectX-6** 200Gb/s HDR Infiniband (**Compute Network**) 1x (or **2x***) **Dual-Port Mellanox ConnectX-6** 200Gb/s HDR Infiniband (**Storage Network** also used for Eth*)
Storage	OS: 2x 1.92TB M.2 NVME drives Internal Storage: 15TB (4x 3.84TB) U.2 NVME drives
Software	Ubuntu Linux OS (**5.3+ kernel**)
System Weight	271 lbs (123 kgs)
Packaged System Weight	315 lbs (143 kgs)
Height	**6U**
Operating temp range	**5°C to 30°C** (41°F to 86°F)



https://cambrian-ai.com/nvidia-provides-more-details-on-selene-supercomputer/

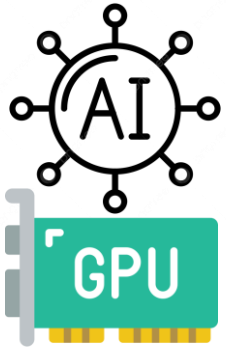
Cloud AI vs. On-Device AI



대용량 AI 모델

고성능 컴퓨팅 환경

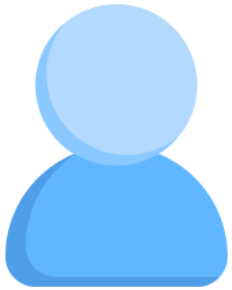
클라우드 인프라



Cloud GPU Server

데이터 입력

AI 연산 결과



User

데이터 입력

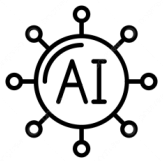
AI 연산 결과



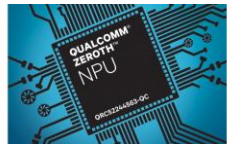


Device

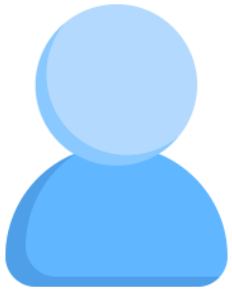
1. 빠른 응답
2. 보안 강화
3. 네트워크 연결 불필요
4. 비용 절감
5. 개인화된 서비스



경량화된 AI 모델



지능형 반도체
(NPU, PIM)



User

데이터 입력

AI 연산 결과





Device

Smartphone Memory



Galaxy S24 Ultra



Galaxy S24+



Galaxy S24



Galaxy A24



iPhone15



iPhone15-Pro

RAM	12GB LPDDR5X		8GB LPDDR5X		4GB	6GB	8GB
Storage	256GB/512GB/1TB UFS 4.0	256GB/512GB UFS 4.0	128GB UFS 3.1	256GB UFS 4.0	128GB	128~512GB	128GB~1TB





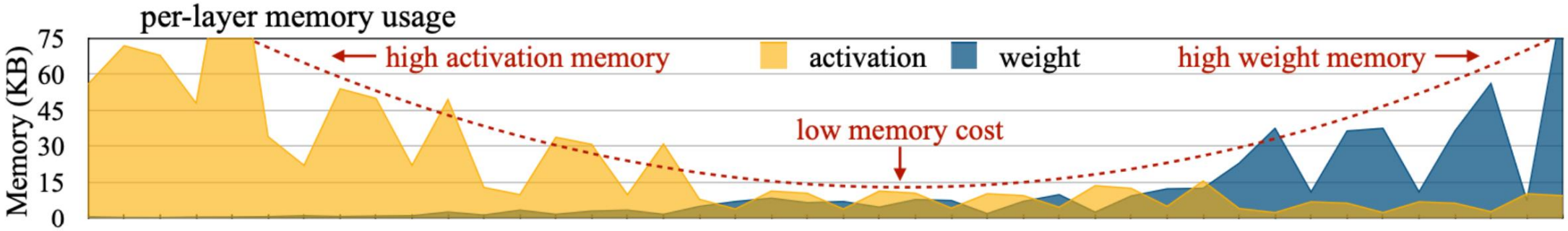
Model	Parameters	FP16	INT4
Gemma2	2B	4GB	1GB
Phi-3.5-mini	3.8B	7.6GB	1.9GB
Llama2	7B, 13B, 70B	14, 26, 140GB	3.5, 6.5, 35GB

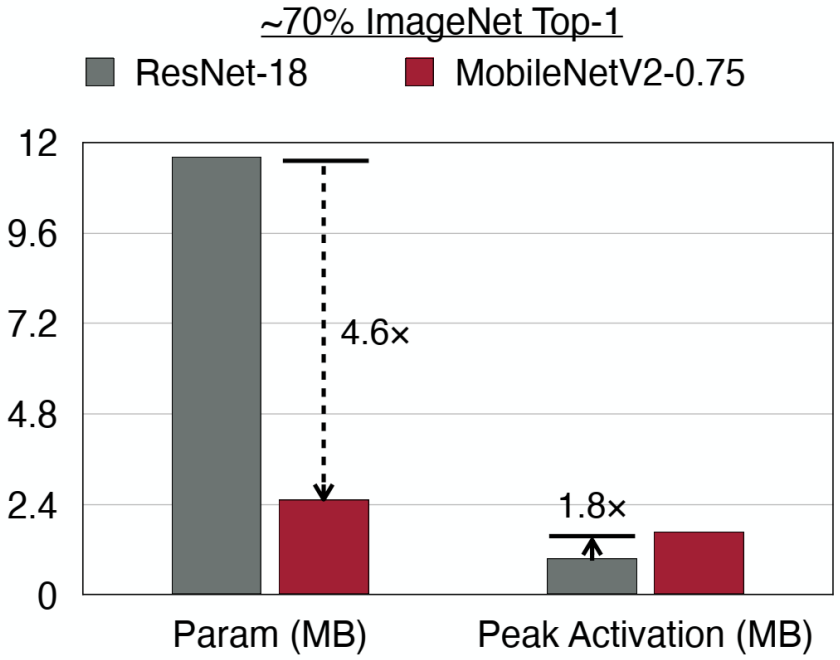
Parameters, Activations, FLOPs

AlexNet	C x H x W = #Activations		#Parameters (bias is ignored) $c_{out} \times c_{in} \times k^2/g$	MACs (one MAC = two FLOPs) $c_{out} \times c_{in} \times k^2 \times h \times w/g$
Image (3x244x224)	3x244x224	= 150,528		
	150,528+290,400=440,928 (peak)			
11×11 Conv, channel 96, stride 4, pad 2	96x55x55	= **290,400**	96x3x11x11 = 24,848	96x3x11x11x55x55 = 105,415,200
3×3 MaxPool, stride 2	96x27x27	= 69,984		
5×5 Conv, channel 256, pad 2, groups 2	256x27x27	= **186,624**	256x96x5x5/2 = 307,200	256x96x5x5x27x27/2 = **223,948,800**
3×3 MaxPool, stride 2	256x13x13	= 43,264		
3×3 Conv, channel 384, pad 1	384x13x13	= 64,896	384x256x3x3 = 884,736	384x256x3x3x13x13 = **149,520,384**
3×3 Conv, channel 384, pad 1, groups 2	384x13x13	= 64,896	384x384x3x3/2 = 663,552	384×384×3×3×13×13/2 = 112,140,288
3×3 Conv, channel 256, pad 1, groups 2	256x13x13	= 43,264	256x384x3x3/2 = 442,368	256×384×3×3×13×13/2 = 74,760,192
3×3 MaxPool, stride 2	256x6x6	= 9,216	$c_{out} \times c_{in}\ (k = 1)$	$c_{out} \times c_{in}\ (k, h, w = 1)$
Linear, channel 4096	4096	= 4,096	4096x(256x6x6) = **37,748,736**	4096x(256x6x6) = 37,748,736
Linear, channel 4096	4096	= 4,096	4096x4096 = **16,777,216**	4096x4096 = 16,777,216
Linear, channel 1000	1000	= 1,000	1000x4096 = **4,096,000**	1000x4096 = 4,096,000
	Total 932,264 = 3.7MB (32bits)		**Total 61M = 224MB (32bits)**	**Total 724M = 1.4G FLOPs**

From https://efficientml.ai

Memory





MobileNetV2 reduces model size but not peak memory.

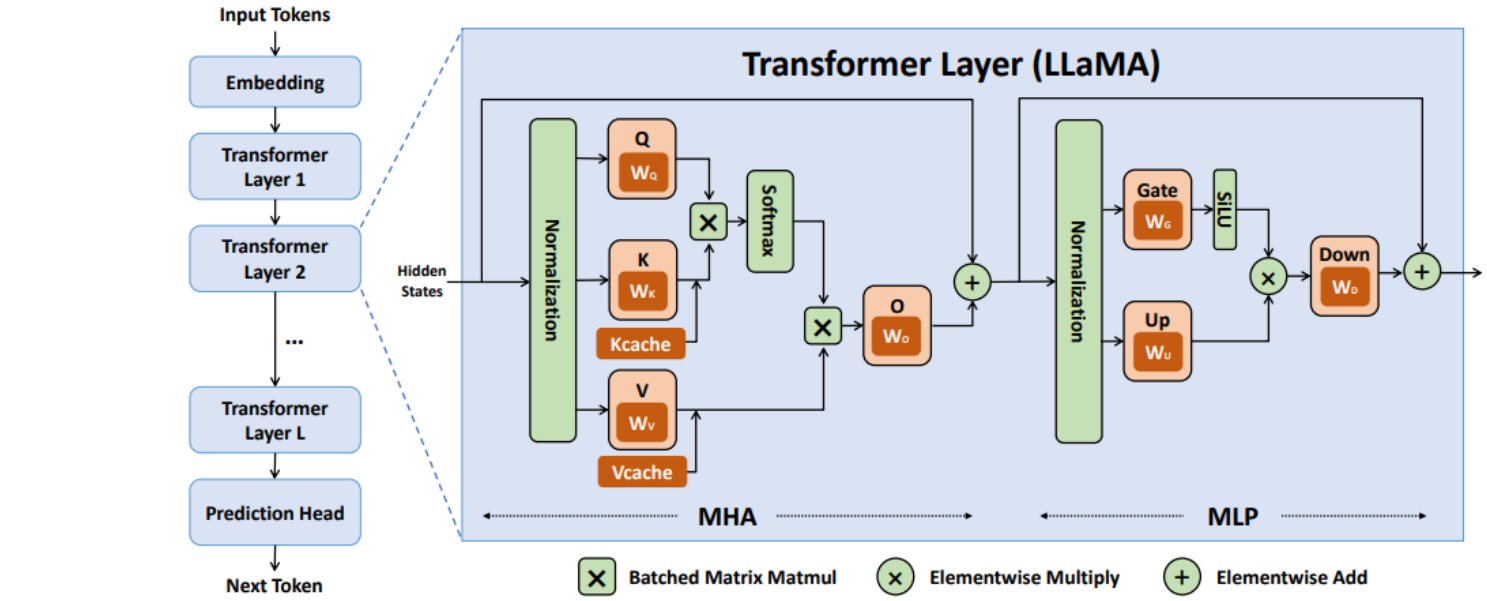
*** All parameters and activations are Integer numbers (8 bits).**

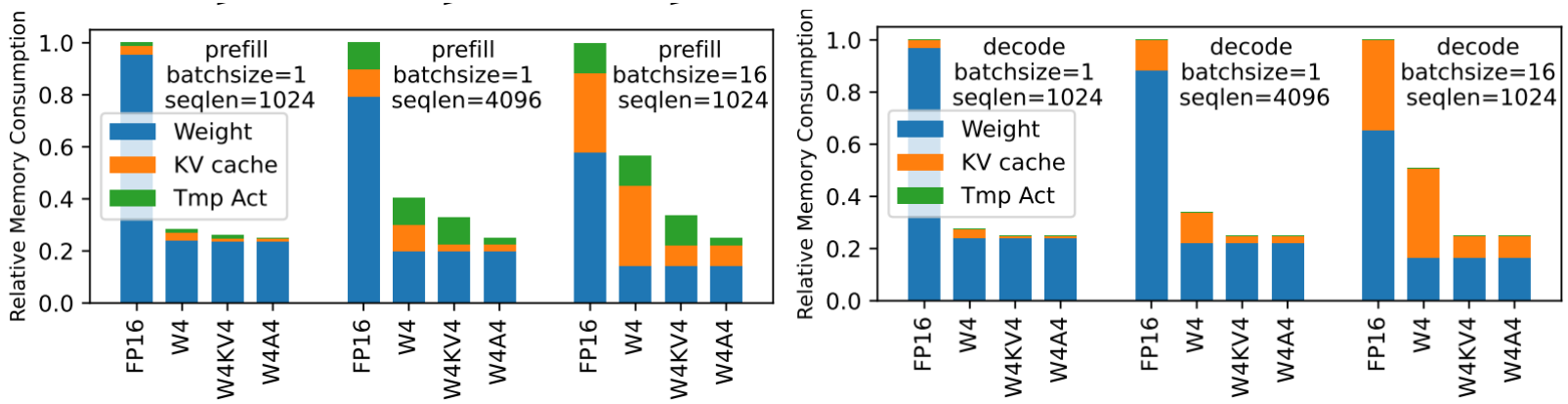
From https://efficientml.ai

Memory - LLM Inference

Table 1. Analysis for layers in Llama-2-7b using the Roofline model of Nvidia A6000 GPU. In this example, the sequence length is 2048 and the batch size is 1.

Layer Name	OPs	Memory Access	Arithmetic Intensity	Max Performance	Bound
			Prefill		
q_proj	69G	67M	1024	155T	compute
k_proj	69G	67M	1024	155T	compute
v_proj	69G	67M	1024	155T	compute
o_proj	69G	67M	1024	155T	compute
gate_proj	185G	152M	1215	155T	compute
up_proj	185G	152M	1215	155T	compute
down_proj	185G	152M	1215	155T	compute
qk_matmul	34G	302M	114	87T	memory
sv_matmul	34G	302M	114	87T	memory
softmax	671M	537M	1.25	960G	memory
norm	59M	34M	1.75	1T	memory
add	8M	34M	0.25	192G	memory
			Decode		
q_proj	34M	34M	1	768G	memory
k_proj	34M	34M	1	768G	memory
v_proj	34M	34M	1	768G	memory
o_proj	34M	34M	1	768G	memory
gate_proj	90M	90M	1	768G	memory
up_proj	90M	90M	1	768G	memory
down_proj	90M	90M	1	768G	memory
qk_matmul	17M	17M	0.99	762G	memory
sv_matmul	17M	17M	0.99	762G	memory
softmax	328K	262K	1.25	960G	memory
norm	29K	16K	1.75	1T	memory
add	4K	16K	0.25	192G	memory



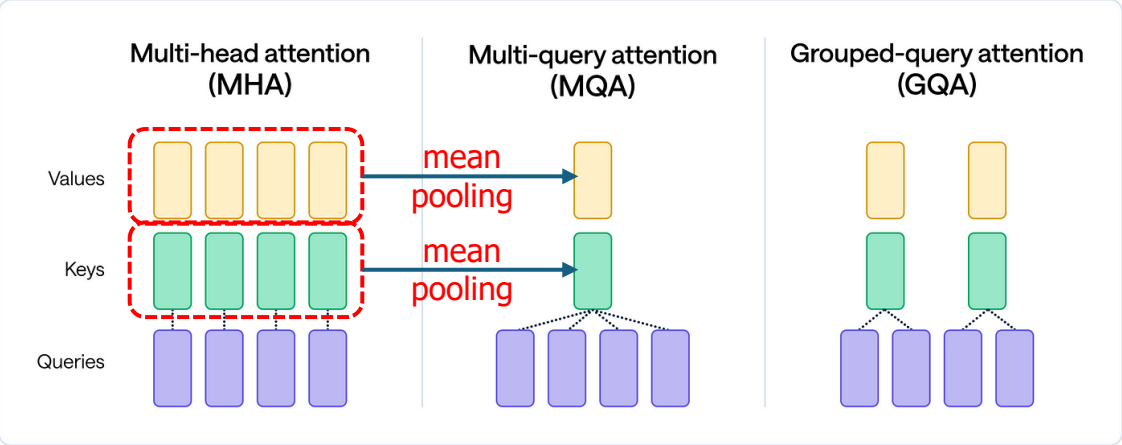


Llama-2-13b

[LLM Inference Unveiled: Survey and Roofline Model Insights [ArXiv 2024]]

Memory - LLM Inference

- Llama2-7B, 16-bit precision (FP16 or BF16)
  - Model weights: 7B×FP16 ~= 14 GB.
  - KV cache: 2×B×S×L×D×P
    - B: Batch Size, S: Sequence Length, L: #Layers, D: Hidden Dimension, P: Precision
  - if batch size = 1, 2×1×4096×32×4096×2B ~ 2 GB (0.5MB/token)
- How to reduce KV cache?
  - Reduce batch size? Low GPU utilization
  - Reduce sequence length (token pruning, sparse attention)? Accuracy drop
  - Reduce layers? Smaller models usually have less layers
  - Reduce attention heads?
    - Multi-query attention (MQA),
    - Grouped-query attention (GQA)
  - Reduce precision? Quantization
- How about memory offloading?



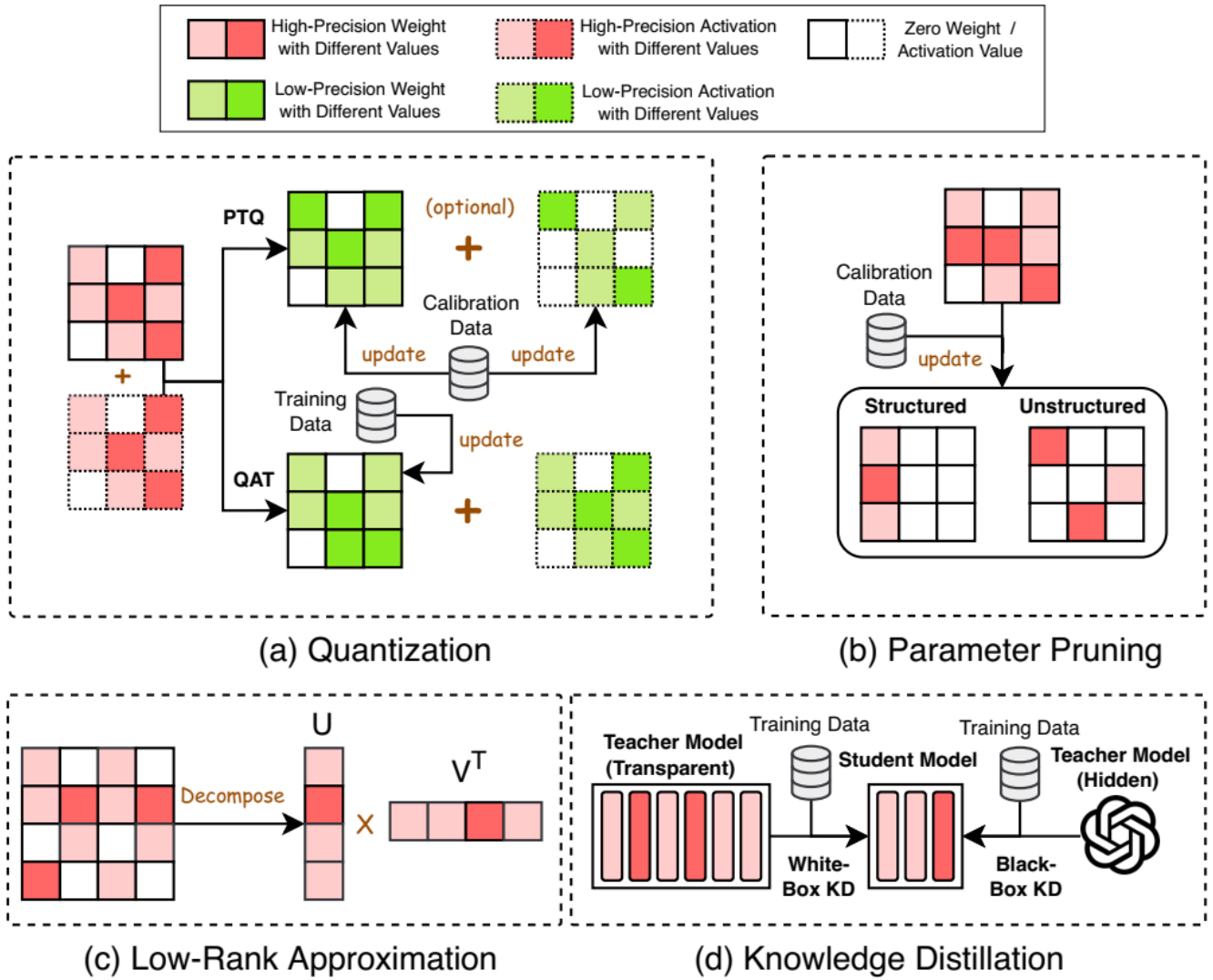
On-Device AI Enablers

Efficient AI Models
Small Models, Model Compression

Edge AI Frameworks
LiteRT, ExecuTorch, ONNX Runtime, Apple Core ML,
Qualcomm's AI Engine,
Llama.cpp, MLC-LLM

Hardware Accelerators
GPU, NPU, PIM, …

Model Compression Techniques



Efficient Large Language Models: A Survey [Wan *et al.*, arXiv 2023]

Small LLMs

- **Google**
  - Gemini Nano (Android14)
  - Gemma: 2B, 7B
    - Gemini's Open Source version
  - Gemma 2: 2B, 9B
- **Meta**
  - Llama3.2
    - Lightweight (1B, 3B)
    - Multimodal (11B, 90B)
  - MobileLLM
    - 125M, 350M for on-device
- **MS Phi-3.5**
  - Phi-3.5-mini-instruct (3.82B)
  - Phi-3.5-vision-instruct (4.15B)
- **Mistral**
  - Ministral 3B, 8B
- **NVIDIA**
  - Minitron 4B
- **Apple**
  - OpenELM 270M, 450M, 1.1B and 3B
- **Stability.ai**
  - Stable LM 2 (1.6B), Stable LM Zephr 3B
- **Alibaba**
  - Qwen2 0.5B, 1.5B
- **EleutherAI**
  - GPT-Neo 125M, 350M, 1.3B, 2.7B

Llama.cpp



- Georgi Gerganov implemented Meta's LLaMa architecture within the versatile and widely-used C/C++ programming languages.
- Broad compatibility (Apple M, Intel AVX512, NVIDIA)
- Performance optimizations inherent to C/C++.
- Integer Quantization (1.5b, 2b, 3b, 4b, 5b, 6b, 8b)
- Support CPU+GPU hybrid inference
- GGUF and GGML: tensor library used by llama.cpp
- Backed by an active open-source community, continuous improvements and collaborative development.
- https://github.com/ggerganov/llama.cpp

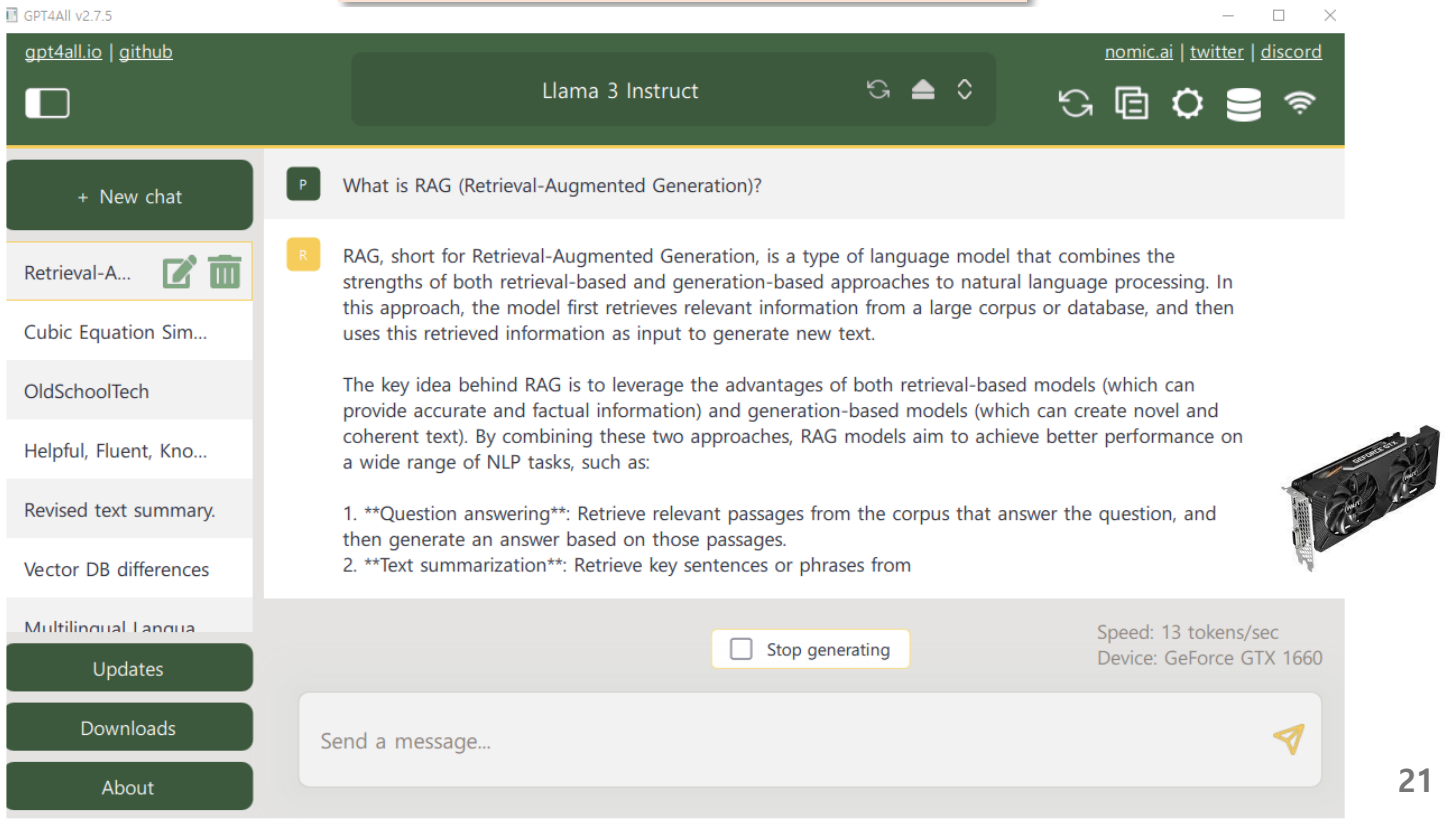
Llama.cpp

Supported models

- LLaMA 1/2/3
- Mistral 7B, Mixtral MoE
- Falcon
- Koala
- Bloom
- Qwen models
- PLaMo-13B
- Phi models
- GPT-2
- Gemma
- Mamba
- LLaVA 1.6 models
- ShareGPT4V

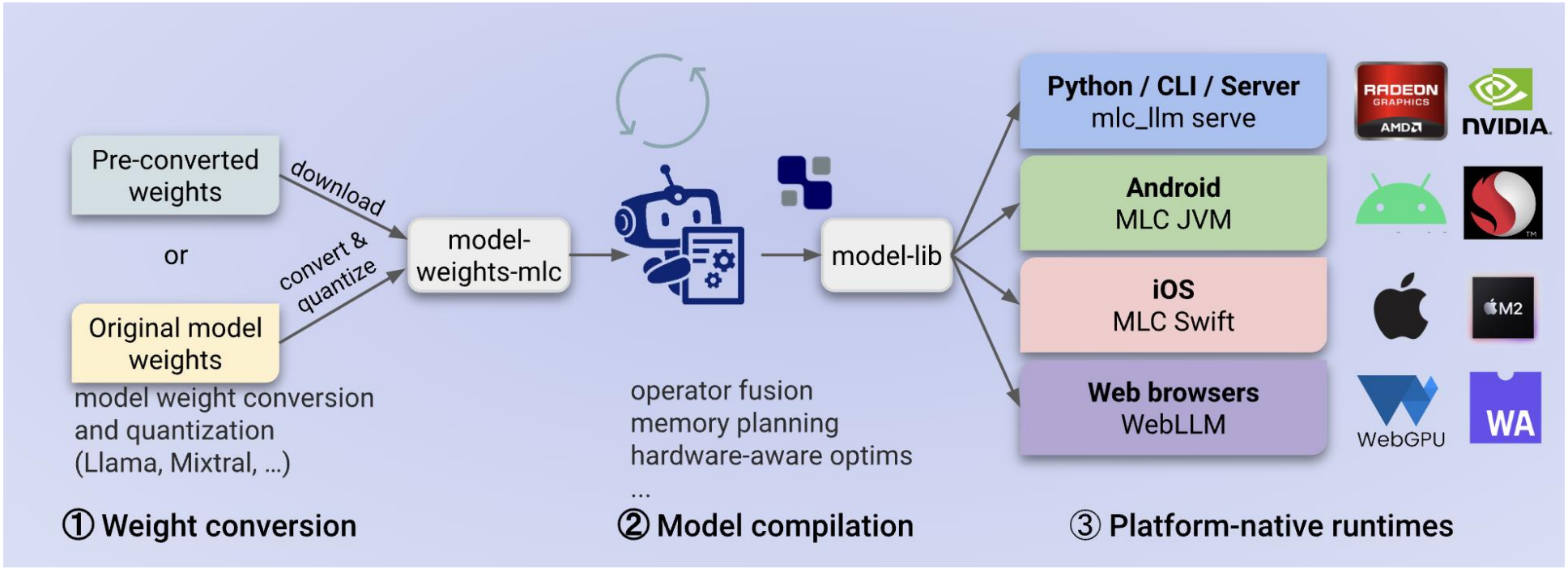
UIs

- Faraday
- LMStudio
- Mozilla-Ocho/llamafile
- nomic-ai/gpt4all
- ollama/ollama
- Msty



MLC-LLM

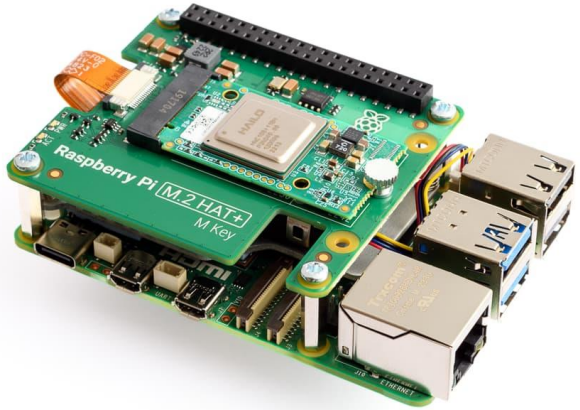
- Machine learning compiler (TVM)
- MLCEngine – a unified high-performance LLM inference engine
- OpenAI-compatible API available through REST server, python, javascript, iOS, Android.

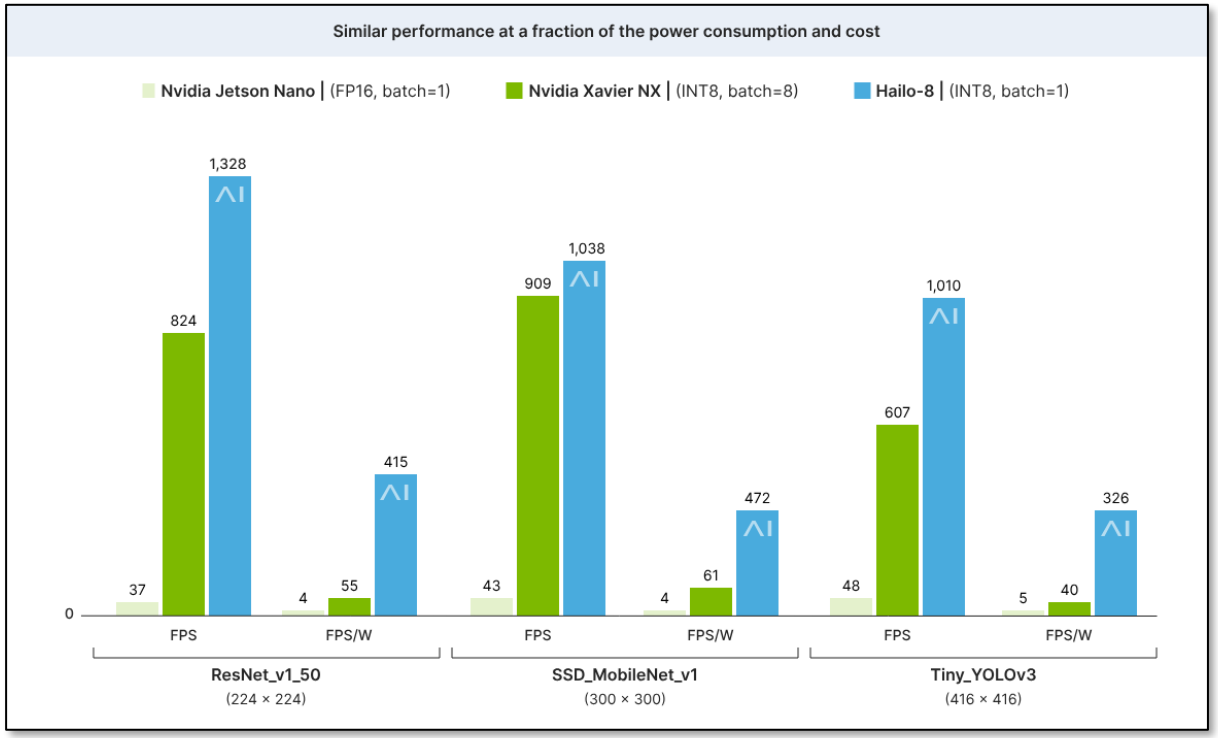


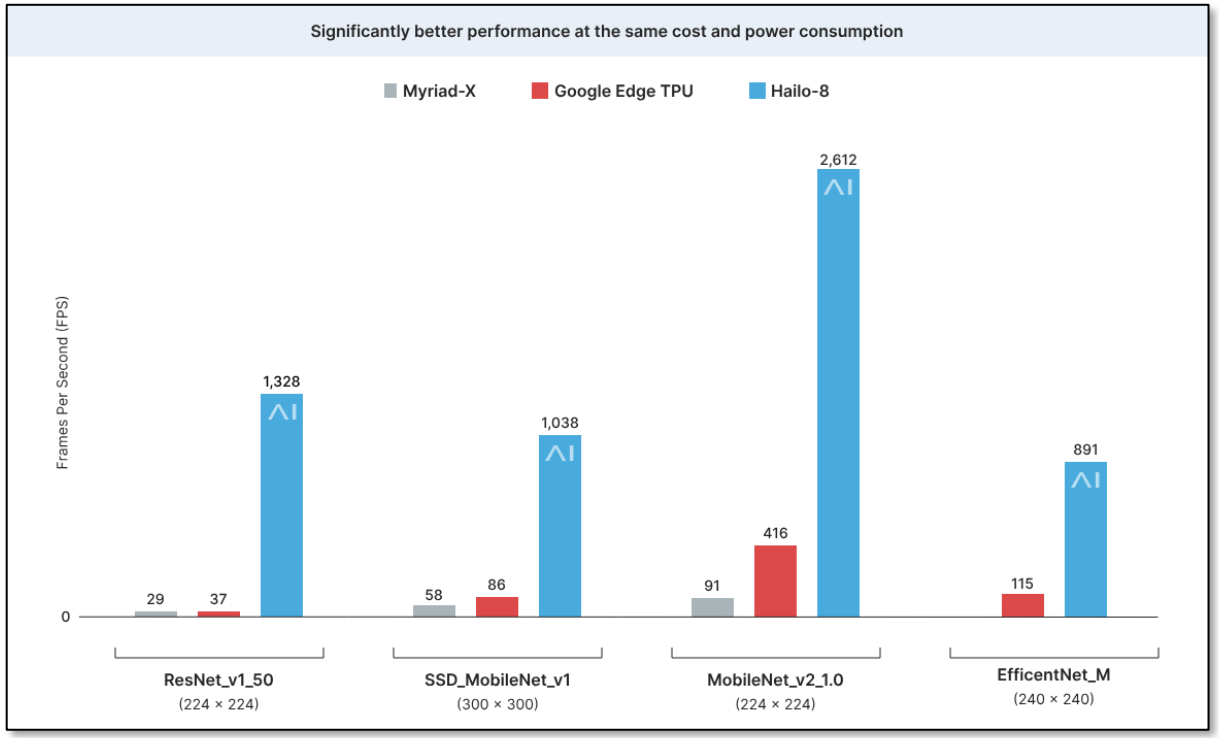
https://llm.mlc.ai/

Hailo-8

- 26 TOPS (2.5W), 13 TOPS (1.5W)

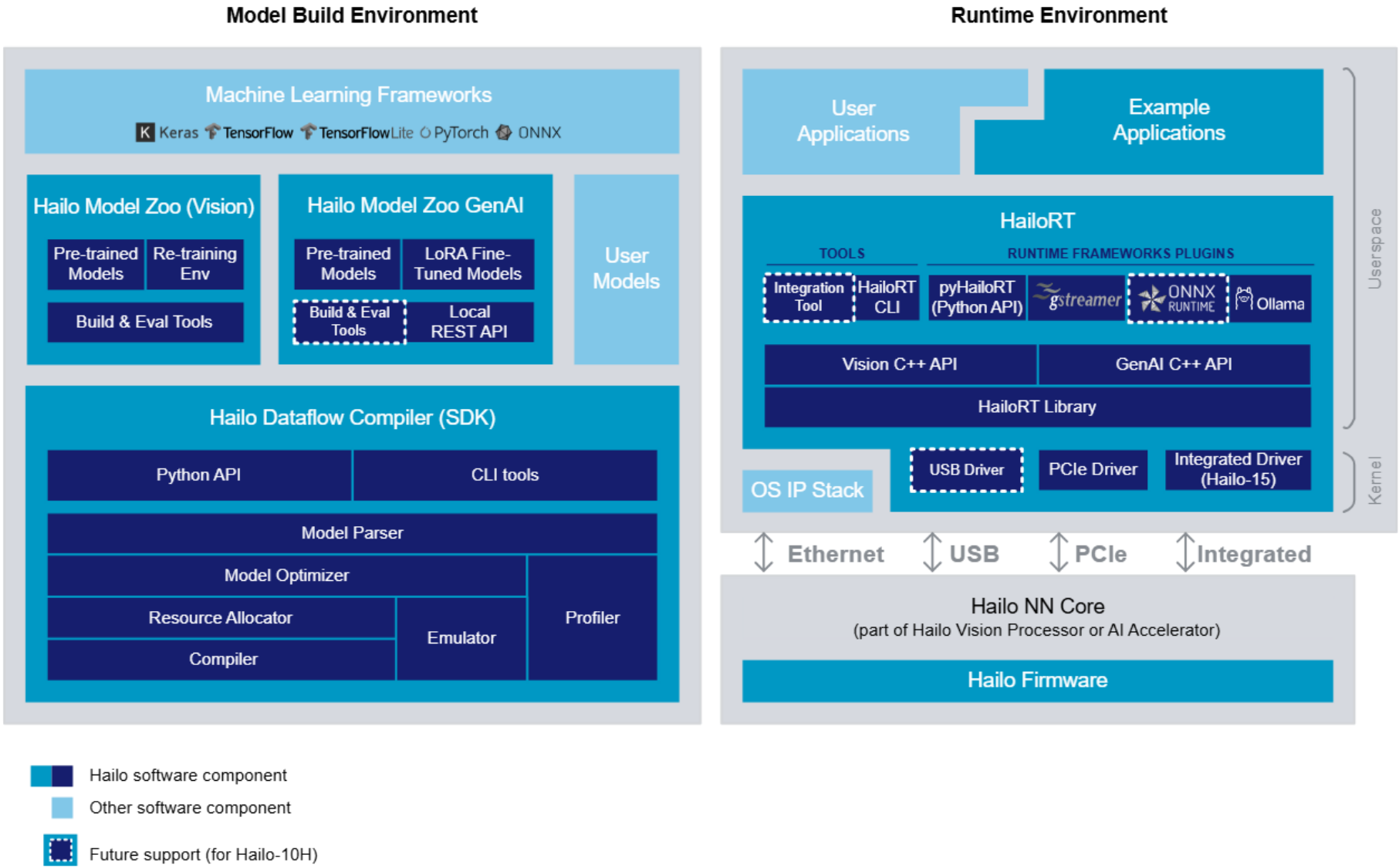






https://hailo.ai/products/ai-accelerators/hailo-8-ai-accelerator/

Hailo Software Stack



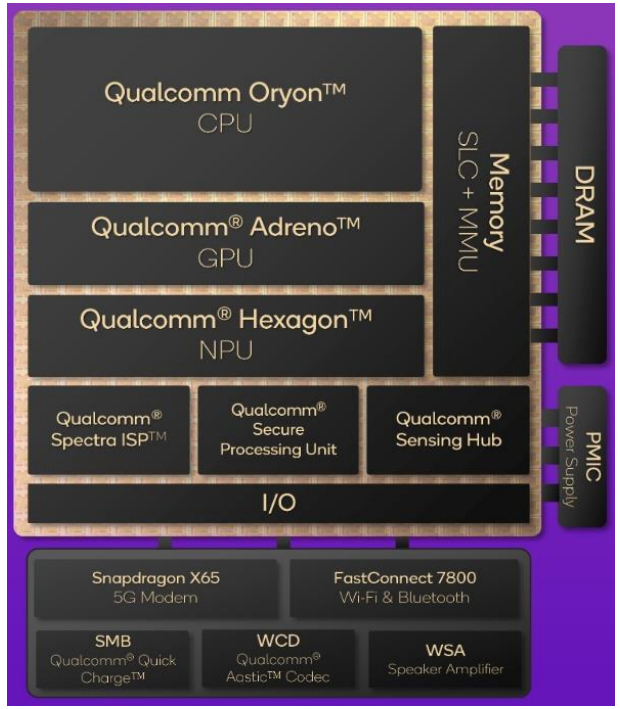
On-Device AI PC

- Qualcomm Snapdragon X
  - Hexagon – 40 TOPS
- Intel Core Ultra
  - Lunar Lake – 45 TOPS NPU
- AMD Ryzen AI
  - Strix Point – 50 TOPS NPU
- Apple
  - M4 chip – 38 TOPS NPU

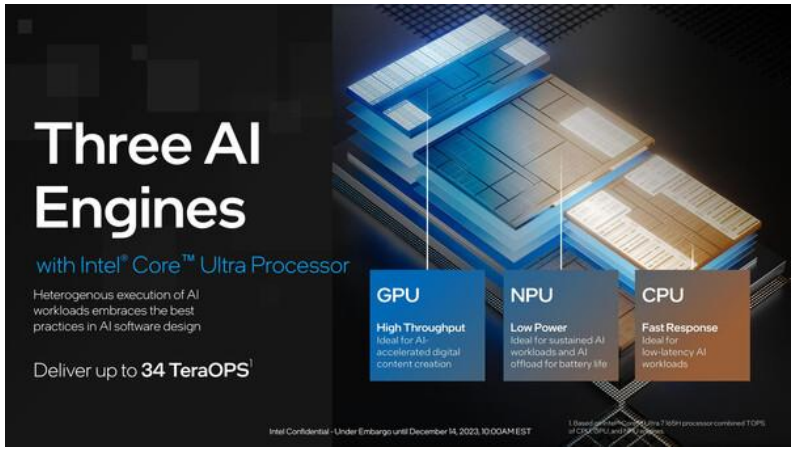
Copilot+PC

Snapdragon X Elite

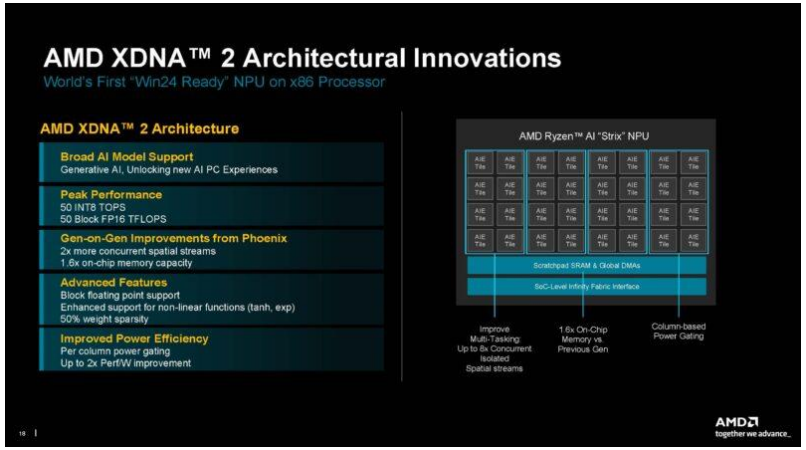
Snapdragon X Plus



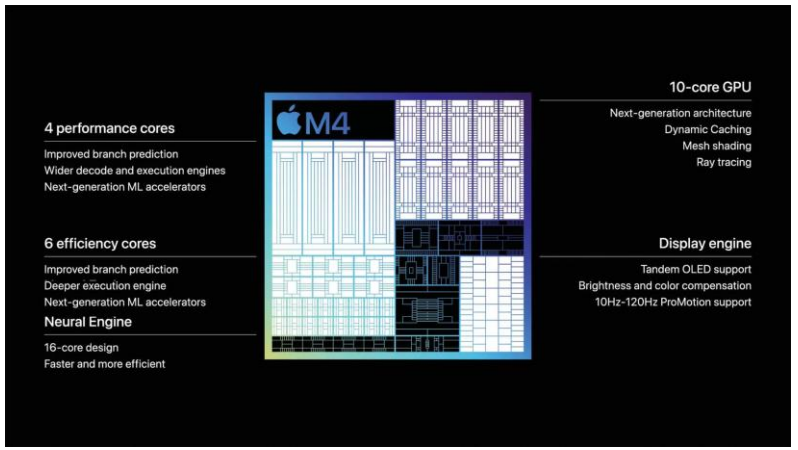
https://www.notebookcheck.biz/Qualcomm-publie-les-tests-officiels-des-Snapdragon-X-Plus-et-Snapdragon-X-Elite-pour-le-NPU-Hexagon-45-TOPS.841823.0.html



https://northgate.com.ph/2024/04/04/the-future-of-the-ai-pc-starts-with-the-intel-core-ultra-processor/

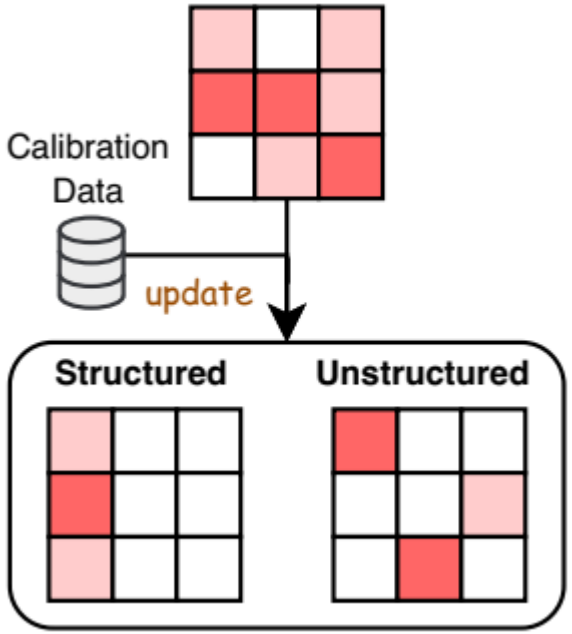


https://www.servethehome.com/more-details-on-the-amd-ryzen-ai-300-series-ryzen-9000-series-and-zen-5/



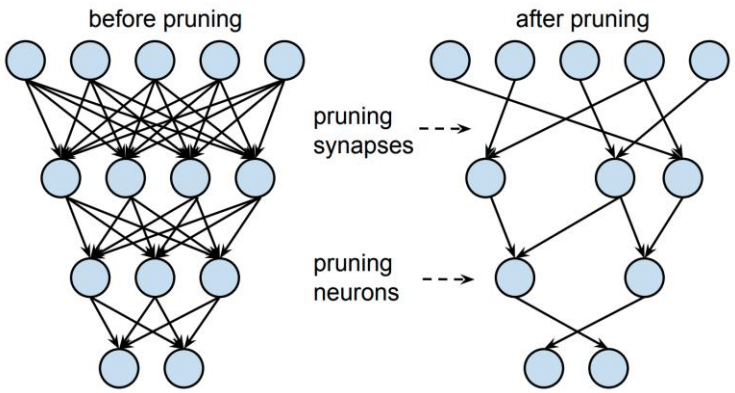
https://www.pickr.com.au/news/2024/apples-m4-brings-more-speed-more-ai-to-devices/

Pruning



Network Pruning

- Network Redundancy
  - For easy training, DNNs are usually over-parameterized
  - Weights can be removed (i.e., set to zero)
- Optimal Brain Damage (LeCun, 1989)
  - Compute the impact of each weight on the training loss (weight saliency)
  - Low saliency weights were removed and the remaining weights were fine tuned
- Magnitude-based pruning (2015)
  - Too difficult to compute saliency for the large-scaled DNNs
  - Simply based on the magnitude of the weights
  - Small weights were pruned and the model was fine tuned to restore the accuracy
  - Use L1/L2 regularization to penalize non-zero parameters resulting in more parameters near zero



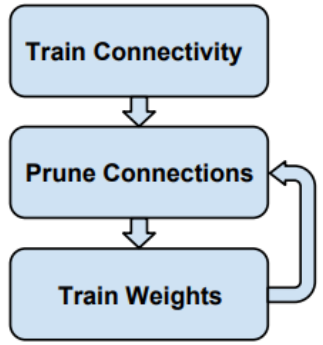
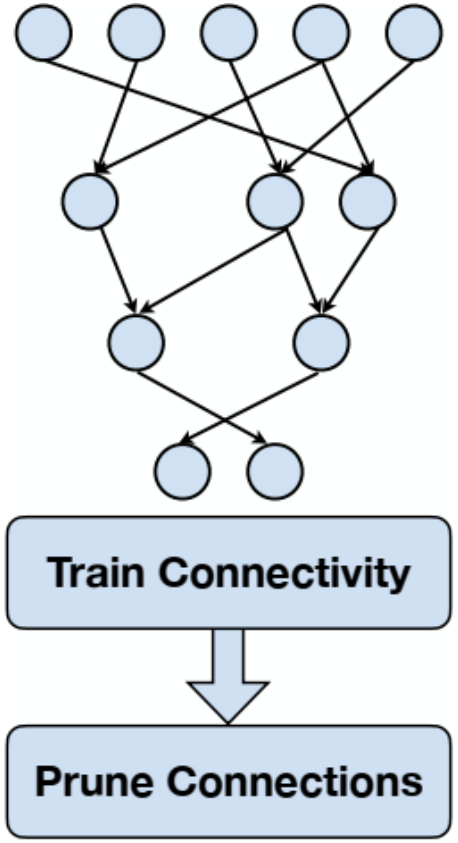


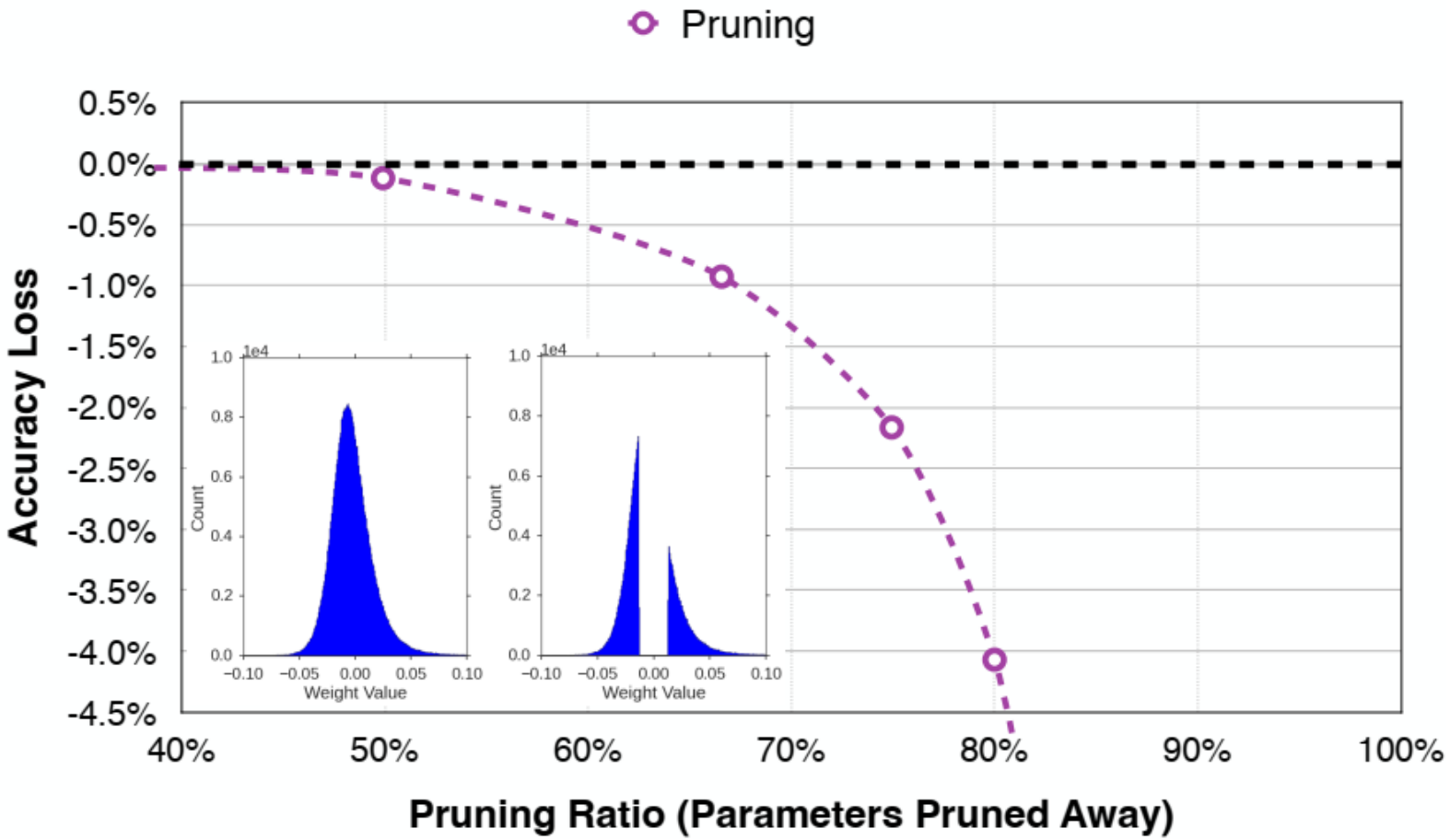
Figure 2: Three-Step Training Pipeline.

Learning both Weights and Connections for Efficient Neural Networks (NIPS'15)

Network Pruning

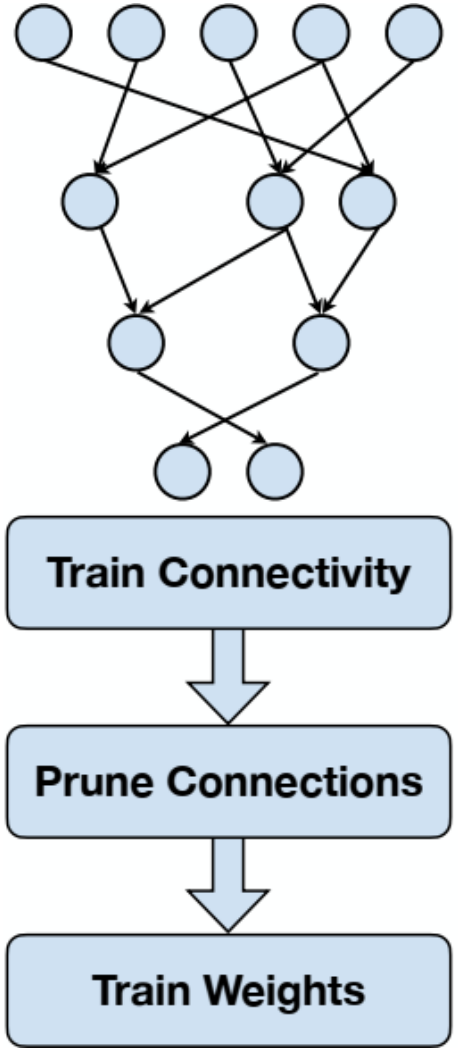
From https://efficientml.ai

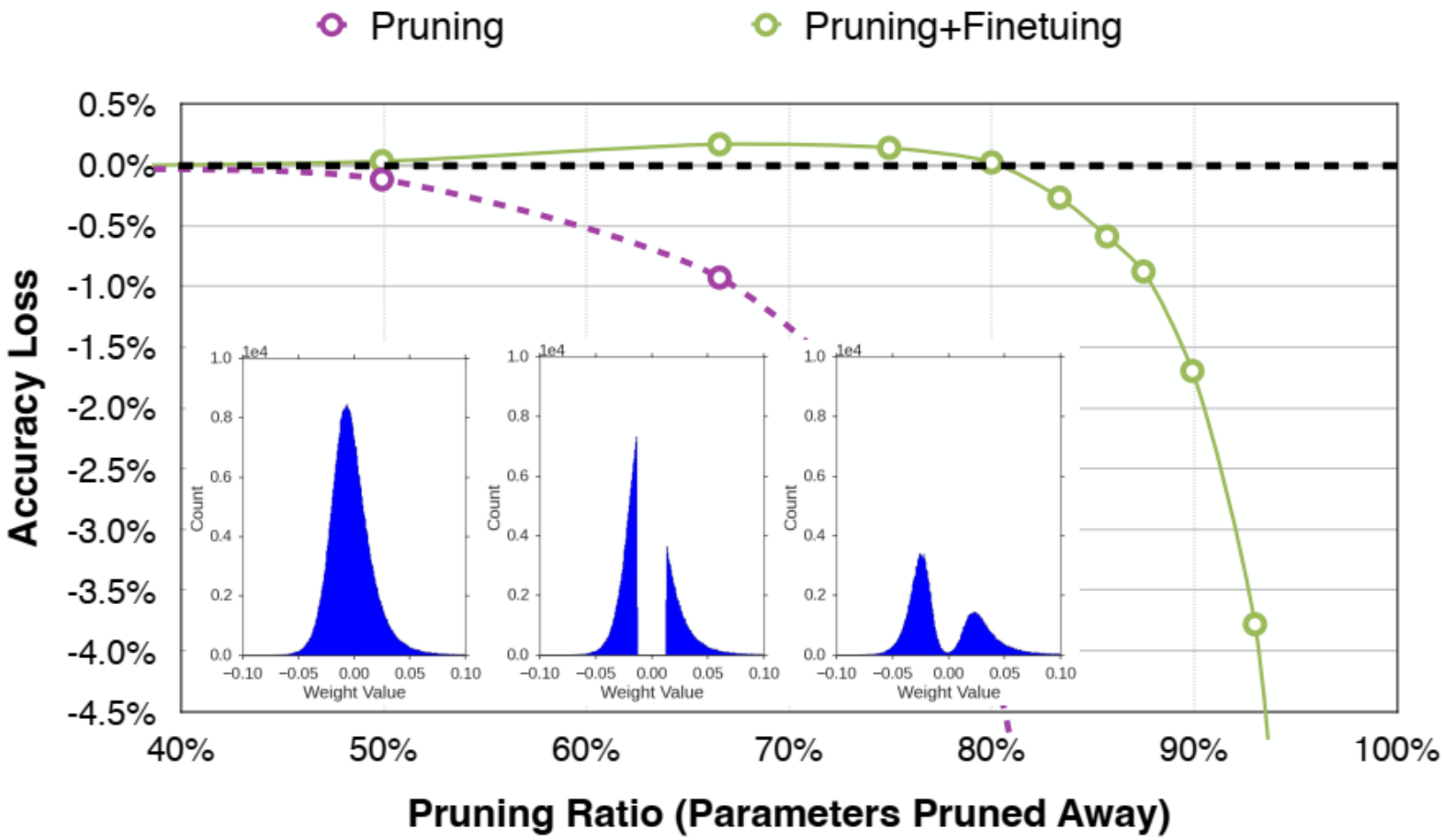




Network Pruning

From https://efficientml.ai



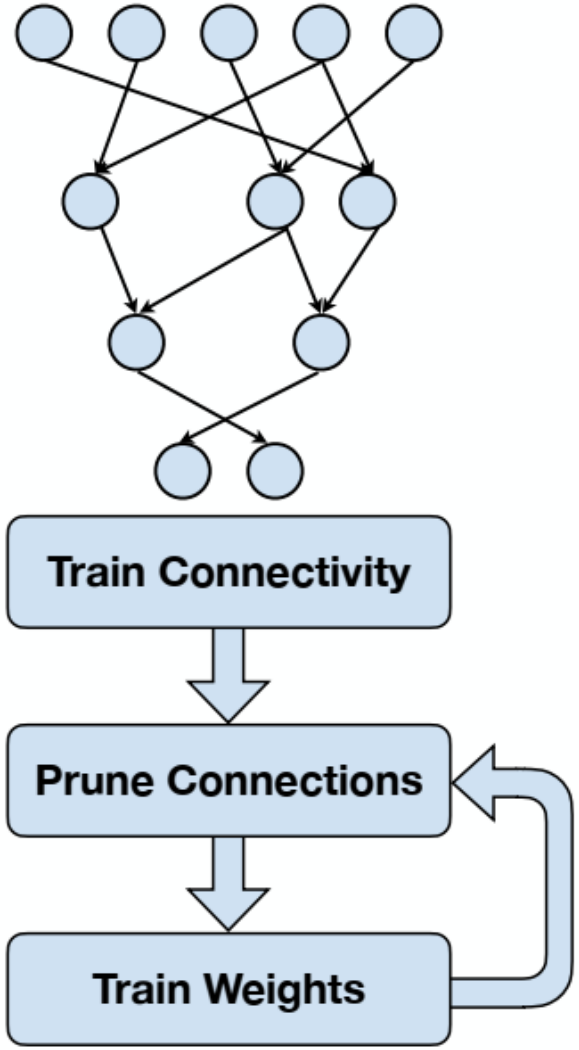


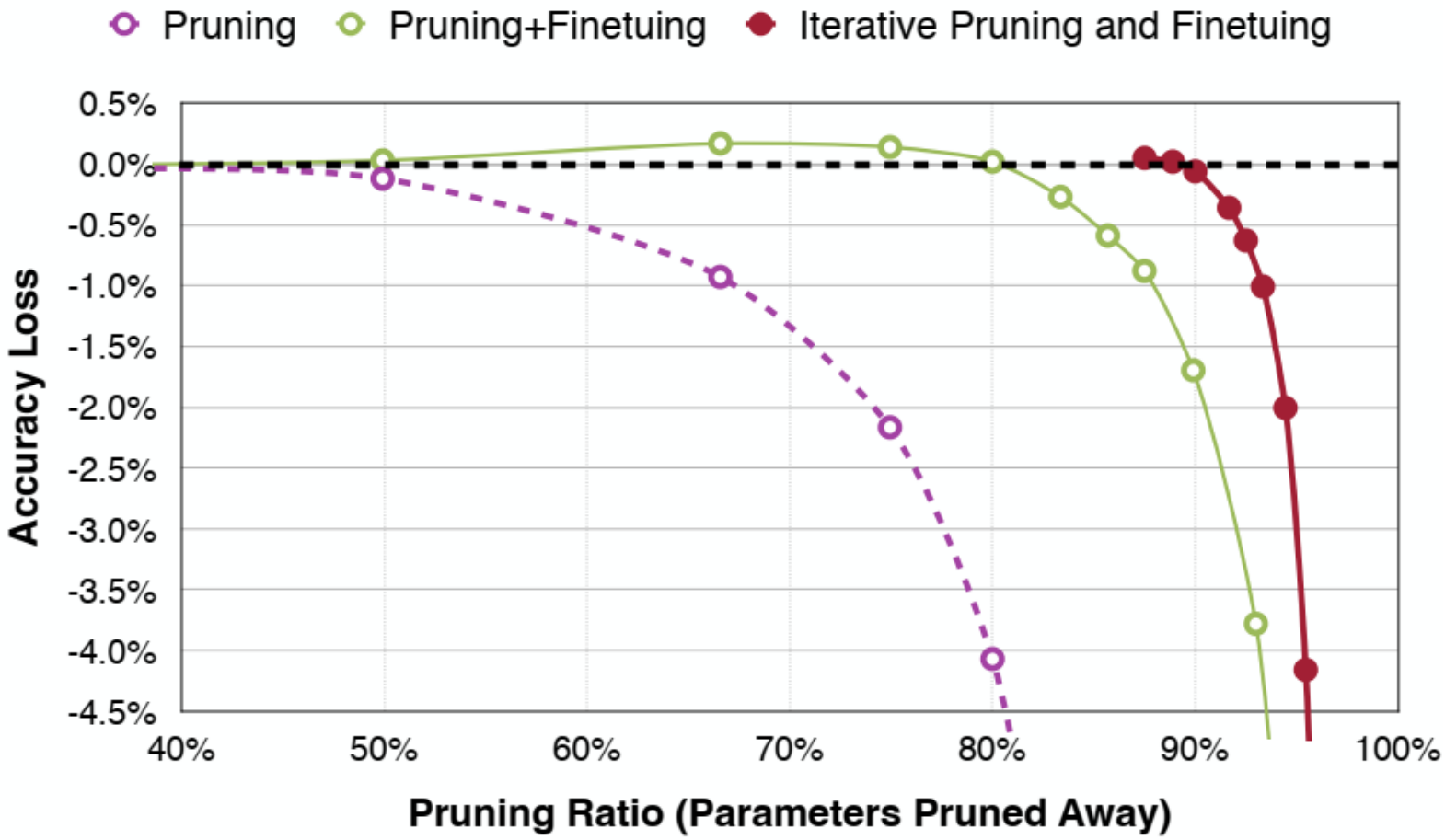
After pruning, the model may decrease, especially for larger pruning ratio.

- Fine-tuning can recover the accuracy and push the pruning ratio higher.
- Learning rate for fine-tuning is usually 1/100 or 1/10 of the original LR.

Network Pruning

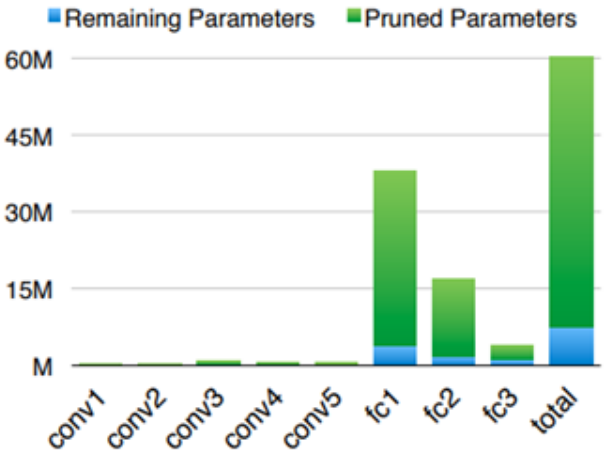
From https://efficientml.ai





Iterative pruning gradually increases the target sparsity in each iteration.

Reduced OPs and Model Size



Most of the weight reduction comes from FC layers

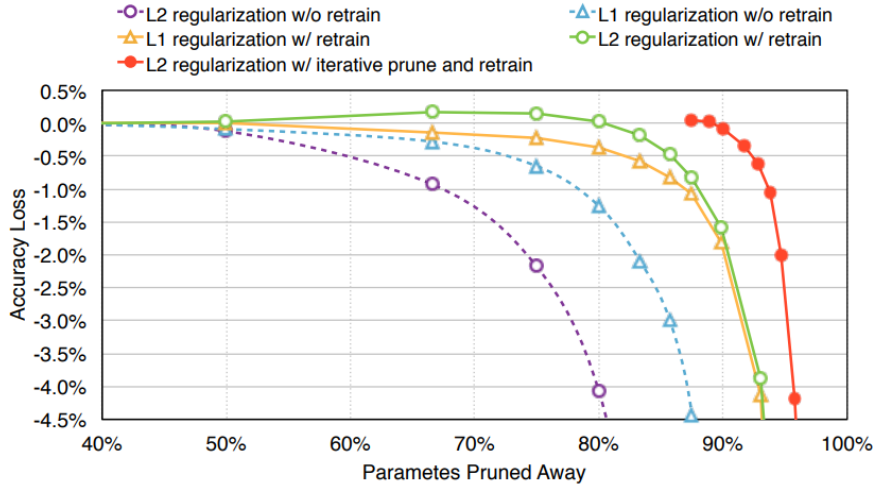
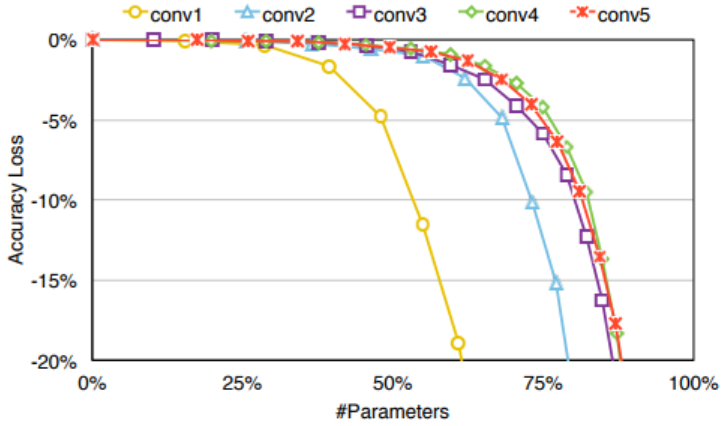
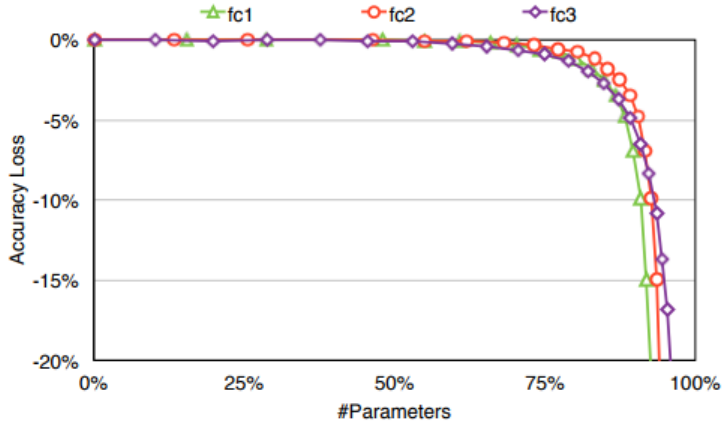


Figure 5: Trade-off curve for parameter reduction and loss in top-5 accuracy. L1 regularization performs better than L2 at learning the connections without retraining, while L2 regularization performs better than L1 at retraining. Iterative pruning gives the best result.

Conv layers are more sensitive to pruning.





FC layers have more redundancy.

Figure 6: Pruning sensitivity for CONV layer (left) and FC layer (right) of AlexNet.

Learning both Weights and Connections for Efficient Neural Networks (NIPS'15)

Reduced OPs and Model Size

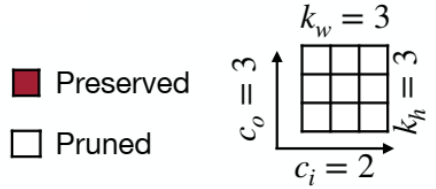
Network	Top-1 Error	Top-5 Error	Parameters	Compression Rate
LeNet-300-100 Ref	1.64%	-	267K	
LeNet-300-100 Pruned	1.59%	-	**22K**	**12**×
LeNet-5 Ref	0.80%	-	431K	
LeNet-5 Pruned	0.77%	-	**36K**	**12**×
AlexNet Ref	42.78%	19.73%	61M	
AlexNet Pruned	42.77%	19.67%	**6.7M**	**9**×
VGG-16 Ref	31.50%	11.32%	138M	
VGG-16 Pruned	31.34%	10.88%	**10.3M**	**13**×

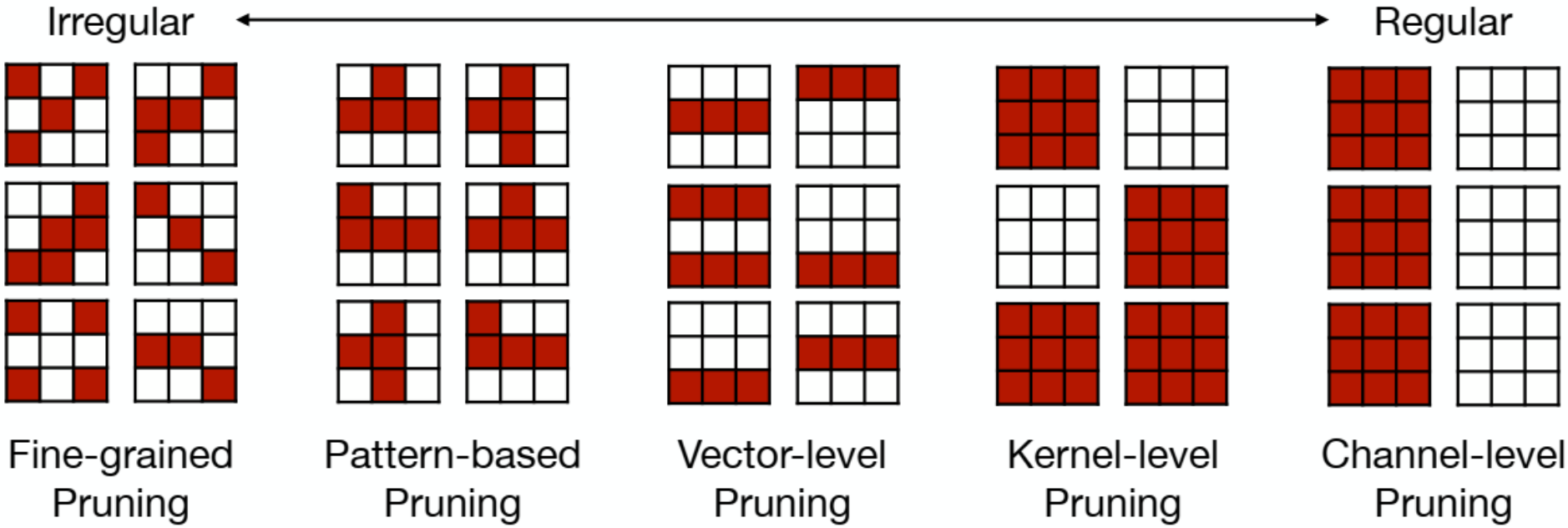
Learning both Weights and Connections for Efficient Neural Networks (NIPS'15)

Design Choice of Pruning

- Pruning Granularity/Pattern
- Pruning Criterion
  - What synapses/neurons should we prune?
- Pruning Ratio
- Fine-tuning after Pruning or Training Pruned Neural Network

Pruning Granularities





Fine-grained/Unstructured

- More flexible pruning index choice
  - Higher Accuracy
- Hard to accelerate (irregular data expression)

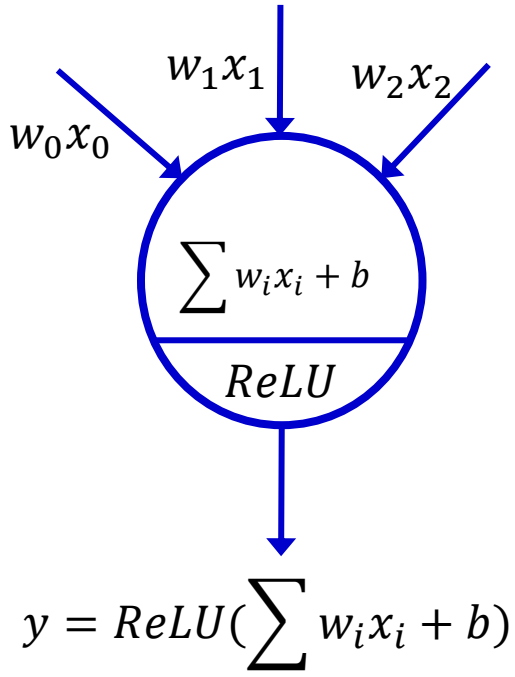
Coarse-grained/Structured

- Less flexible pruning index choice
- Easy to accelerate (just a smaller matrix!)

From https://efficientml.ai

Parameter Selection

- When removing parameters from a neural network model,
  - the less important the parameters being removed are,
  - the better the performance of pruned neural network is

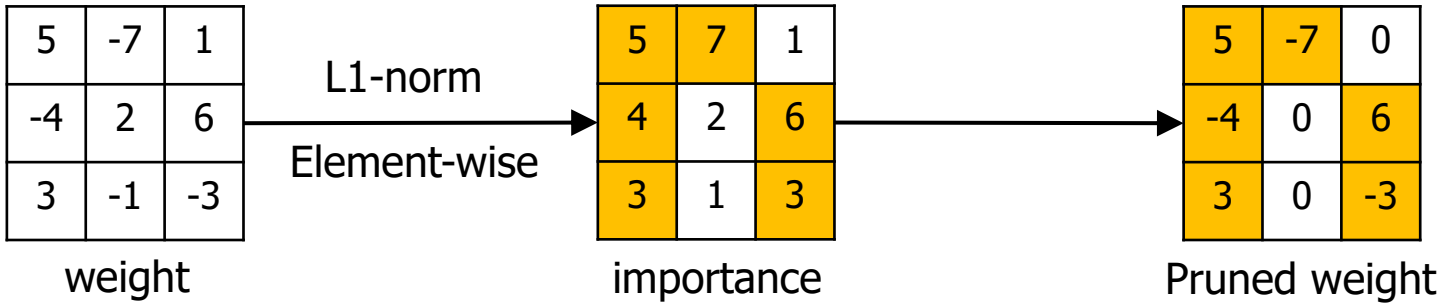


If W = [15, 0.2, -9], $y = ReLU(15x_0 + 0.2x_1 - 9x_2)$

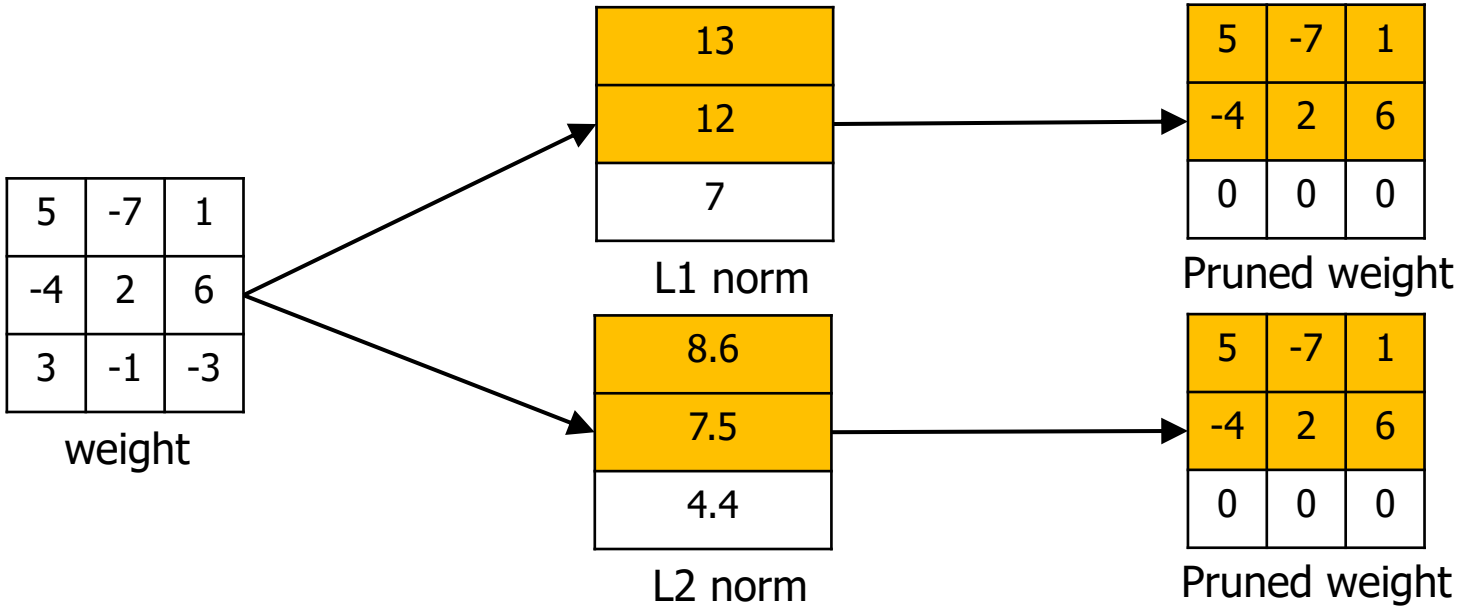
If you want remove one weight parameter, which one?
To remove a weight parameter is to change it to zero.

Magnitude-based Weight Pruning

- Element-wise pruning: importance($W_{i,j}$) = $|W_{i,j}|$



- Row vector pruning and L1/L2-norm magnitude



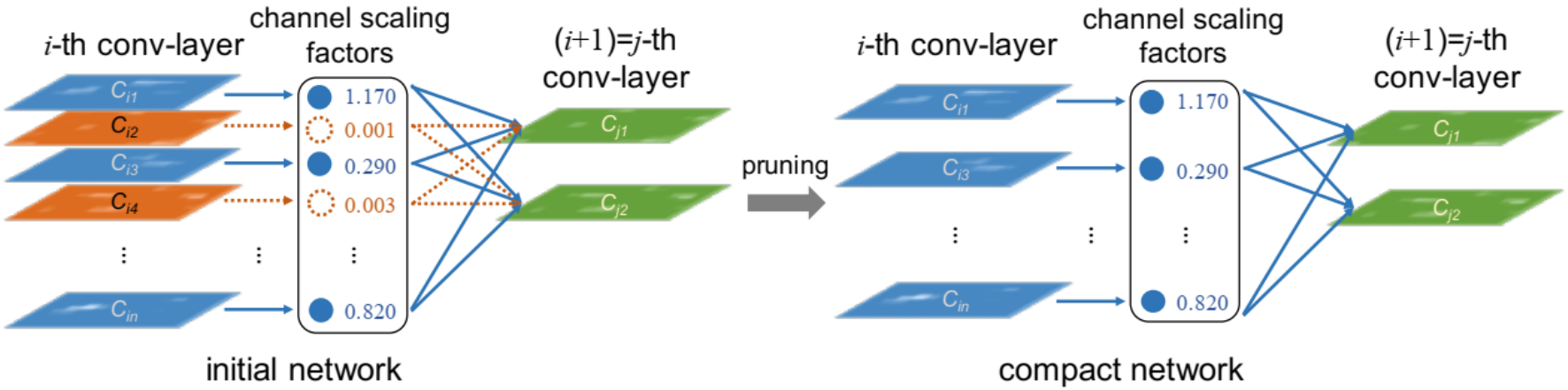
$$\|x\|_2 = \sqrt[2]{\sum_{i=1}^{n} |x_i|^2}$$

Scaling-based Channel Pruning

- BN transformation $\hat{z} = \frac{z_{in} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}; \quad z_{out} = \gamma\hat{z} + \beta$

- γ and β are trainable affine transformation parameters (scale and shift).
- Add an ℓ_1-norm regularization onto the scaling factors of BN layers.
- During training, γ regularization as $L = \sum_{(x,y)} l(f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma)$ $g(s) = |s|$ (L1-norm)

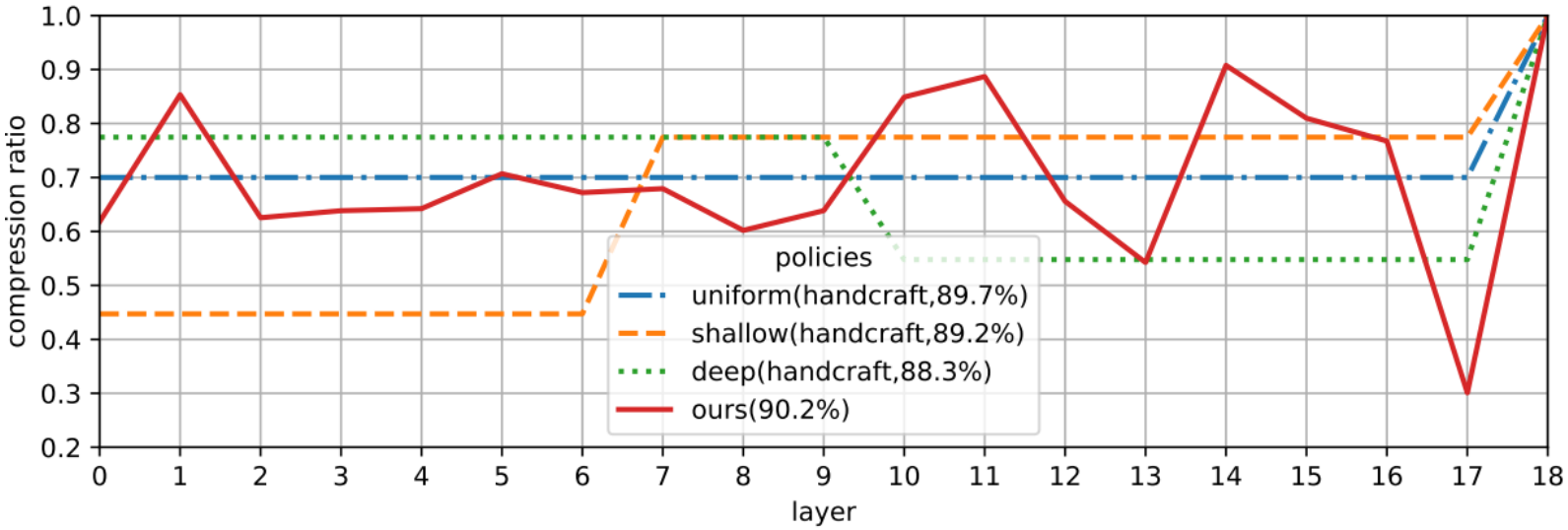
- After training, the output FMs with small γ (resulting small z_{out}) can be pruned, as well as the connected weights.

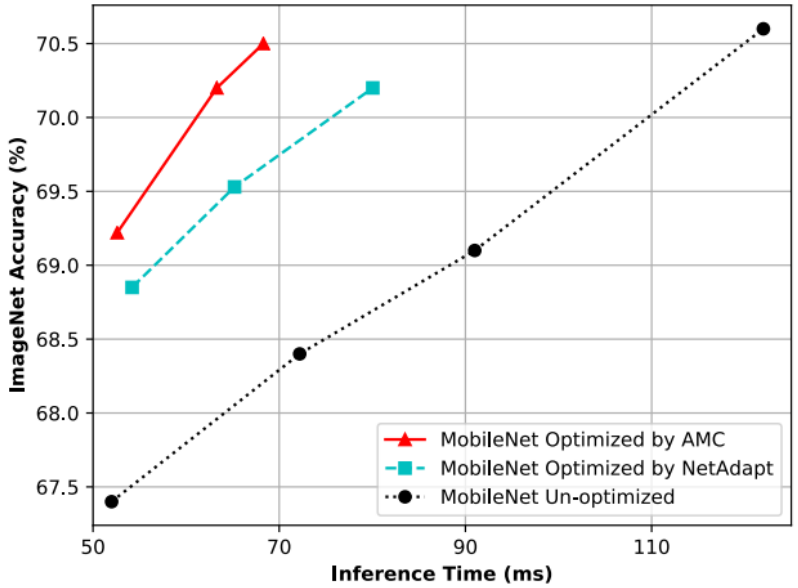


Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Layer-wise Channel Pruning

- Different pruning ratios for each layer since different layers have different sensitivity
  - Some layers are more sensitive (e.g., first layer), some layers are more redundant
- Need to analyze the sensitivity of each layer

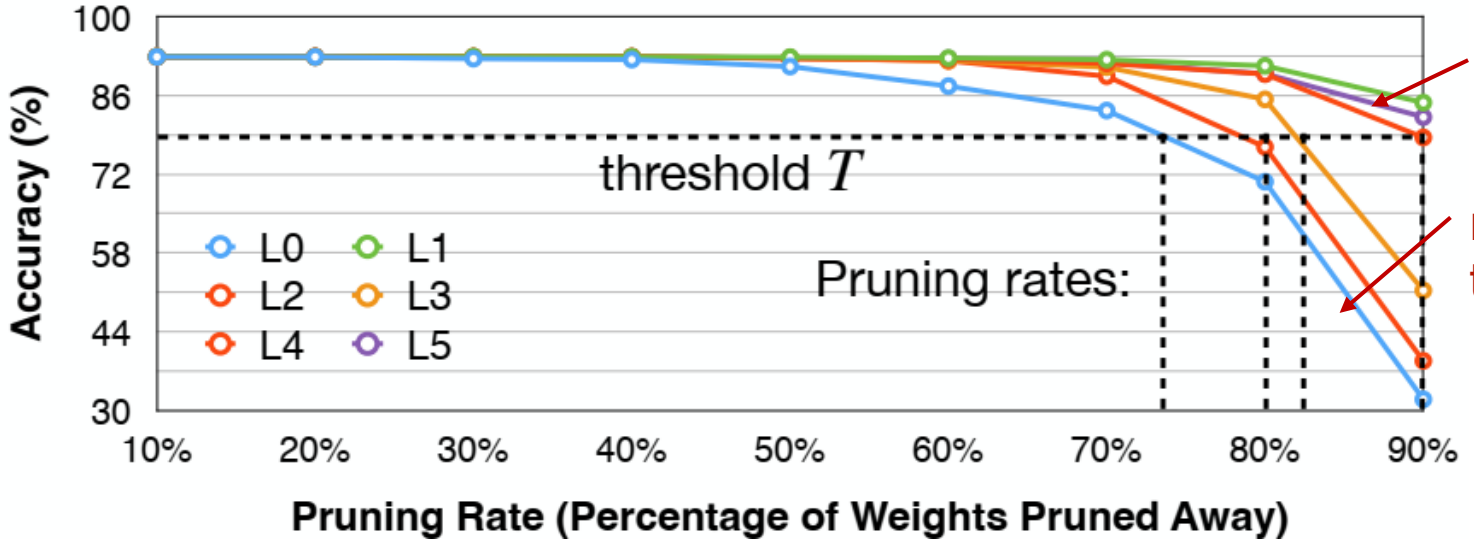




AMC: AutoML for Model Compression and Acceleration on Mobile Devices (ECCV'18)

Finding Pruning Ratios

- Sensitivity Analysis
  - Pick a layer L_i in the model and Prune with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$
  - Observe the accuracy degrade $\Delta acc_{i,r}$ for each pruning ratio
  - Repeat the process for all layers
  - Pick a degradation threshold T such that the overall pruning rate is desired



less sensitive to pruning

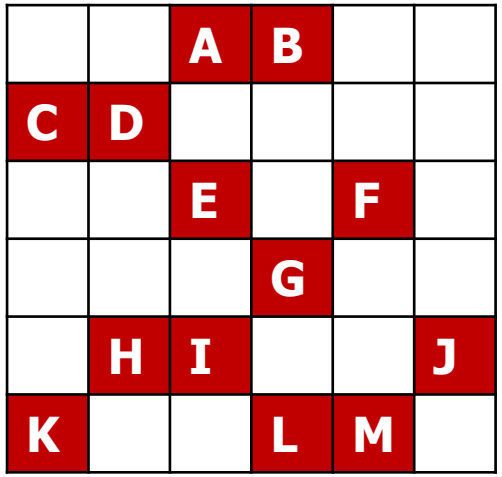
more sensitive to pruning

Is this optimal? Maybe not. We do not consider the interaction between layers.

* VGG-11 on CIFAR-10 dataset

From https://efficientml.ai

CSR Format for Sparse Matrices



0	2	4	6	7	10	13

Row Pointer

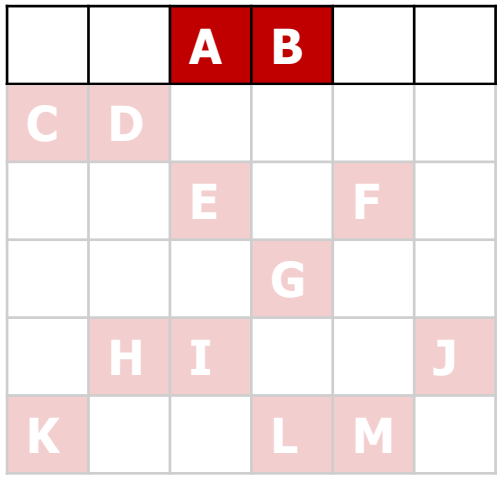
2	3	0	1	2	4	3	…

Column Indices

A	B	C	D	E	F	G	…

Values

CSR Format for Sparse Matrices



Because there is no element before the current row.

0	2	4	6	7	10	13

Row Pointer

2	3	0	1	2	4	3	…

A[0].nonzeros = [2, 3]

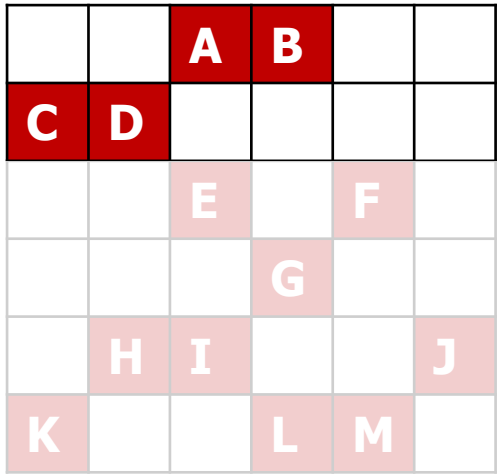
Column Indices

A	B	C	D	E	F	G	…

Nonzero values for row 0

Values

CSR Format for Sparse Matrices



Because there is two element before the current row.

0	2	4	6	7	10	13

Row Pointer

2	3	0	1	2	4	3	…

A[1].nonzeros = [0, 1]

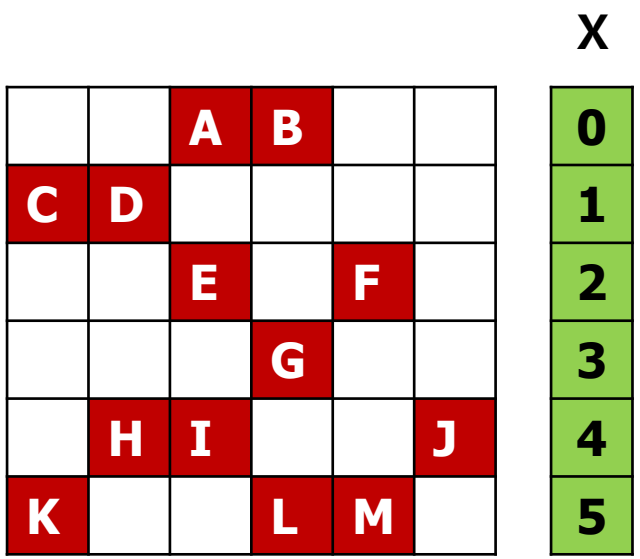
Column Indices

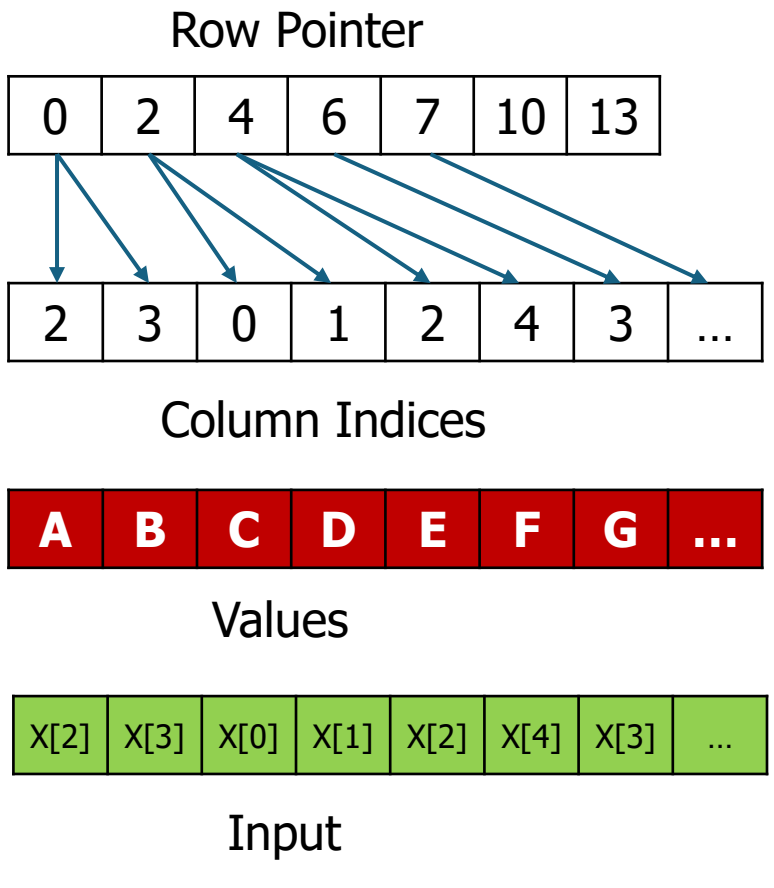
A	B	C	D	E	F	G	…

Nonzero values for row 1

Values

CSR Format for Sparse Matrices



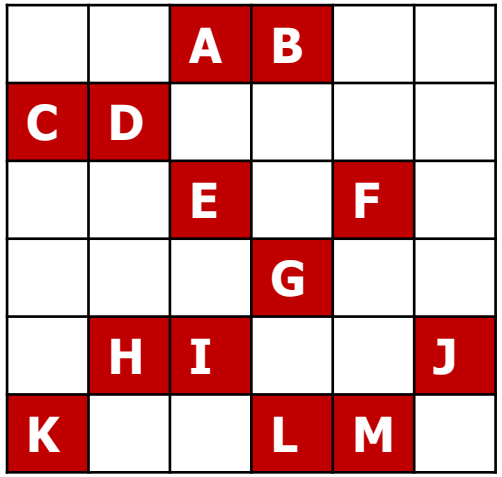


Algorithm 1: Standard CSR SpMV

```
1 for i = 0 to nRows-1 do  // allow parallelization
2 |   sum = 0
3 |   for j = row_pointers[i] to row_pointers[i+1]
  |    do
4 |   |  sum += values[j] * x[col_indices[j]]
5 |   y[i] = sum
```

Bitmap-Based Sparse Matrix-Vector Multiplication with Tensor Cores (ICPP'24)

CSC Format for Sparse Matrices



Column Pointer

0	2	4	7	10	12	13

1	5	1	4	0	2	4	…

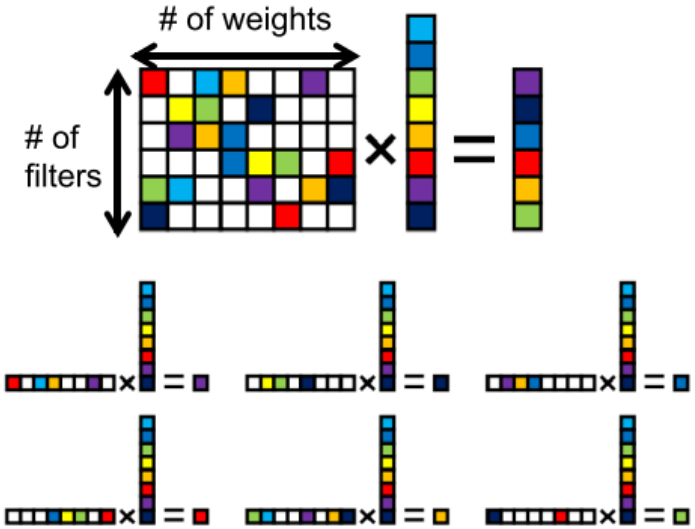
Row Indices

C	K	D	H	A	E	I	…

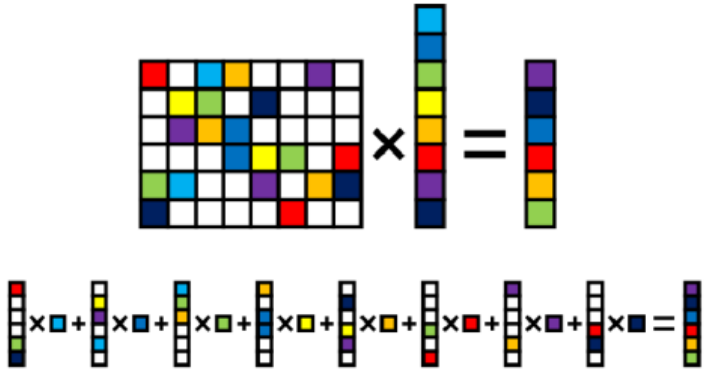
Values

Sparse matrix compressed format

- Compressed sparse row (CSR)
  - Input vector needs to be read in multiple times
  - Only a subset of input vector is used since each row of the matrix is sparse.
- Compressed sparse column (CSC)
  - Output is updated several times
  - Only one element of input vector is read at a time
  - Lower memory bandwidth than CSR
    - if the output is smaller than the input
    - if the number of filters is not significantly larger than the number of weights in the filter



CSR

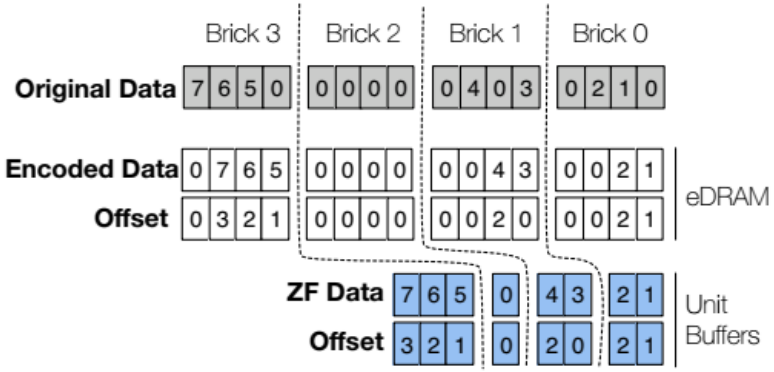


CSC

Efficient Processing of Deep Neural Networks: A Tutorial and Survey [Sze et al. 2017]

Sparse Architecture – Input Sparsity

- ReLU produces abundant zeros
- Simple run-length activation compression
- Zero input detection
  - can be aggressively extended to small inputs
- PE data gating
- Structured activation sparsity such as FM-wise pruning can be used (e.g., Scalpel)



Zero-Free Neuron Array format (ZFNAf)

Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing [ISCA'16]

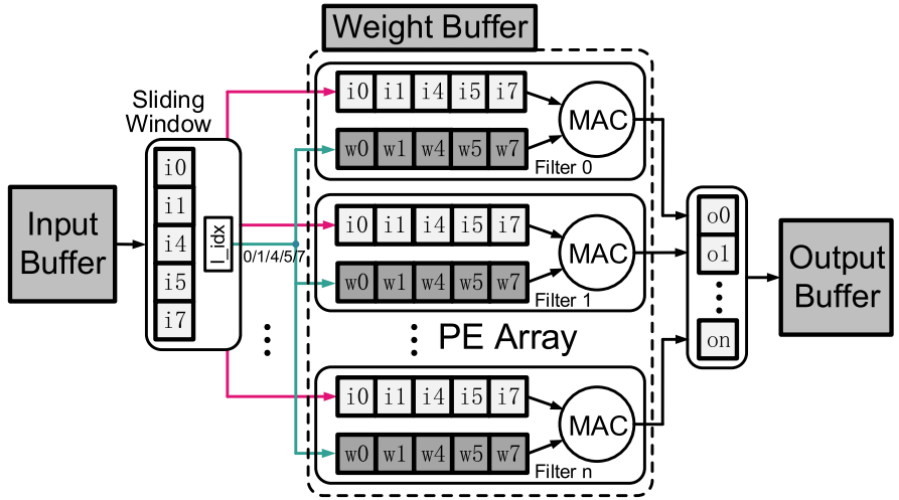


Image from "Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al. 2020]

Sparse Architecture – Irregular Weight Sparsity

- Leverage weight sparsity by naively skipping the MACs with zero weights.
- Weights are stored in a sparse format with addressing indexes.
- Each PE accesses the required activations according to the weight index and then performs the residual MACs.
- Reduced OPs, low latency and energy.
- The distribution of nonzero weights is usually very irregular
  - large indexing overhead, PE imbalance, and memory access inefficiency

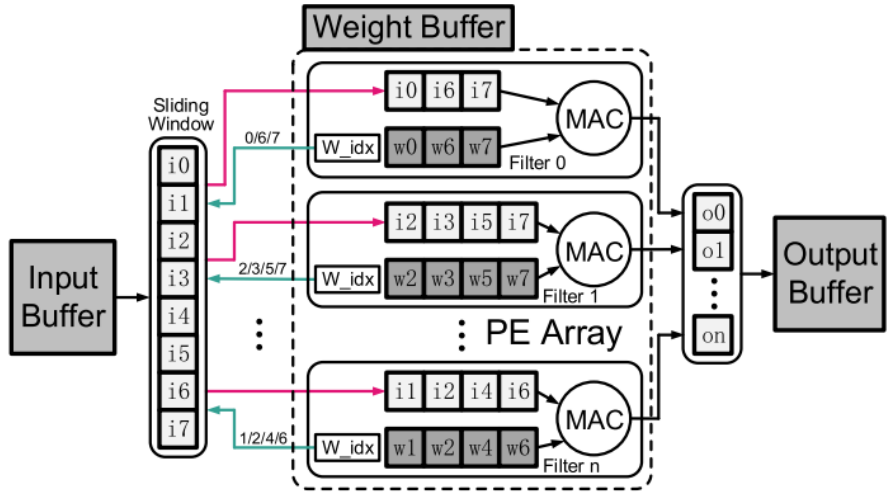
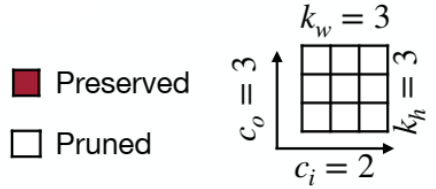
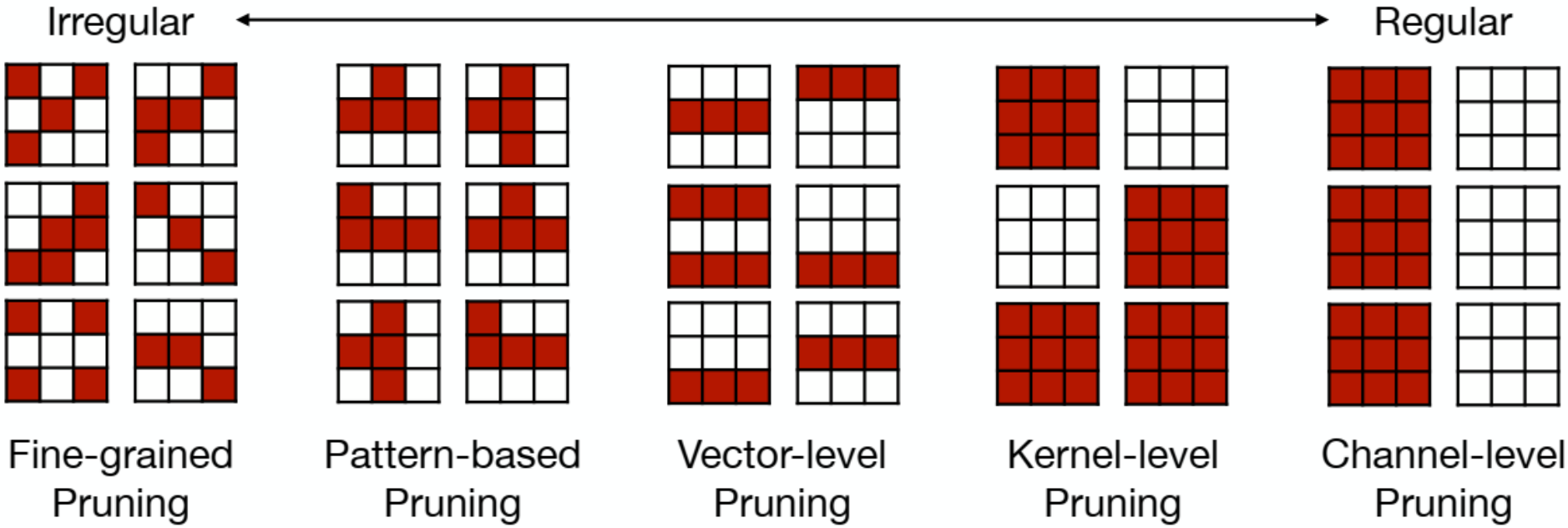


Image from "Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al. 2020]

Cambricon-X: An Accelerator for Sparse Neural Networks [Micro'16]

Pruning Granularities





Fine-grained/Unstructured

- More flexible pruning index choice
  - Higher Accuracy
- Hard to accelerate (irregular data expression)

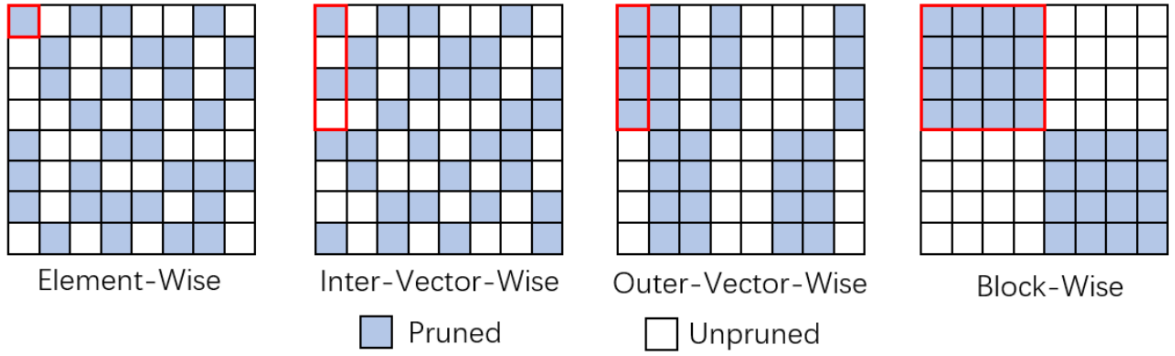
Coarse-grained/Structured

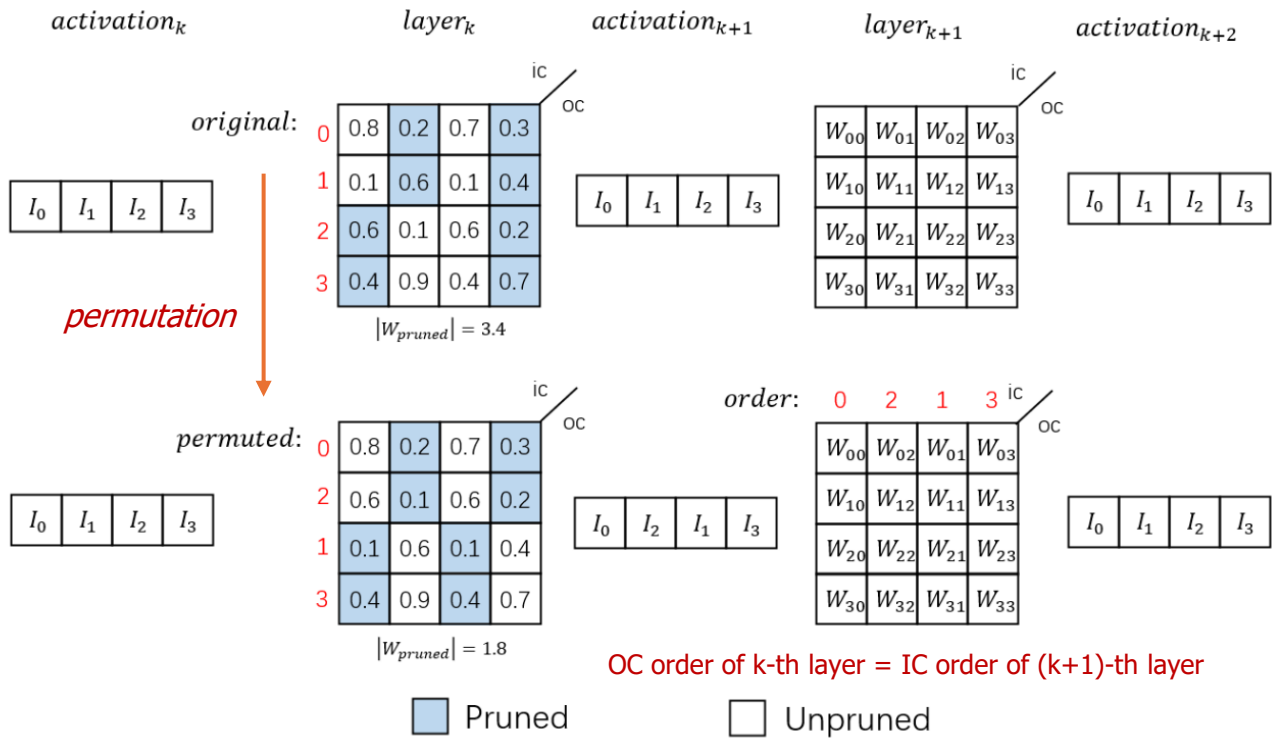
- Less flexible pruning index choice
- Easy to accelerate (just a smaller matrix!)

From https://efficientml.ai

Vector-Wise Sparsity

- IVW prunes a certain proportion of weights inside each vector
- OVW prunes the entire vector of weights
- Channel Permutation (Reordering) will be beneficial

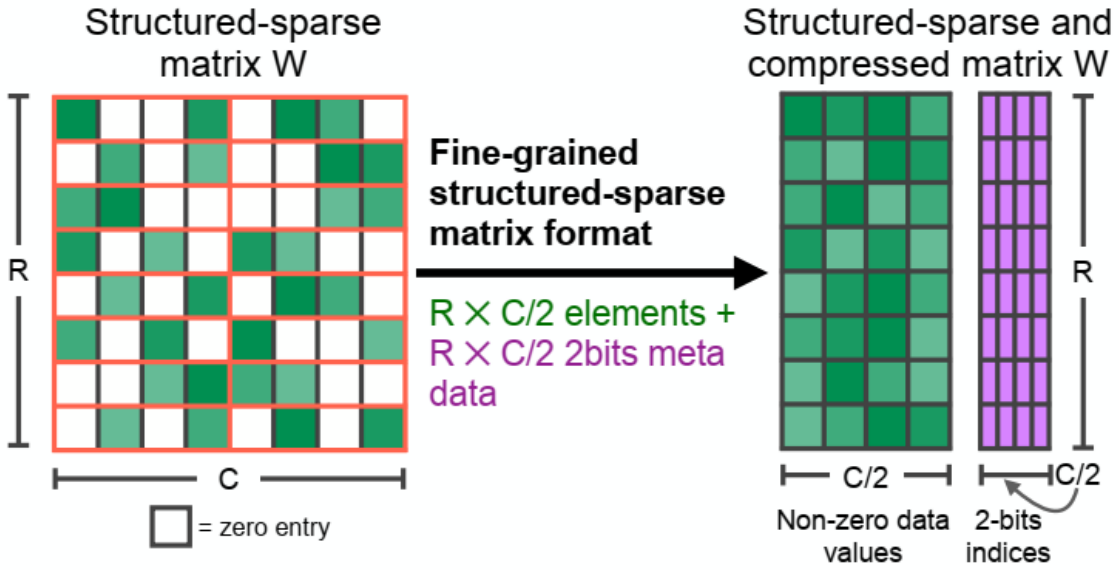




Accelerating Sparse Convolution with Column Vector-Wise Sparsity [NeurIPS'22]

N:M sparsity

- In each contiguous M elements, N of them is pruned
- A classic case is 2:4 sparsity (50% sparsity)
- Supported by NVIDIA's Ampere GPU Architecture (Tensor Core)
- Usually maintains accuracy

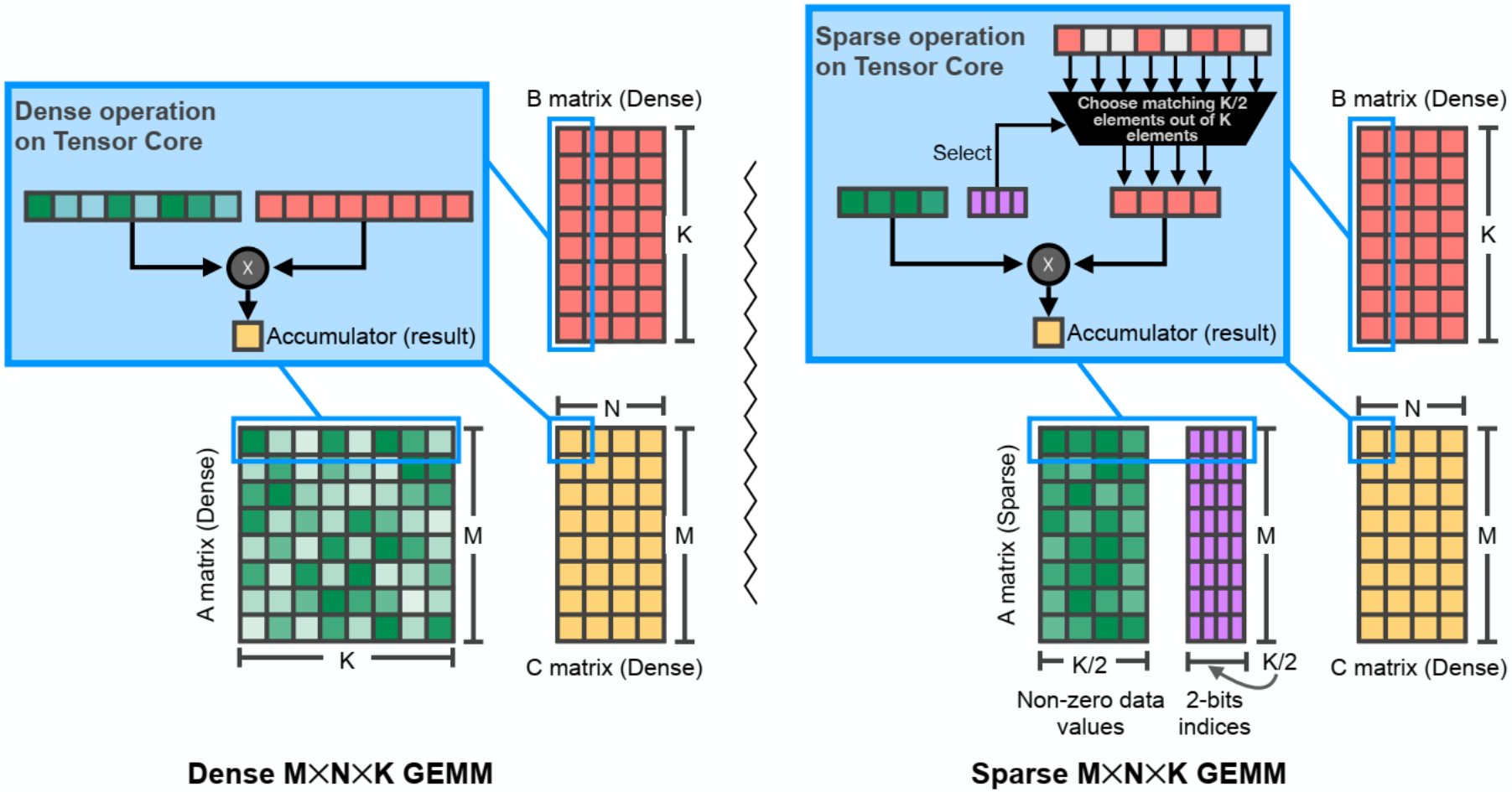


Network	Data Set	Metric	Dense FP16	Sparse FP16
ResNet-50	ImageNet	Top-1	76.1	76.2
ResNeXt-101_32x8d	ImageNet	Top-1	79.3	79.3
Xception	ImageNet	Top-1	79.2	79.2
SSD-RN50	COCO2017	bbAP	24.8	24.8
MaskRCNN-RN50	COCO2017	bbAP	37.9	37.9
FairSeq Transformer	EN-DE WMT'14	BLEU	28.2	28.5
BERT-Large	SQuAD v1.1	F1	91.9	91.9

Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

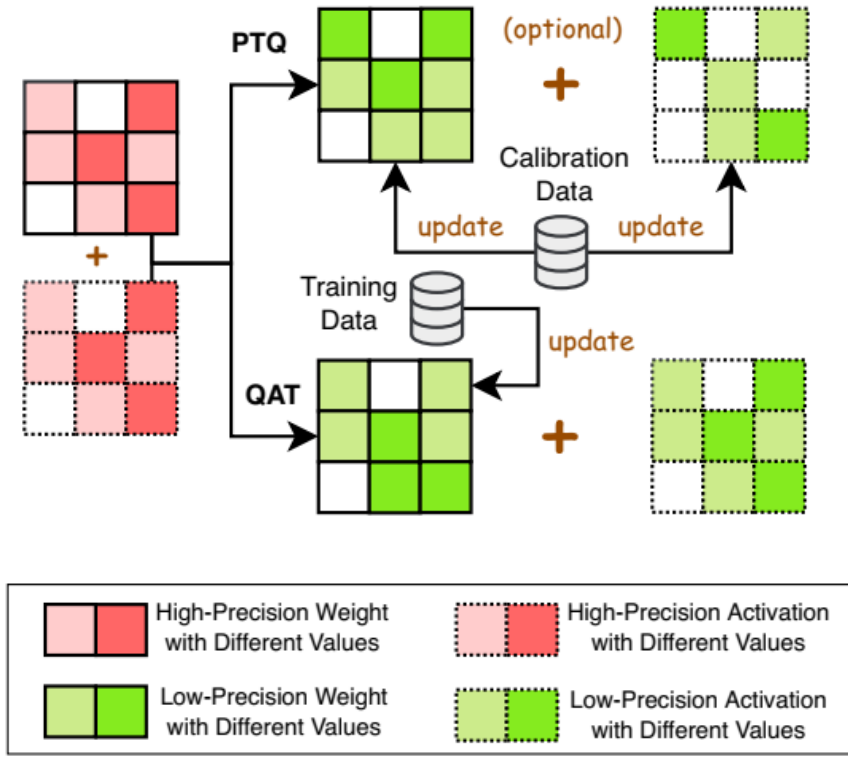
N:M sparsity

- Mapping M:N sparse matrices onto NVIDIA tensor cores



Accelerating Sparse Deep Neural Networks [Mishra et al., 2021]

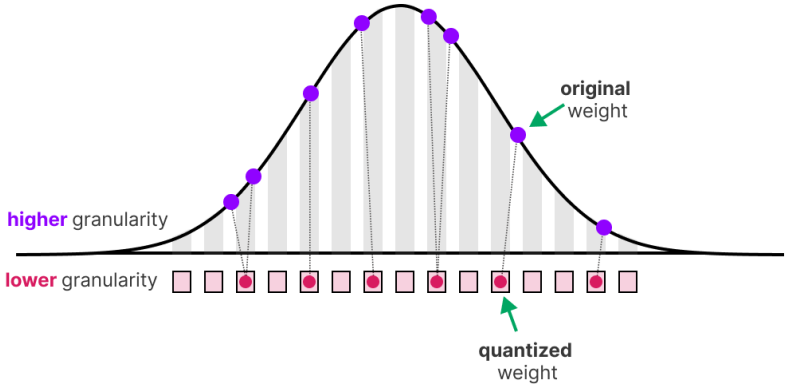
Quantization

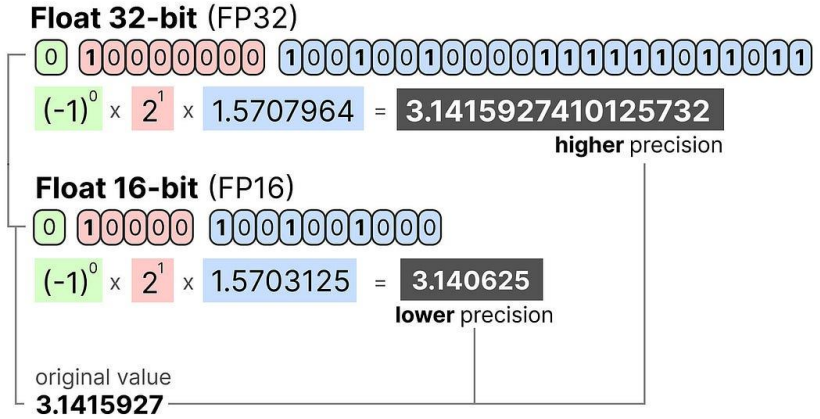


Quantization

- Mapping data to a smaller set of quantization levels
  - FP32 → FP16 → Int8 → Int4 → Int2 → Binary
- Reduced storage cost, reduced computation requirements.
- Must minimize the quantization error
- The number of quantization levels reflects the precision and the number of bits required to represent the data

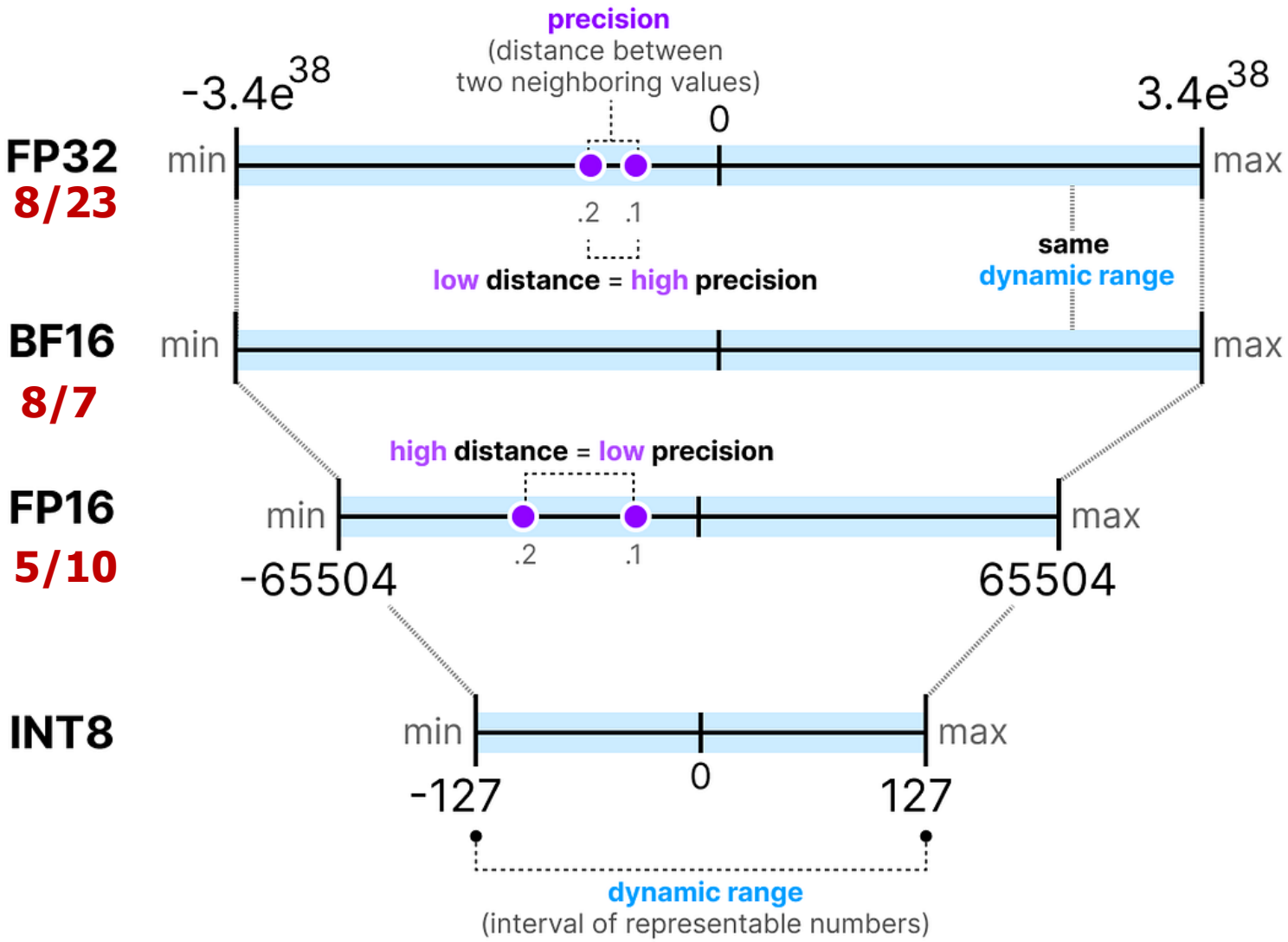




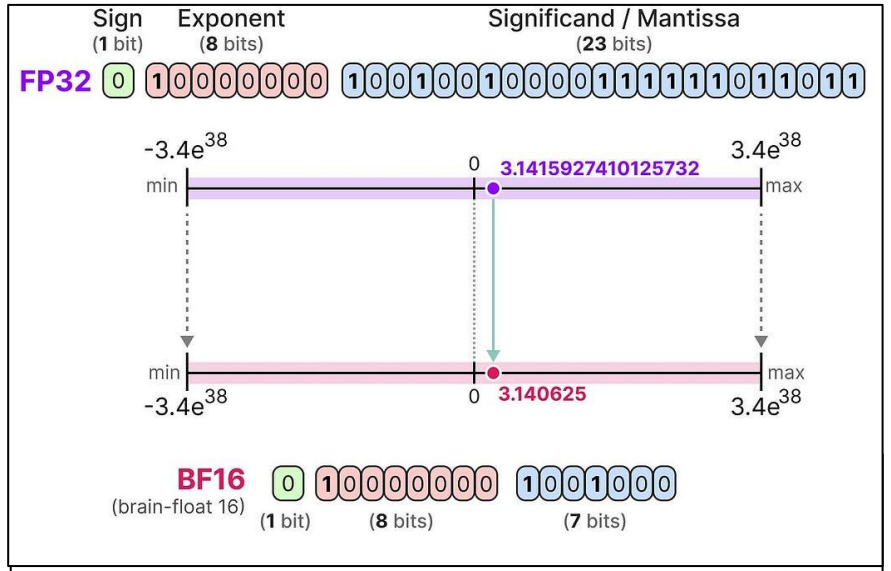


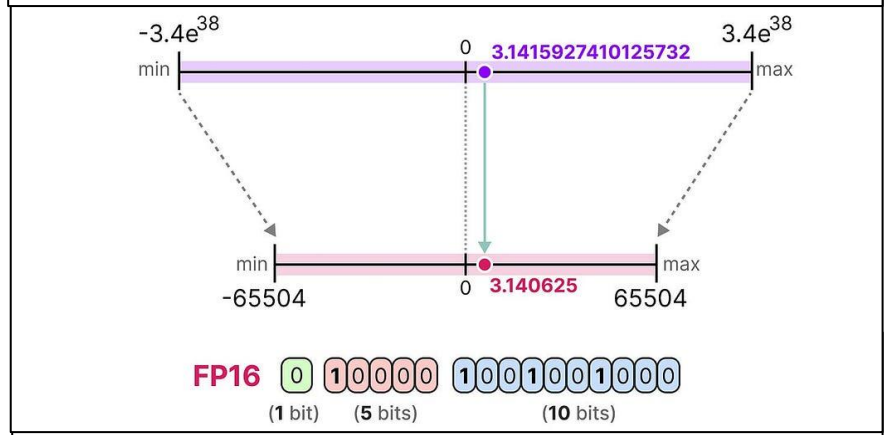
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

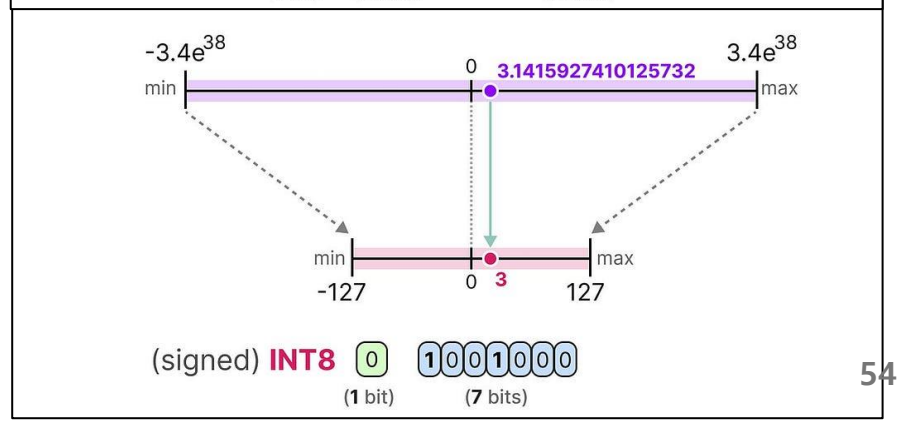
Numeric Data Types



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

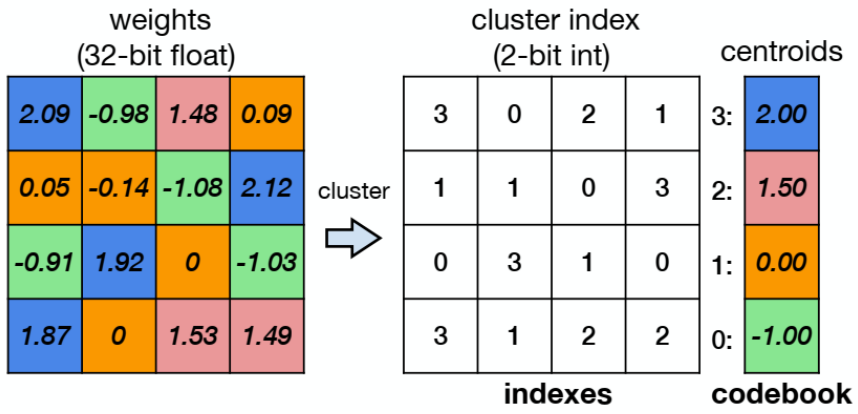




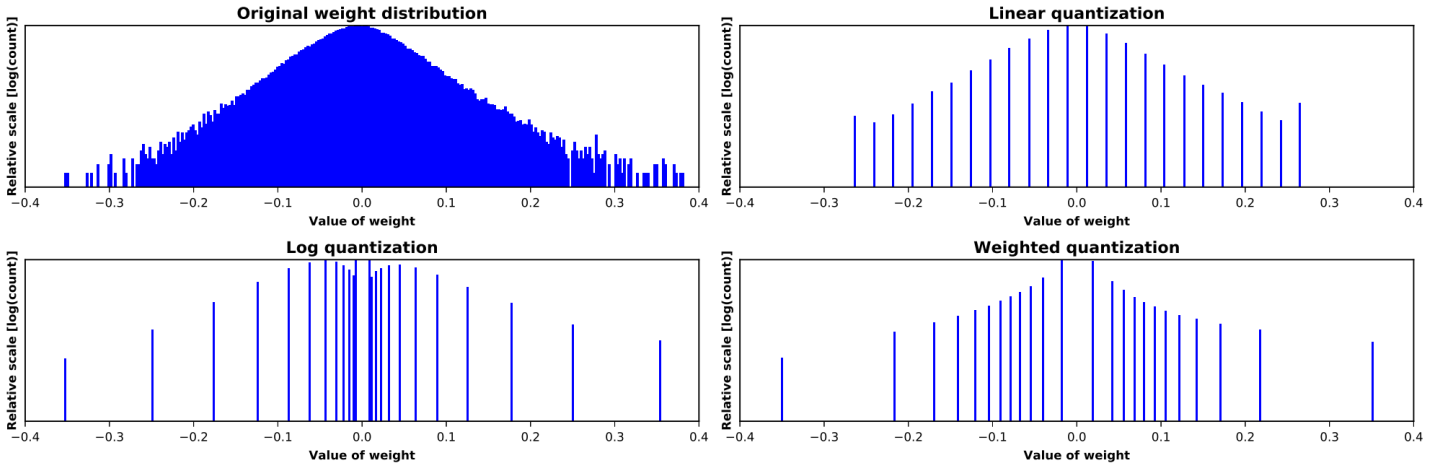


Quantization

- **Learned Quantization**
  - Quantization levels are determined or learned from the data (e.g., k-means clustering)
  - Mapping is usually implemented with a lookup table.
- **Uniform Quant (Linear Quant)**
  - Affine mapping of integers to real numbers, $r = S(q - Z)$
  - Uniform distance between each quantization level
- **Non-Uniform Quant (Log Quant)**
  - Distance between the levels varies
  - Can often be implemented with simple logic such as a shift.



Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding (ICLR'16)



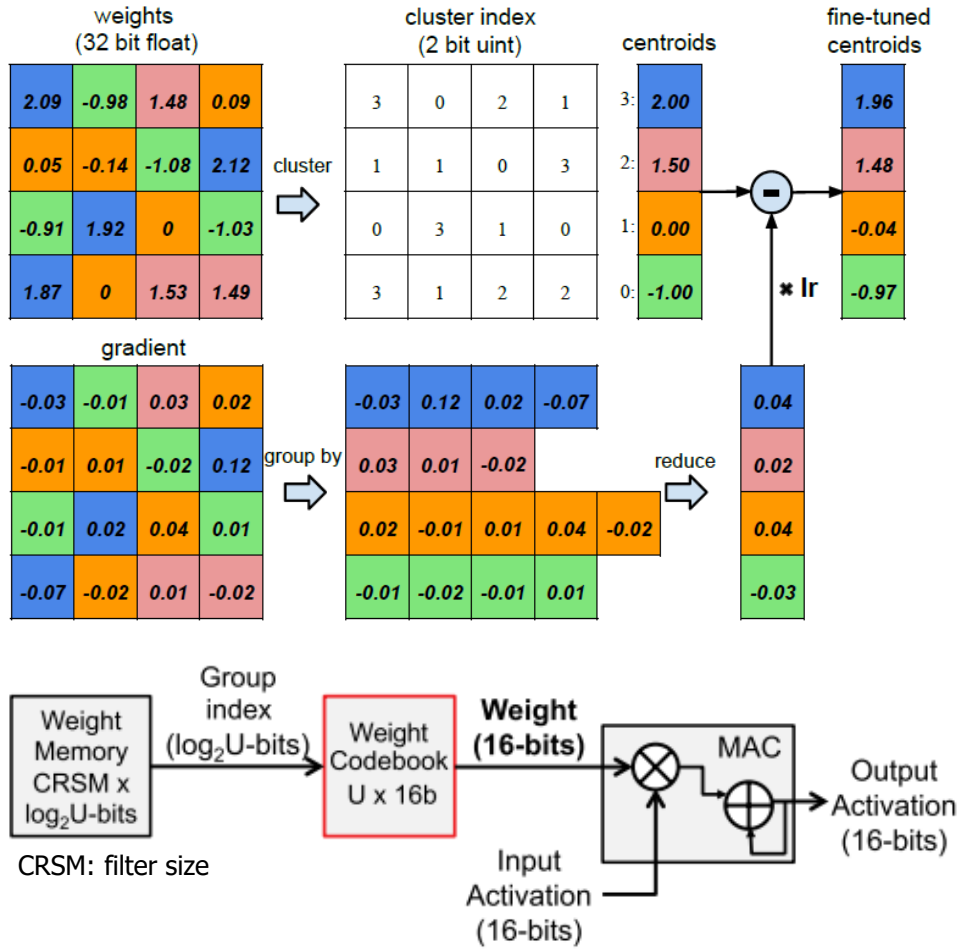
Weighted-Entropy-based Quantization for Deep Neural Networks (CVPR'17)

Static vs Dynamic Quantization

- Static quantization
  - Clipping range is pre-calculated and static during inference.
  - No additional computational overhead
  - Lower accuracy
  - Pre-calculation method
    - minimizing Mean Squared Error (MSE) between original unquantized weight distribution and the corresponding quantized values or entropy
- Dynamic quantization
  - Clipping range is dynamically calculated for each activation map during runtime
  - requires real-time computation of the signal statistics (min, max, percentile, etc.)
  - Higher accuracy

Learned Quantization

- Weight sharing
  - Weights are grouped using a hashing function or a k-means algorithm
  - One value is assigned to each group
  - Reduce # of unique weights in a filter
- Codebook-based mapping
  - Each group of weights has an index
  - The index is stored for each position in the filter rather than a weight value.
  - Two-step process to fetch the weight
- Reduce the cost of weight access
  - if bitwidth of group index < bitwidth of weight
  - not reduce the precision of the MAC computation itself



Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding (ICLR'16)

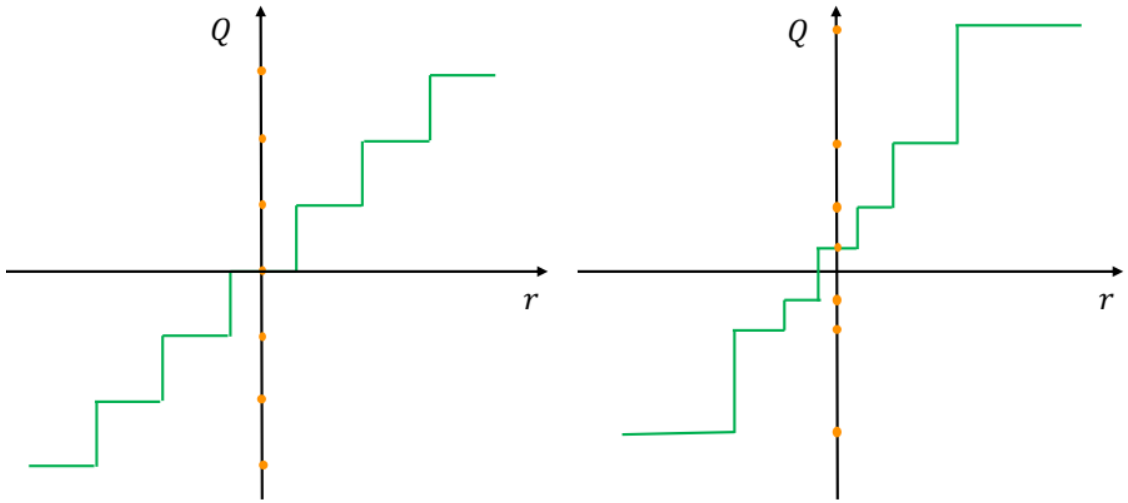
Learned Quantization

Table 1: The compression pipeline can save 35× to 49× parameter storage with no loss of accuracy.

Network	Top-1 Error	Top-5 Error	Parameters	Compress Rate
LeNet-300-100 Ref	1.64%	-	1070 KB	
LeNet-300-100 Compressed	1.58%	-	**27 KB**	**40×**
LeNet-5 Ref	0.80%	-	1720 KB	
LeNet-5 Compressed	0.74%	-	**44 KB**	**39×**
AlexNet Ref	42.78%	19.73%	240 MB	
AlexNet Compressed	42.78%	19.70%	**6.9 MB**	**35×**
VGG-16 Ref	31.50%	11.32%	552 MB	
VGG-16 Compressed	31.17%	10.91%	**11.3 MB**	**49×**

Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding (ICLR'16)

Uniform vs Non-Uniform

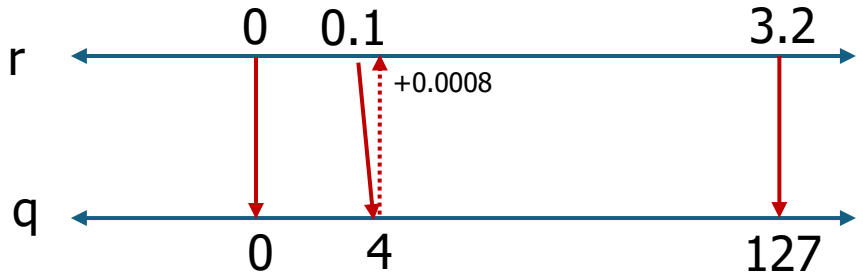


$$q = Int\left(\frac{r}{S}\right) + Z \qquad \tilde{r} = S(q - Z)$$

Quantization Dequantization

r : real valued input (activation or weight)
S : real valued scaling factor = max_real /max_quant
Z : integer zero point
Int : rounding operation (e.g., round to nearest and truncation)

$$Q(r) = X_i, \text{if } r \in [\Delta_i, \Delta_{i+1})$$



$$\mathbf{X}_{\text{quant}} = \text{round}\left(\frac{127}{\max|\mathbf{X}|} \cdot \mathbf{X}\right)$$

$$\mathbf{X}_{\text{dequant}} = \frac{\max|\mathbf{X}|}{127} \cdot \mathbf{X}_{\text{quant}}$$

If Z= 0, max|X| = 3.2, 8-bit linear quantization,
A weight of 0.1 is quantized to round(0.1 × 127/3.2) = 4.
4 is dequantized to 4 × 3.2/127 = 0.1008 (quantization error = 0.0008)

Uniform vs Non-Uniform

- Uniform (Linear) Quantization
  - Even step length between adjacent levels
  - Cannot match the realistic data distribution well

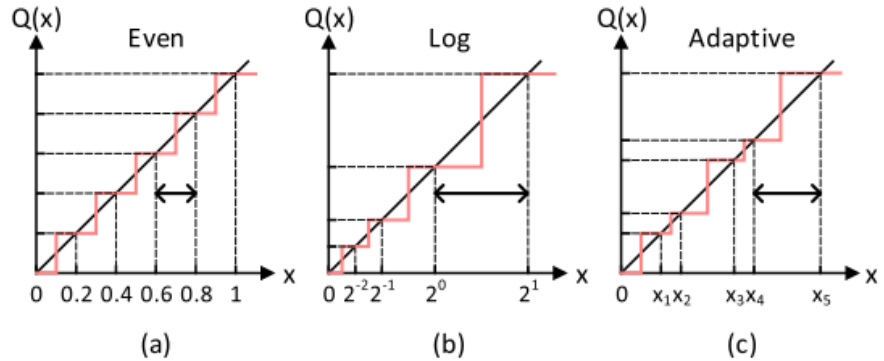


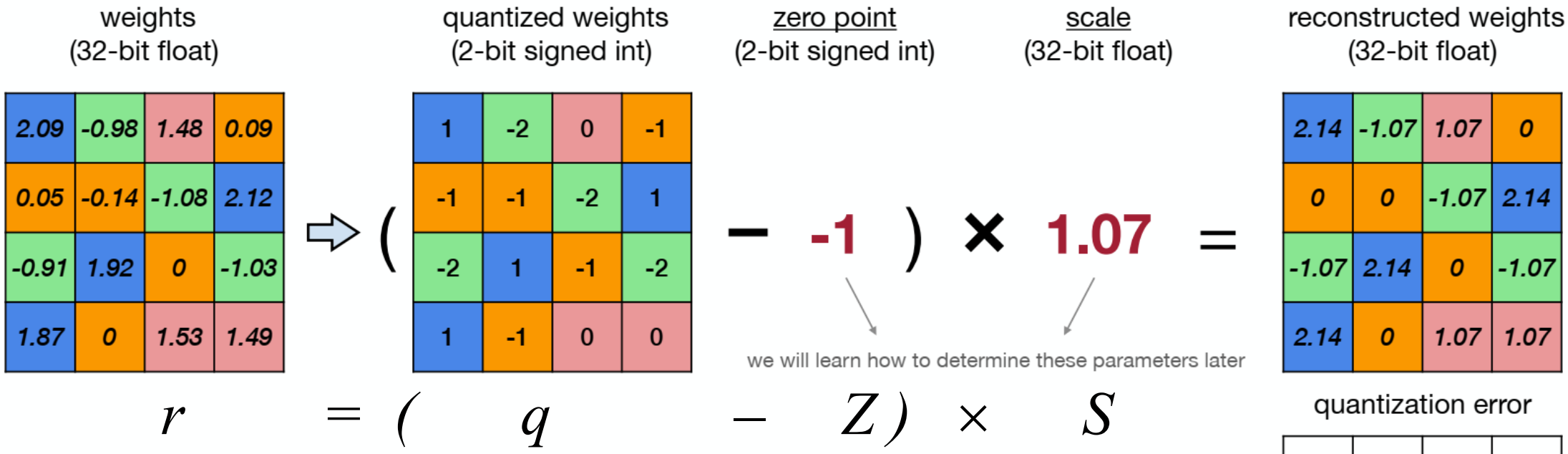
Fig. 15. ***Distribution of discrete levels. (a) Uniform distribution with even step length. Nonuniform distribution with (b) logarithmic or (c) adaptive step length.***

Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al. 2020]

- Non-uniform distribution
  - Variable step length => difficult to deploy efficiently on real H/W
  - Higher accuracy by focusing more on important value regions and finding appropriate dynamic ranges
    - 27.8% loss for 4b uniform quant vs. 5% loss for log-2 quant (VGG-16 LogNet)
  - Logarithmic approach
    - Exponential variance on step length. Multiplication ➔ bit-shift
    - Accumulation needs a full precision storage.
  - Adaptive step length
    - Find a discrete space to approximate the original high-precision one, Learnable quantizer
    - Determined according to the distribution of real data and the learning target

Linear Quantization

An affine mapping of integers to real numbers



Binary	Decimal
01	1
00	0
11	-1
10	-2

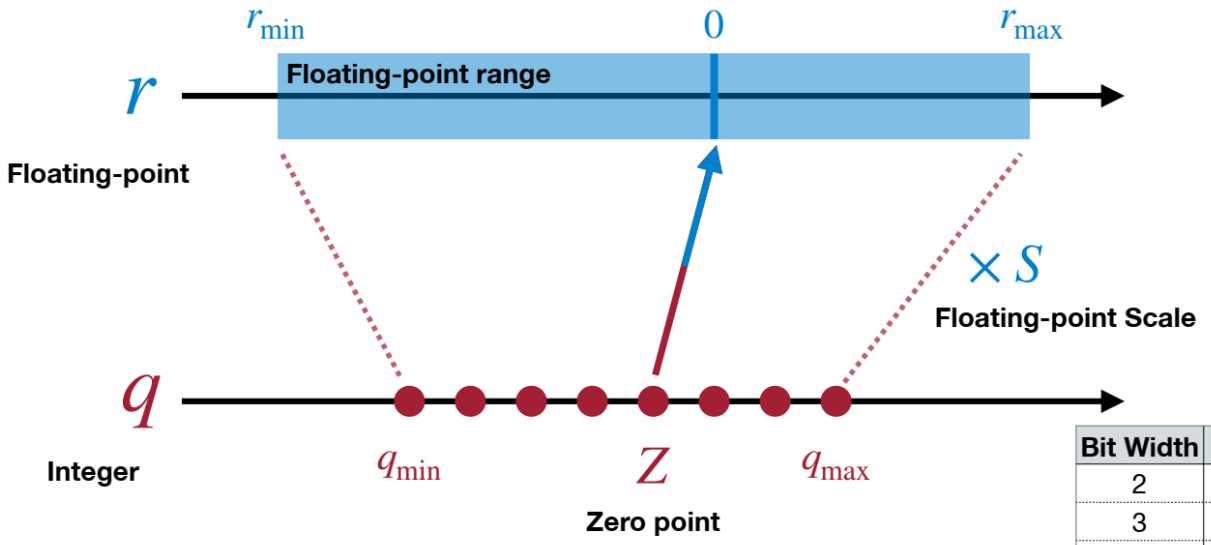
quantization error

-0.05	0.09	0.41	0.09
0.05	-0.14	-0.01	-0.02
0.16	-0.22	0	0.04
-0.27	0	0.46	0.42

From https://efficientml.ai

Linear Quantization

- An affine mapping of integers to real numbers $r = S(q - Z)$



Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

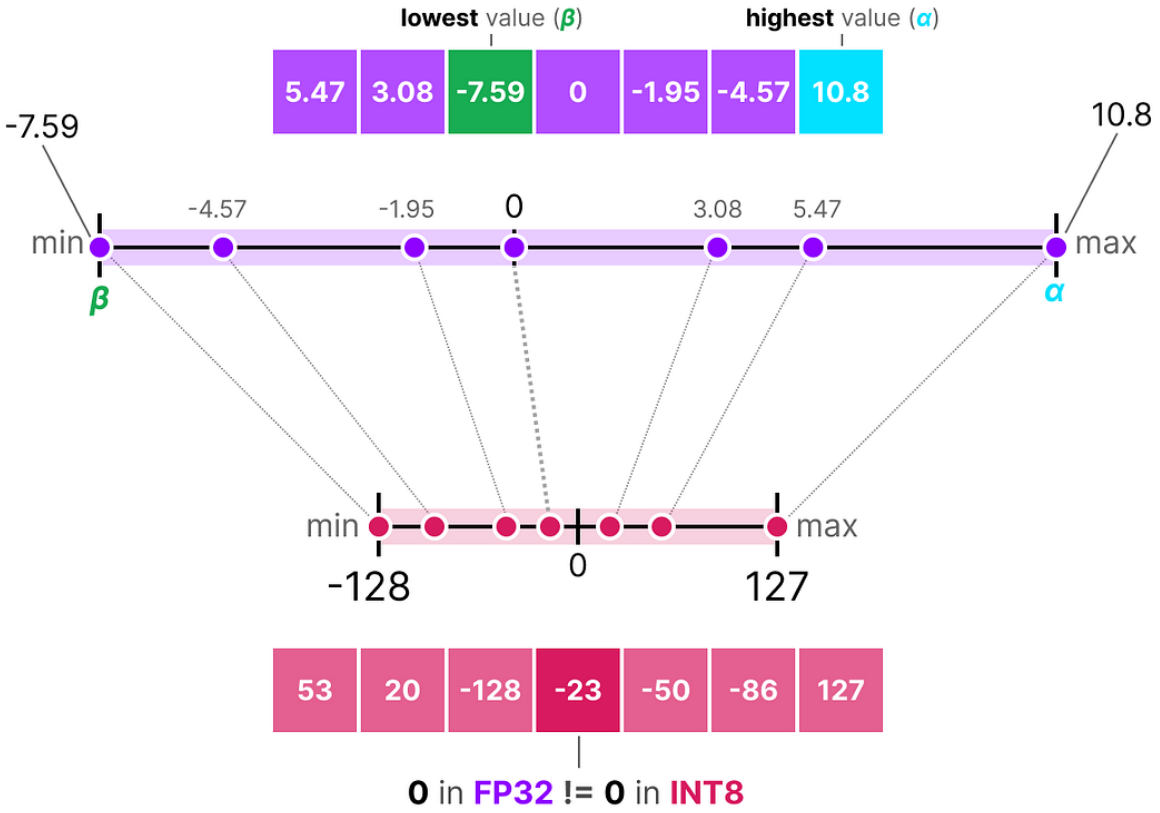
$$r_{max} = S(q_{max} - Z)$$
$$r_{min} = S(q_{min} - Z)$$
$$r_{max} - r_{min} = S(q_{max} - q_{min})$$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$r_{min} = S(q_{min} - Z)$$
$$Z = q_{min} - \frac{r_{min}}{S}$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$

Linear Quantization



$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$= \frac{10.8 - (-7.59)}{127 - (-128)} = \frac{18.39}{255} = 0.072$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$

$$= round\left(-128 - \frac{-7.59}{0.072}\right)$$

$$= round(-22.58) = -23$$

Quantize: $q = round\left(\frac{r}{S}\right) + Z$

$$= round\left(\frac{5.47}{0.072}\right) - 23 = 53$$

Dequnatize: $r = S(q - Z)$

$$= 0.072(53 + 23) = 5.472$$

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

Linear Quantized Matrix Multiplication

$$Y = WX$$

$$S_Y(q_Y - Z_Y) = S_W(q_W - Z_W) \cdot S_X(q_X - Z_X)$$

$$r = S(q - Z)$$

$$q_Y = \frac{S_W S_X}{S_Y}(q_W - Z_W)(q_X - Z_X) + Z_Y$$

$$q_Y = \frac{S_W S_X}{S_Y}(q_W q_X - Z_W q_X - \boxed{Z_X q_W + Z_W Z_X}) + Z_Y$$

Precompute

The only non-integer

N-bit Integer Multiplication
32-bit Integer Addition/Subtraction

N-bit Integer Addition

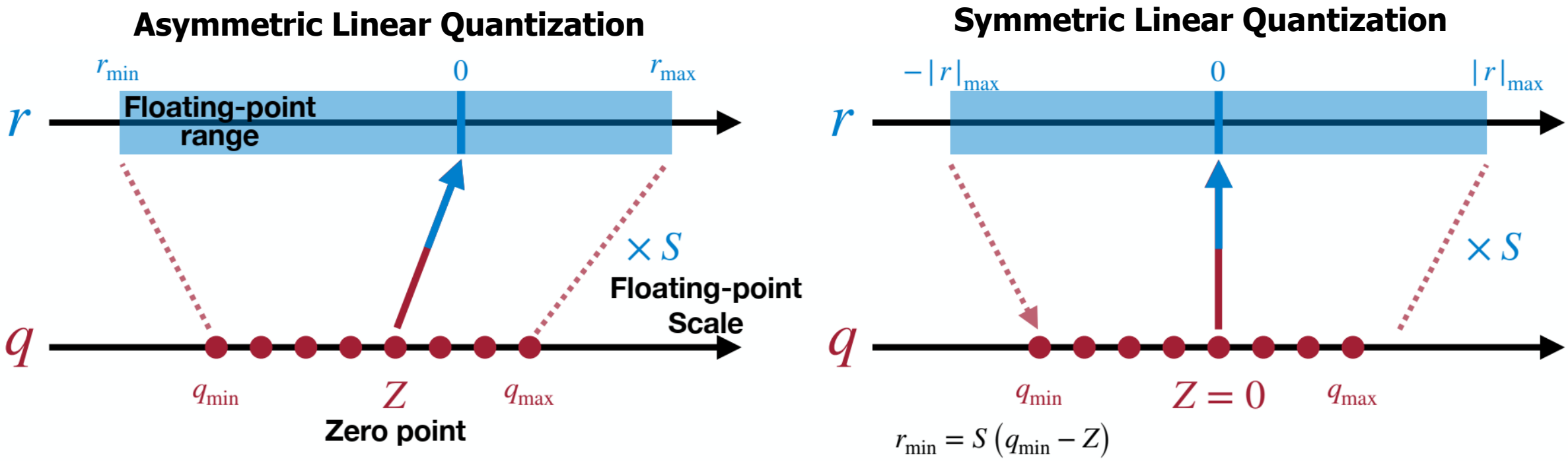
Empirically, $\frac{S_W S_X}{S_Y} \in (0,1)$.

$\frac{S_W S_X}{S_Y} = 2^{-n} M_0$, $M_0 \in [0.5,1)$

(bit shift & fixed-point multiplication)

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Asymmetric vs. Symmetric

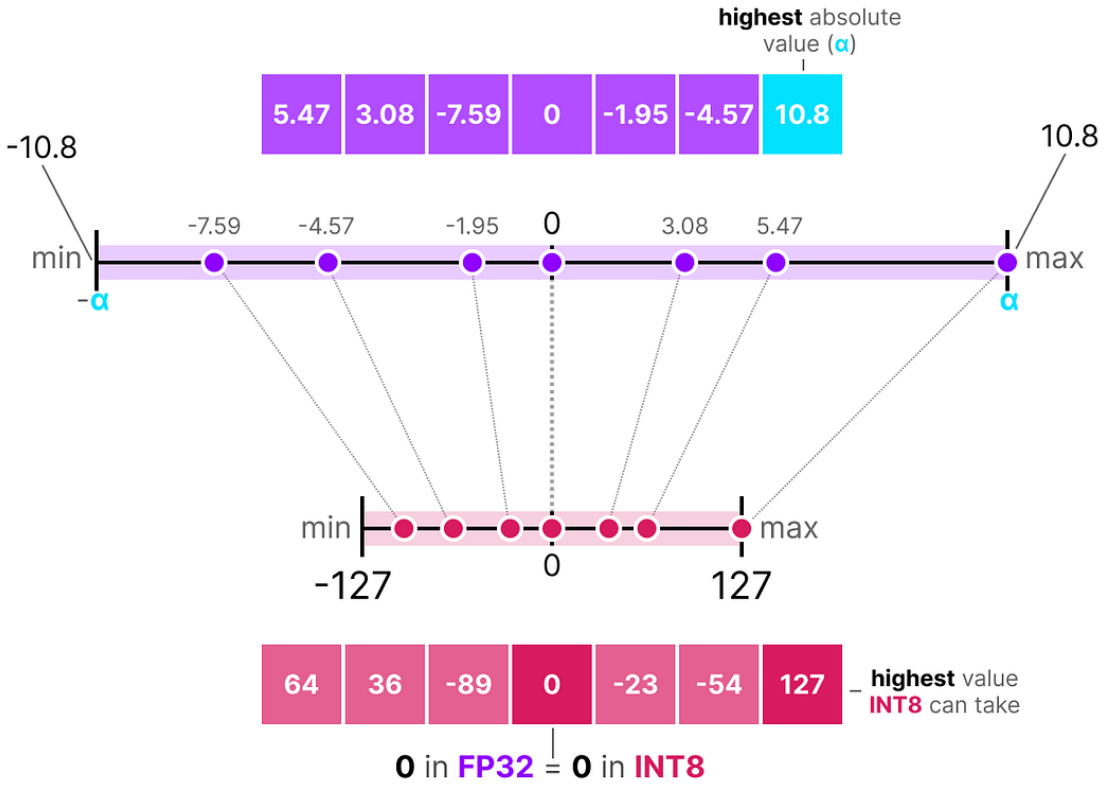


- The quantization range is fully used.
- The implementation is complex.

- The quantized range will be wasted for biased float range.
- The implementation is simpler.

From https://efficientml.ai

Symmetric Quantization



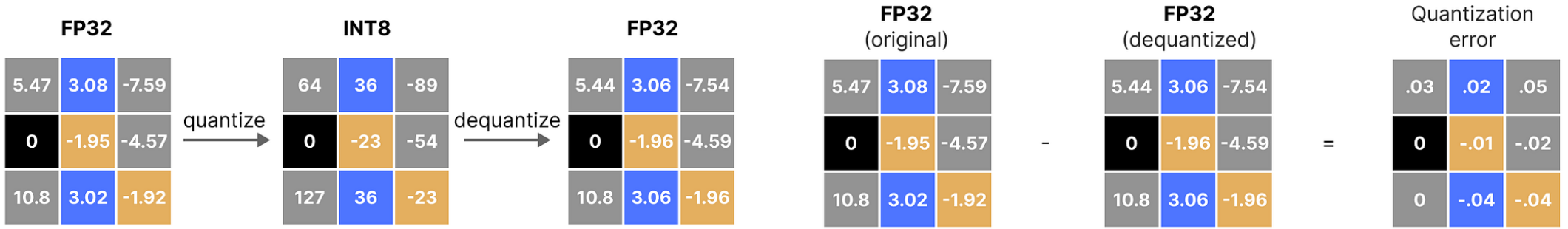
$$S = \frac{r_{max}}{q_{max}} = \frac{10.8}{127} = 0.085$$

Quantize: $q = round\left(\frac{r}{S}\right)$

$$= round\left(\frac{5.47}{0.085}\right) = 64$$

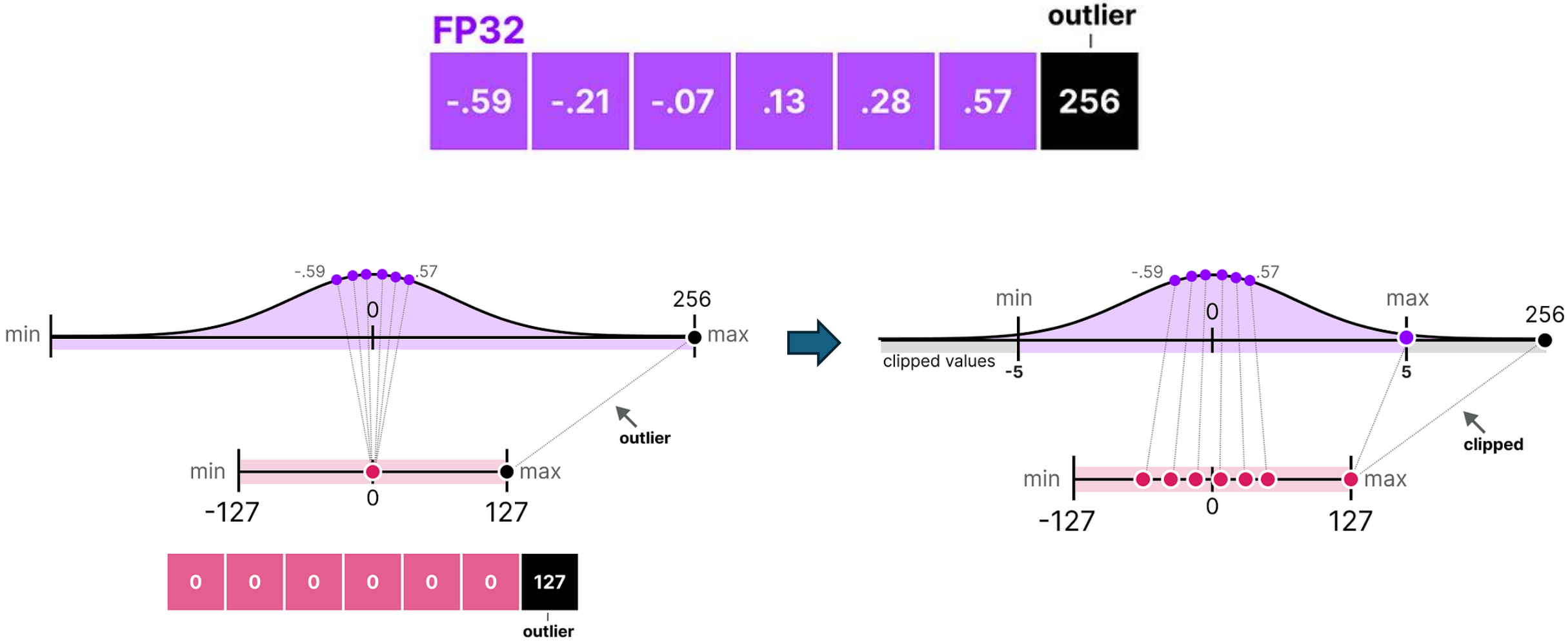
Dequnatize: $r = Sq$

$$= 0.085 \times 64 = 5.44$$



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

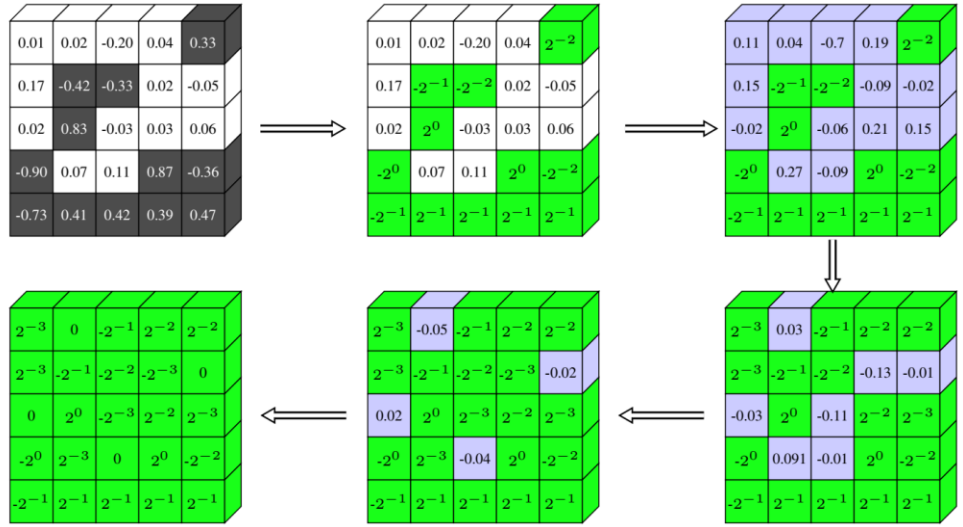
Range Mapping and Clipping



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

Non-Uniform Quantization

- Incremental network quantization (INQ)
  - At each iteration, the unquantized weights are partitioned into two disjoint groups (random or magnitude-aware)
  - One group is quantized into powers of two or zero while the other remained in high precision
  - Retraining to compensate the accuracy loss.
  - The portion of quantized weights gradually increases until it covers all weights.



Incremental Quantization

- Layer-wise incremental quantization (UNIQ)
  - k-quantile quantizer
  - Partitioning the whole network into multiple layer groups and gradually quantizing front groups as training evolves
  - Uniform noise injection: extra noise is injected into the weights in the group right after the quantized groups to emulate the quantization error.

Incremental Network Quantization: Towards Lossless CNNs with Low-Precision Weights (ICLR'17)
UNIQ: Uniform Noise Injection for Non-Uniform Quantization of Neural Networks (ACM Trans. Comput. Syst, 2021)

Non-Uniform Quantization

- Binarycode-based Quantization
  - a real-number vector $\boldsymbol{r} \in \mathbb{R}^n$ is quantized into m binary vectors
  - $\boldsymbol{r} \approx \sum_{i=1}^{m} \alpha_i \boldsymbol{b}_i$
    - Scaling factors $\alpha_i \in \mathbb{R}$, Binary vectors $\boldsymbol{b}_i \in \{-1, +1\}^n$.

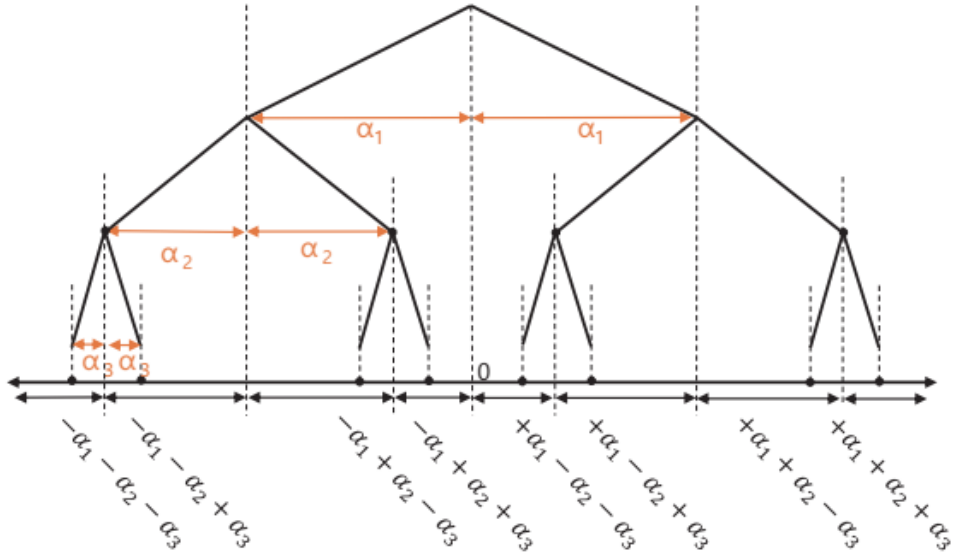


Fig. 3. An illustration of binary-coding-based quantization when the number of quantizaton bits is 3.

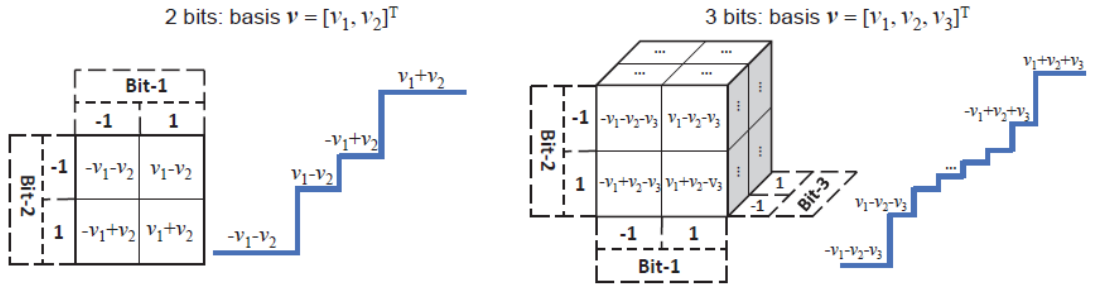


Fig. 2: Illustration of our learnable quantizer on the 2-bit (left) and 3-bit (right) cases. For each case, the left figure shows how quantization levels are generated by the basis vector, and the right figure illustrates the corresponding quantization function.

LQ-Nets (ECCV'18)

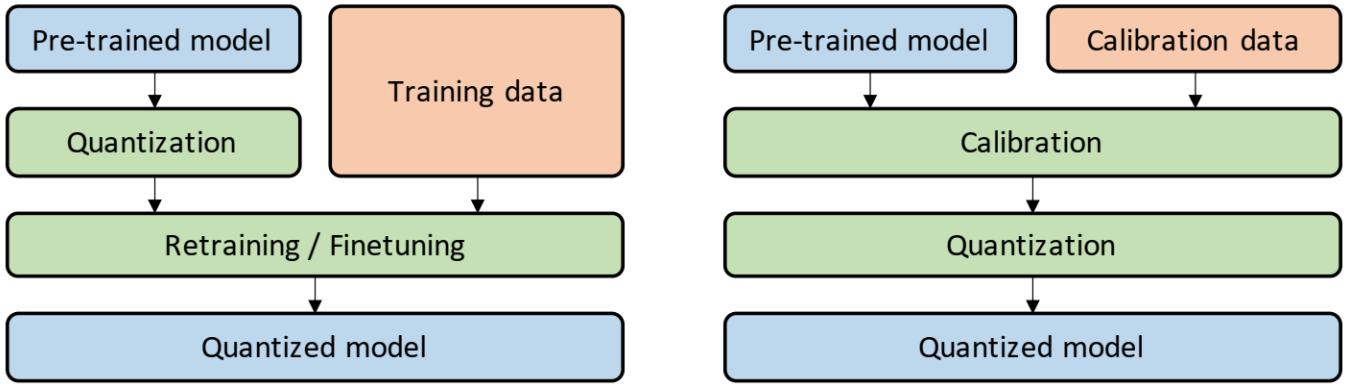
BiQGEMM: Matrix Multiplication with Lookup Table For Binary-Coding-based Quantized DNNs (SC'20)

Post-Training Quantization (PTQ)

How should we get the optimal linear quantization parameters (S, Z)?

Fine-tuning Methods

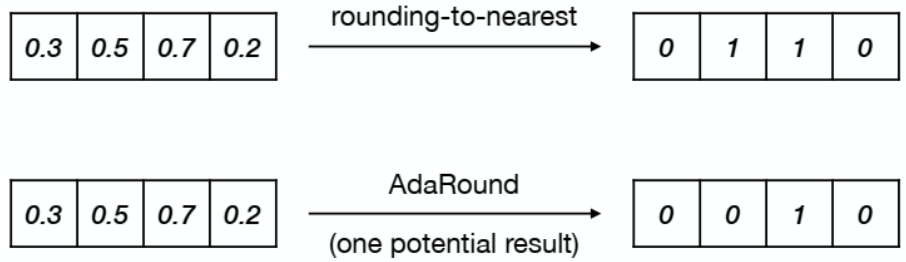
- Quantization may introduce a perturbation to the trained model parameters
- Need to adjust the parameters in the NN after quantization
- Quantization-Aware Training (QAT)
  - A pre-trained model is quantized and then fine-tuned using training data to adjust parameters and recover accuracy degradation
- Post-Training Quantization (PTQ)
  - A pre-trained model is calibrated using calibration data (e.g., a small subset of training data) to compute the clipping ranges and the scaling factors.

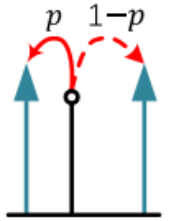


A Survey of Quantization Methods for Efficient Neural Network Inference

Post-Training Quantization

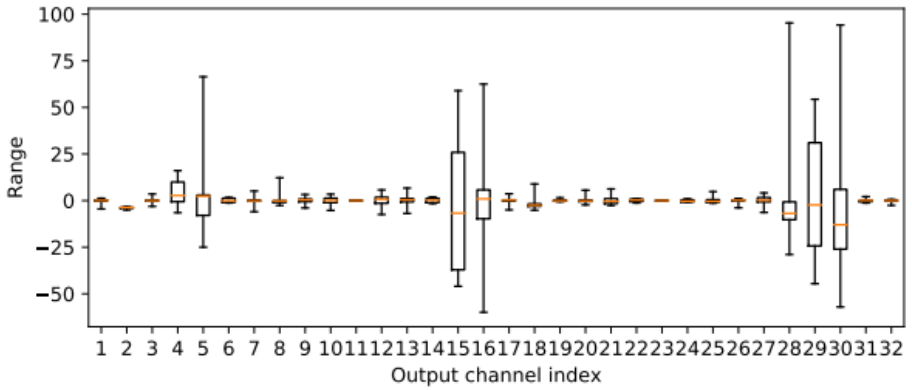
- Perform the quantization and the adjustments of the weights, without any fine-tuning
- Low overhead, but lower accuracy as compared to QAT, especially for low-precision
- Heuristic methods
  - Project data to the nearest discrete level
    - cf> adaptive round
  - $Q(x) = \Delta \cdot round(\frac{x}{\Delta})$
    - x is the original high-precision value in a continuous space
    - $Q(x)$ is the quantized data in a discrete space
    - $round(\cdot)$ is the rounding function
    - Δ is the quantization step length (uniform distribution)
  - k-bit quantization (Q^k)
    - $\Delta = \max(x)/(2^k - 1)$ (when we fully utilize the dynamic range)





Per-Channel Weight Quantization

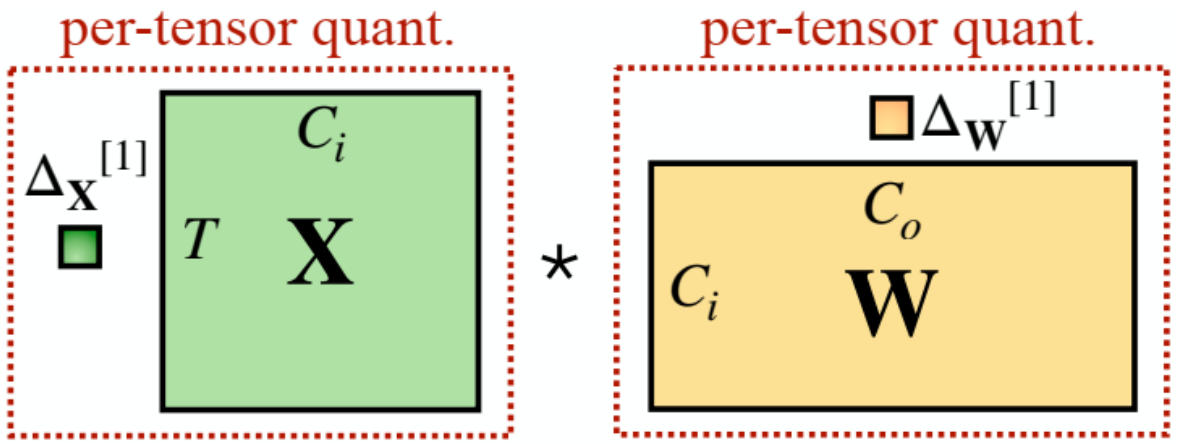
- Using single scale for whole weight tensor (Per-Tensor Quantization)
  - works sufficiently well for large models
  - leads to significant accuracy drops for small models
- Common failure results from
  - Large differences (more than 100×) in ranges of weights for different output channels
  - Outlier weight values that make all remaining weights less precise after quantization
- Solution: Per-Channel Quantization



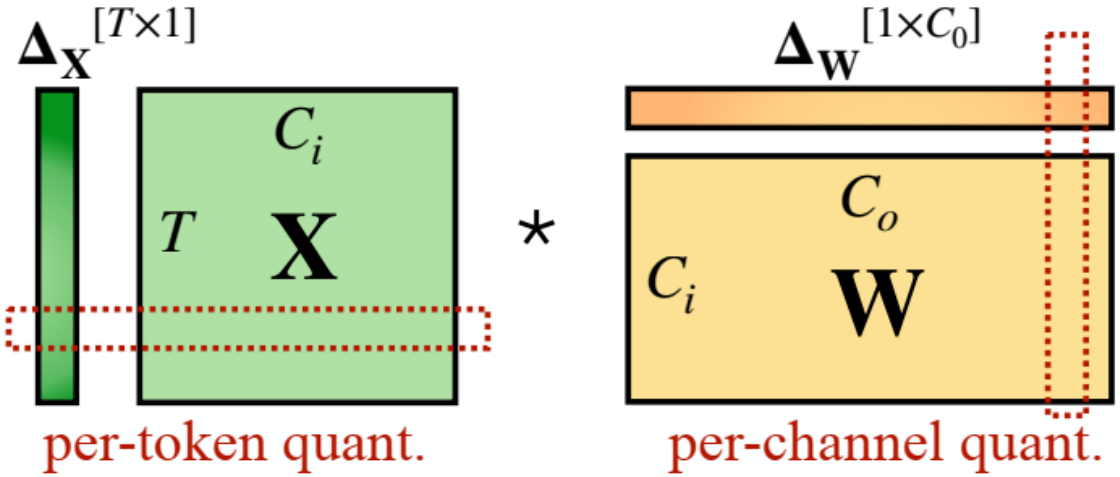
strong differences btwn channel weight ranges

Data-Free Quantization Through Weight Equalization and Bias Correction [ICCV 2019]

Per-Channel Weight Quantization



(a) per-tensor quantization

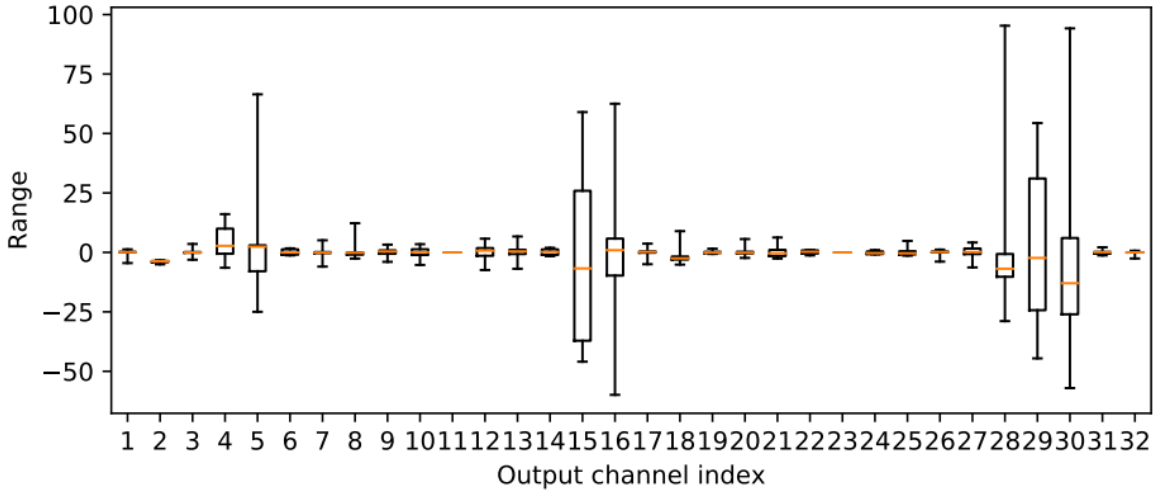


(b) per-token + per-channel quantization

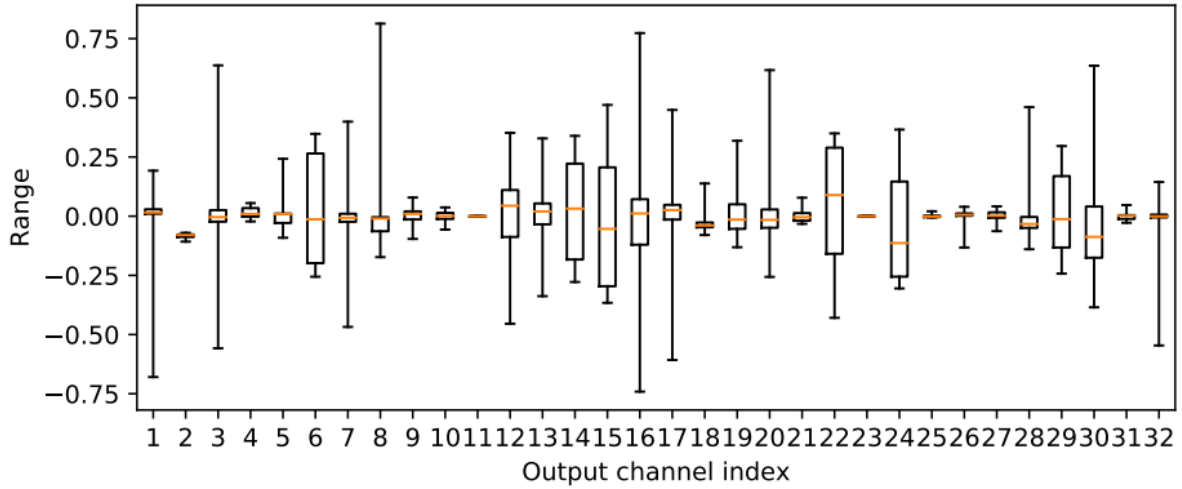
SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

Per-Channel Weight Quantization

Per-Tensor



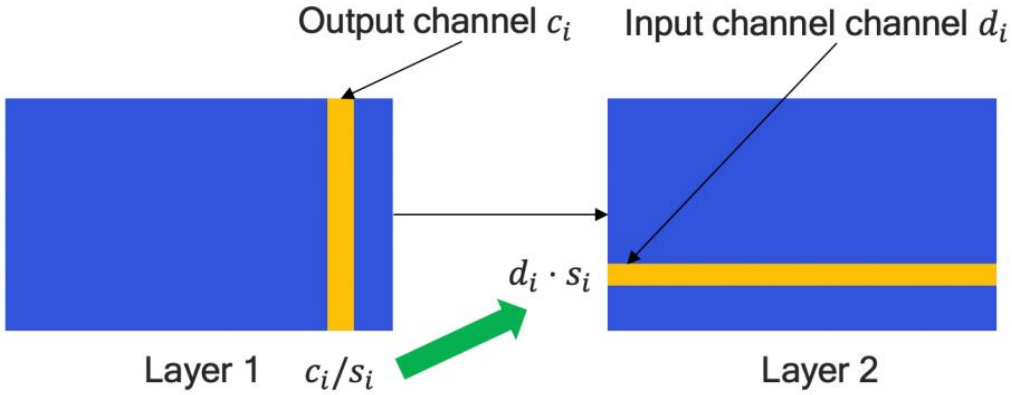
Per-Channel



Data-Free Quantization Through Weight Equalization and Bias Correction [ICCV 2019]

Weight Equalization

- Rescaling to equalize the weight ranges of different output channels
- Scaling equivariance
  - scaling down layer k weights of output channel c_i by s_i
  - scaling up layer $k+1$ weights of input channel d_i by s_i
- How to determine s_i
  - $s_i = \frac{1}{r_{d_i}^{k+1}} \sqrt{r_{c_i}^{k} r_{d_i}^{k+1}}$
  - $r_{c_i}^{k}$: quantization weight range of output channel i (c_i) in Layer k
  - $r_{d_i}^{k+1}$: quantization weight range of input channel i (d_i) in Layer $k+1$



$$\tilde{r}_{c_i}^{k} = \frac{r_{c_i}^{k}}{s_i} = \frac{r_{c_i}^{k} \cdot r_{d_i}^{k+1}}{\sqrt{r_{c_i}^{k} r_{d_i}^{k+1}}} = \sqrt{r_{c_i}^{k} r_{d_i}^{k+1}}$$

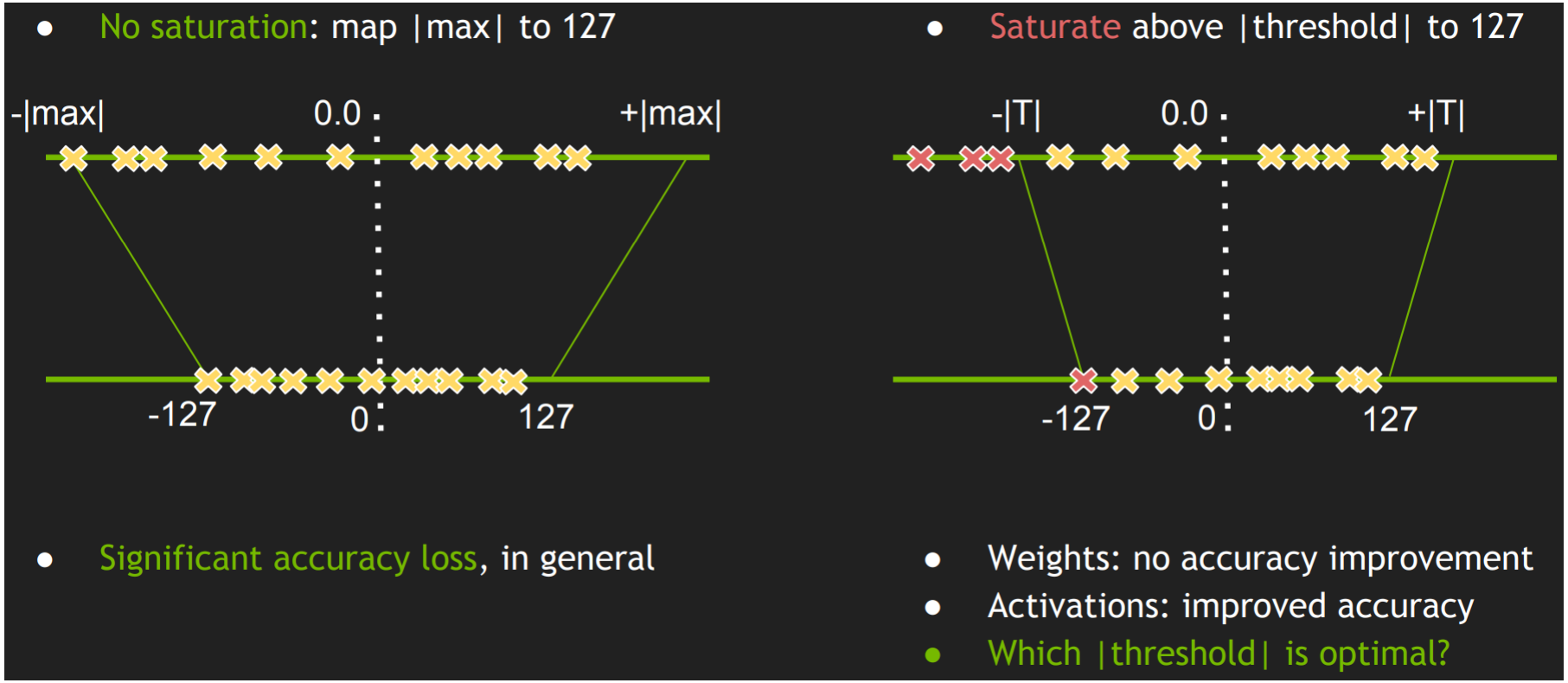
$$\tilde{r}_{d_i}^{k+1} = r_{d_i}^{k+1} \cdot s_i = \sqrt{r_{c_i}^{k} r_{d_i}^{k+1}}$$

Activation function must be linear. (e.g., ReLU)

Data-Free Quantization Through Weight Equalization and Bias Correction [ICCV 2019]
Same, Same But Different - Recovering Neural Network Quantization Error Through Weight Factorization [ICML 2019]

Dynamic Quantization (Activation)

- For different inputs, the floating-point range varies.
- To determine the floating-point range, the activations statistics are gathered before deploying the model



8-bit Inference with TensorRT [Szymon Migacz, 2017]

Post-Training INT8 Linear Quantization

Activation		Symmetric	Asymmertric			Asymmertric
		Per-Tensor	Per-Tensor			Per-Tensor
		Minimize KL-Divergence	Exponential Moving Average (EMA)			mean +/- 6 * std (from BatchNorm)
Weight		Symmetric	Asymmetric	Asymmetric	Symmetric	Asymmetric
		Per-Tensor	Per-Tensor	Per-Channel	Per-Channel	Per-Tensor
		-	-	-	-	Weight Equalization Bias Correction
Neural Network	GoogleNet	-0.45%	0%	0%	0%	-
	ResNet-50	-0.13%	-0.6%	-0.6%	-0.6%	-
	ResNet-152	-0.08%	-1.7%	-1.8%	-1.8%	-
	MobileNetV1	-	-70.8%	-0.6%	-11.8%	-0.3%
	MobileNetV2	-	-71.8%	-2.2%	-2.1%	-0.5%

Smaller models seem to not respond as well to post-training quantization, presumabley due to their smaller representational capacity

➔ How should we improve performance of quantized models?

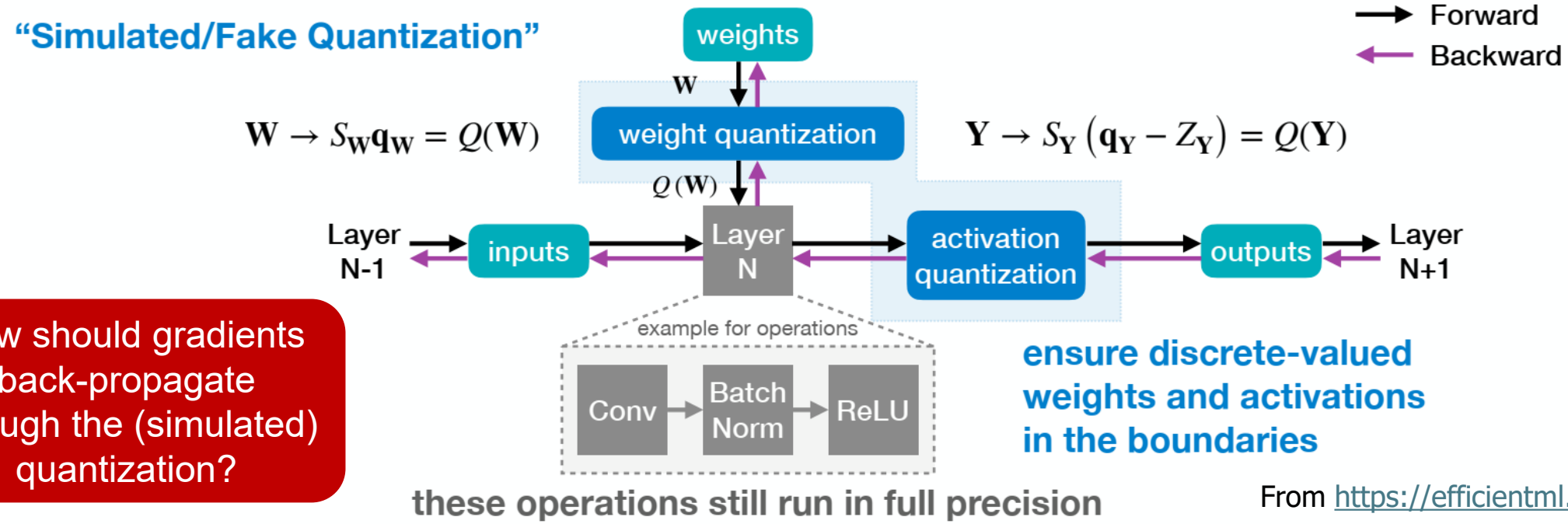
From https://efficientml.ai

Quantization-Aware Training (QAT)

How should we improve performance of quantized models?

Quantization-Aware Training

- Train the model taking quantization into consideration
- A full precision copy of the weights W is maintained throughout the training
- The small gradients are accumulated without loss of precision
- Once the model is trained, only the quantized weights are used for inference



How should gradients back-propagate through the (simulated) quantization?

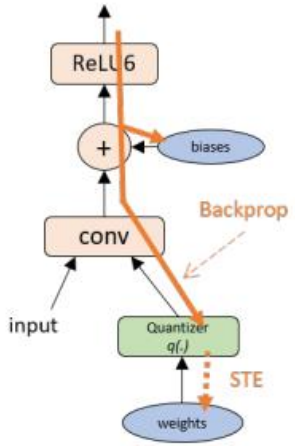
From https://efficientml.ai

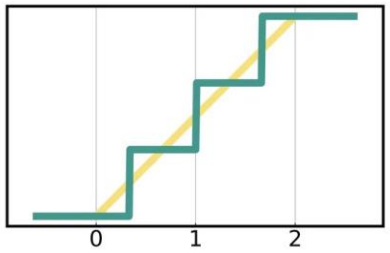
Gradient of Quantization Function

- Quantization function is nondifferentiable with respect to its input
  - Gradient presented as delta-spikes at the transition points
  - Not friendly to the SGD learning
- Straight Through Estimator (STE)
  - Ignore the rounding operation and approximates it with an identity function
  - Simply bypasses the quantization function during the gradient BP.
  - Take the gradient at the quantized value and perform the update on the original real value
  - Works well in practice, except for ultra low-precision quantization such as binary quantization

$$\frac{\partial \mathbf{w}_q}{\partial \mathbf{w}} = \frac{\partial q(\mathbf{w})}{\partial \mathbf{w}} \underset{STE}{\approx} \frac{\partial I(\mathbf{w})}{\partial \mathbf{w}} = 1$$

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial L(\mathbf{w}_q)}{\partial \mathbf{w}_q} \cdot \frac{\partial \mathbf{w}_q}{\partial \mathbf{w}} \underset{STE}{\approx} \frac{\partial L(\mathbf{w}_q)}{\partial \mathbf{w}_q}$$





Forward pass:

$$X_q = \text{Round}(\text{Clip}(0, X_r, 2^n - 1))$$

Backward pass:

$$\frac{\partial \mathcal{L}oss}{\partial X_r} \overset{\text{STE}}{=} \frac{\partial \mathcal{L}oss}{\partial X_q}$$

Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio, arXiv 2013]

Quantization-Aware Training

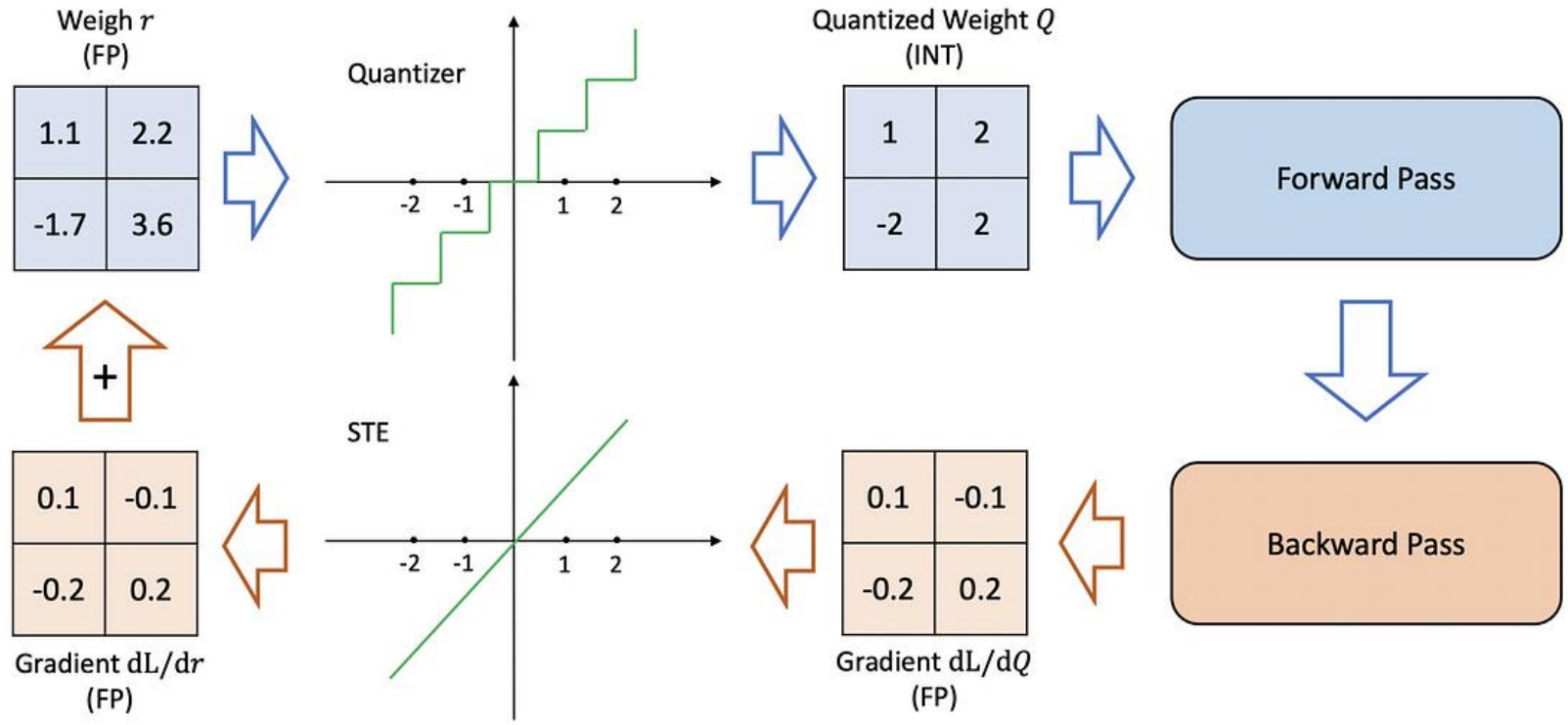


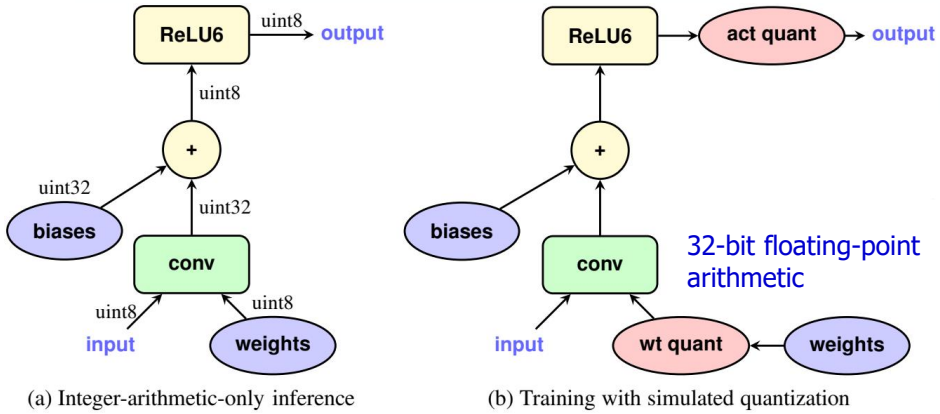
Figure 5: *Illustration of Quantization-Aware Training procedure, including the use of Straight Through Estimator (STE).*

A Survey of Quantization Methods for Efficient Neural Network Inference

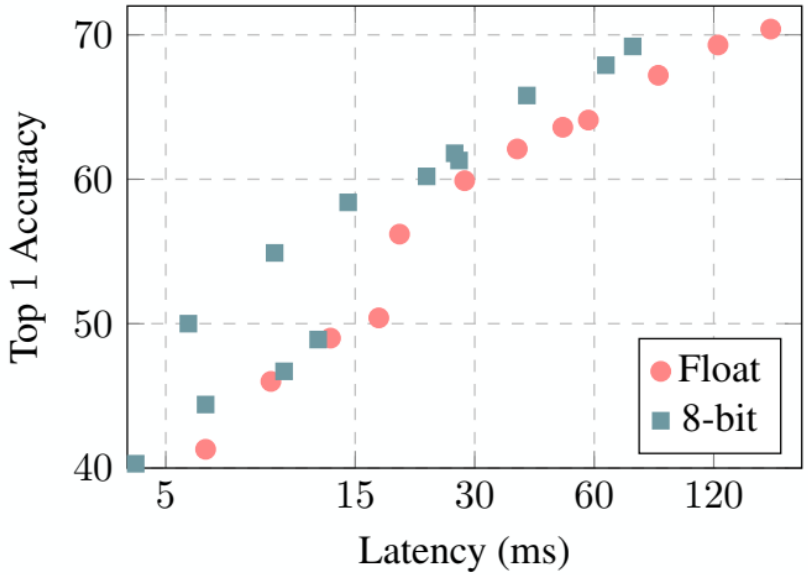
INT8 Linear Quantization-Aware Training

- An affine mapping of integers to real numbers $r = S(q - Z)$

Neural Network	ResNet-50	Inception-V3
Floating-point Accuracy	76.4%	78.4%
8-bit Integer-quantized Acurracy	74.9%	75.4%



(a) Integer-arithmetic-only inference (b) Training with simulated quantization



Latency-vs-accuracy tradeoff of float vs. integer-only MobileNets on ImageNet using Snapdragon 835 big cores.

Weight update during training

$$w_{float} = w_{float} - \eta \frac{\partial L}{\partial w_{out}} . I_{w_{out} \in (w_{min}, w_{max})}$$

$$w_{out} = SimQuant(w_{float})$$

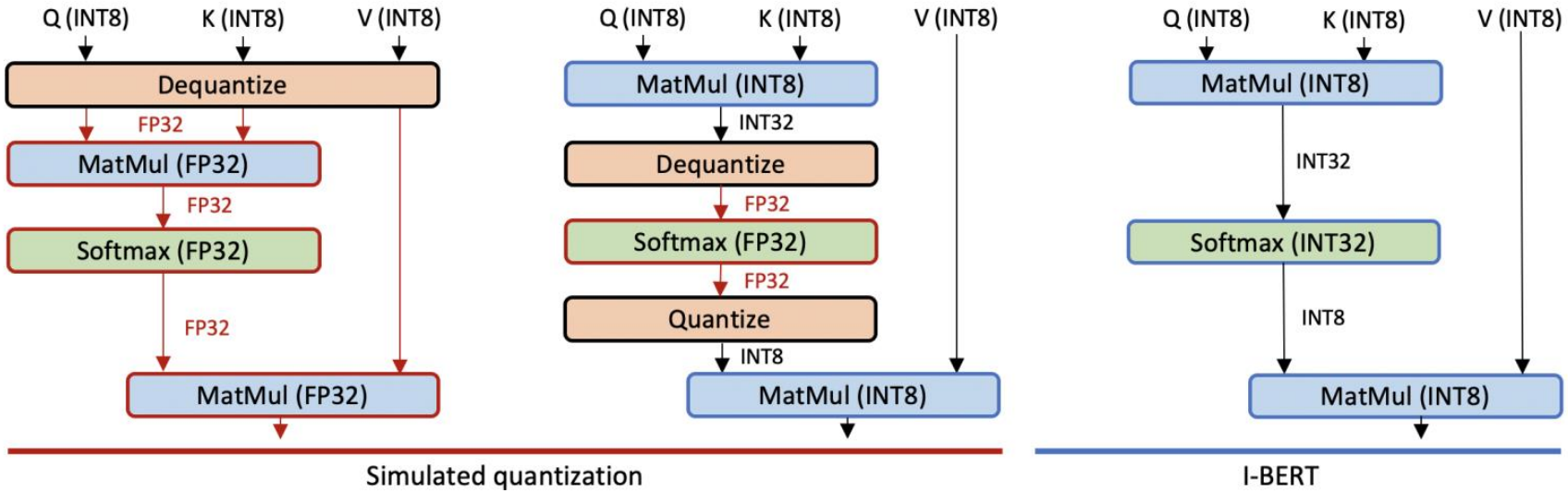
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

INT8 Linear Quantization-Aware Training

Neural Network	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNet-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

From https://efficientml.ai

Approximation for Nonlinear OPs



Integer-Only Approximation for non-linear operations of BERT

$$\text{GELU}(x) := x \cdot \frac{1}{2}\left[1 + \text{erf}(\frac{x}{\sqrt{2}})\right],$$

$$\text{where } \text{erf}(x) := \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2)dt.$$

$$\text{i-GELU}(x) := x \cdot \frac{1}{2}\left[1 + \text{L}(\frac{x}{\sqrt{2}})\right]$$

$$\text{L}(x) = \text{sgn}(x)\left[a(\text{clip}(|x|, \max = -b) + b)^2 + 1\right]$$

$$\text{Softmax}(\mathbf{x})_i = \frac{\exp(x_i - x_{\max})}{\sum_{j=1}^{k} \exp(x_j - x_{\max})}$$

$$\tilde{x}_i = x_i - x_{\max} \qquad \tilde{x} = (-\ln 2)z + p$$

$$\exp(\tilde{x}) = 2^{-z}\exp(p) = \exp(p) >> z$$

$$\text{i-exp}(\tilde{x}) := L(p) >> z$$

$$L(p) = 0.3585(p + 1.353)^2 + 0.344 \approx \exp(p)$$

$$\tilde{x} = \frac{x - \mu}{\sigma} \text{ where } \mu = \frac{1}{C}\sum_{i=1}^{C} x_i \text{ and } \sigma = \sqrt{\frac{1}{C}\sum_{i=1}^{C}(x_i - \mu)^2}$$

Newton's method of square root

function I-SQRT(n)
  if $n = 0$ **then return** 0
  Intialize x_0 to $2^{\lceil Bits(n)/2 \rceil}$ and i to 0
  repeat
    $x_{i+1} \leftarrow \lfloor (x_i + \lfloor n/x_i \rfloor)/2 \rfloor$
    if $x_{i+1} \geq x_i$ **then return** x_i
    else $i \leftarrow i + 1$
end function

I-BERT: Integer-only BERT Quantization (ICML'21)

Knowledge Distillation

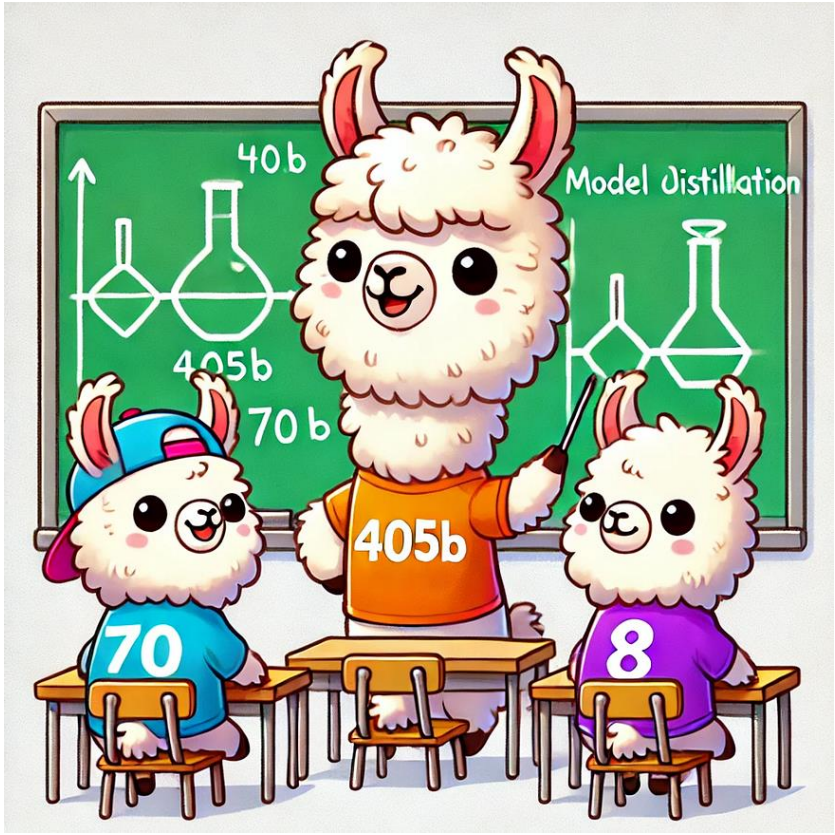
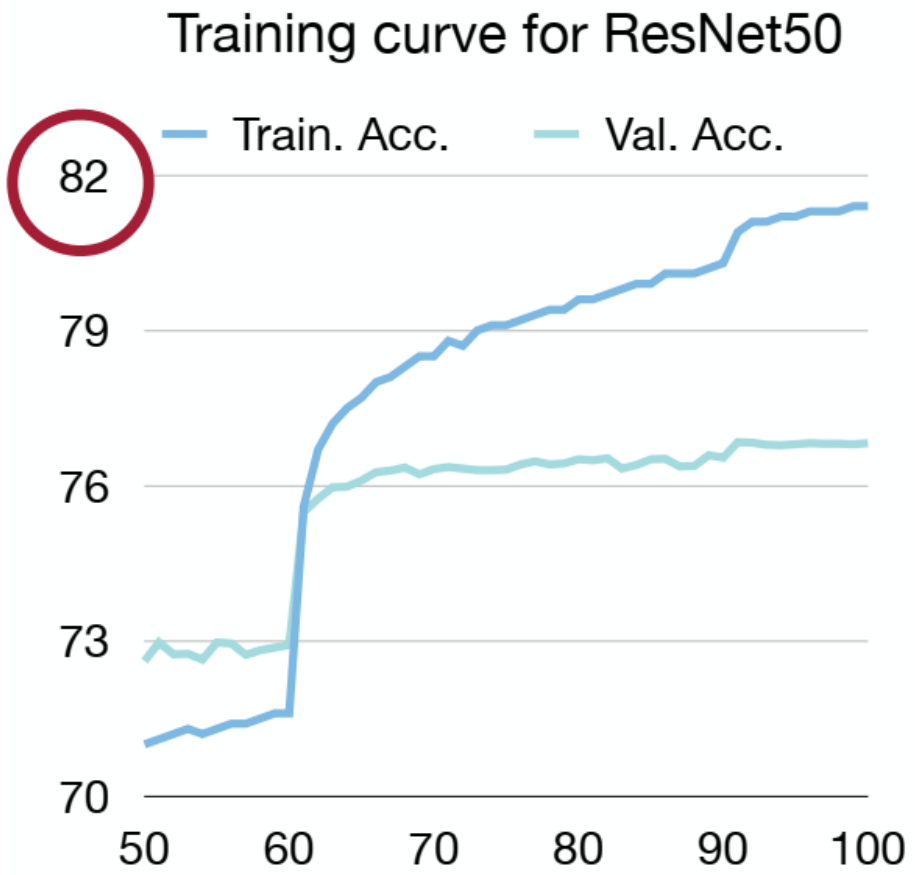
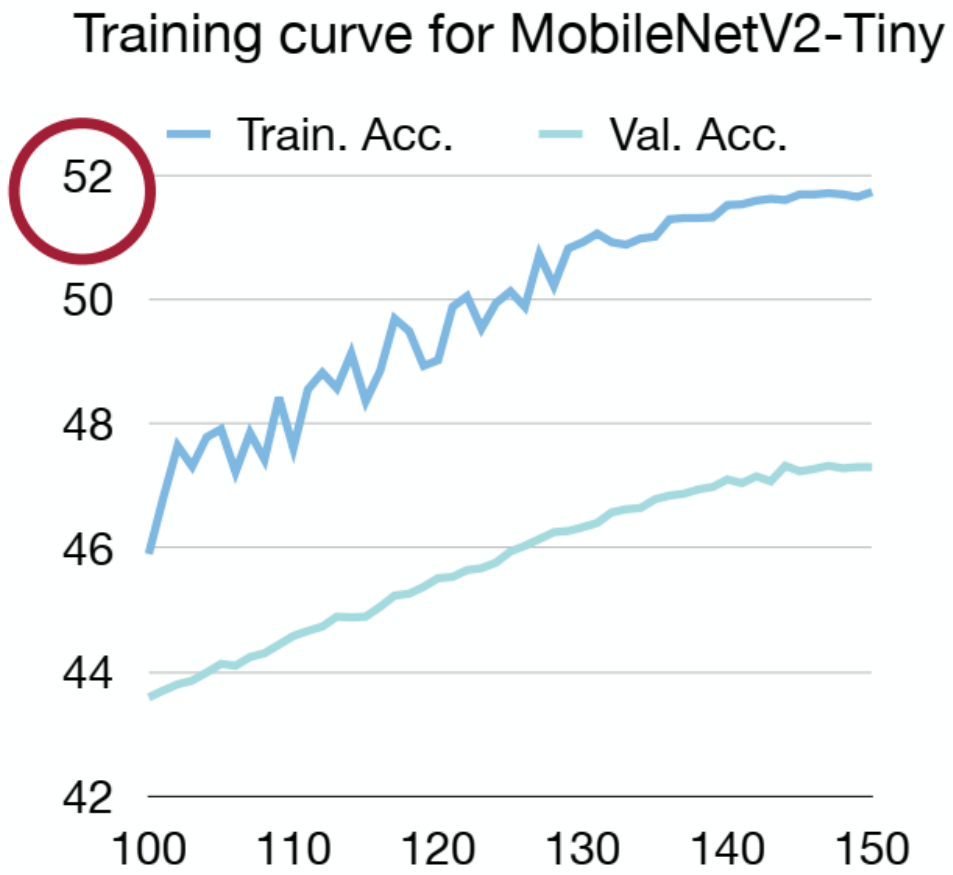


Image from https://pub.towardsai.net/smaller-faster-smarter-the-power-of-model-distillation-d002662308c7

Tiny models are hard to train

- Tiny models underfit large datasets



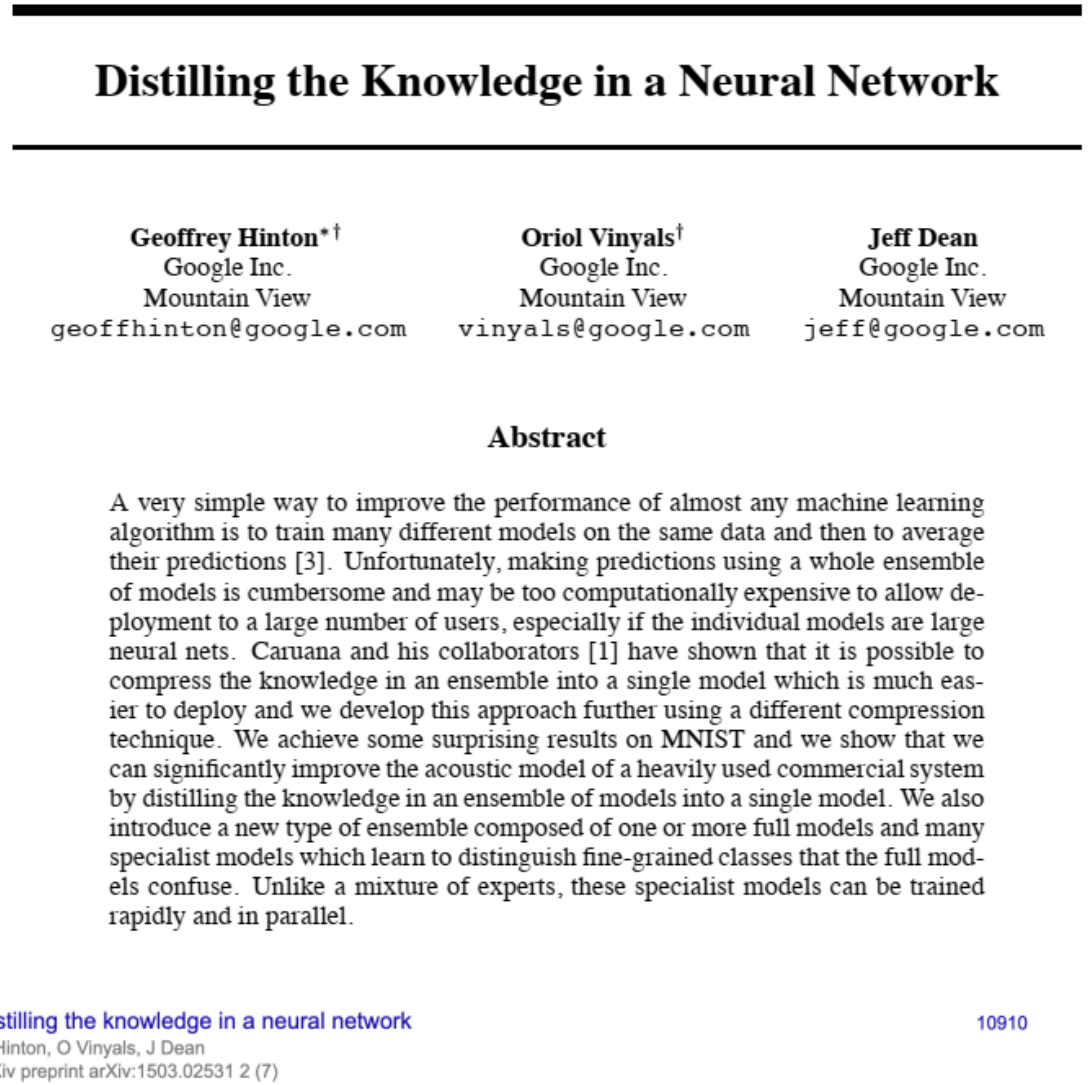


Question: Can we help the training of tiny models with large models?

From https://efficientml.ai

Knowledge Distillation

- Using a deep network or averaging the predictions of different models (ensemble) gives a better accuracy than using a single shallower network.
- knowledge distillation transfers the knowledge learned by the complex model (teacher) to the simpler model (student).
- Student network can achieve an accuracy that would be unachievable if it was directly trained with the same data set.



Distilling the Knowledge in a Neural Network

Geoffrey Hinton[*†]
Google Inc.
Mountain View
geoffhinton@google.com

Oriol Vinyals[†]
Google Inc.
Mountain View
vinyals@google.com

Jeff Dean
Google Inc.
Mountain View
jeff@google.com

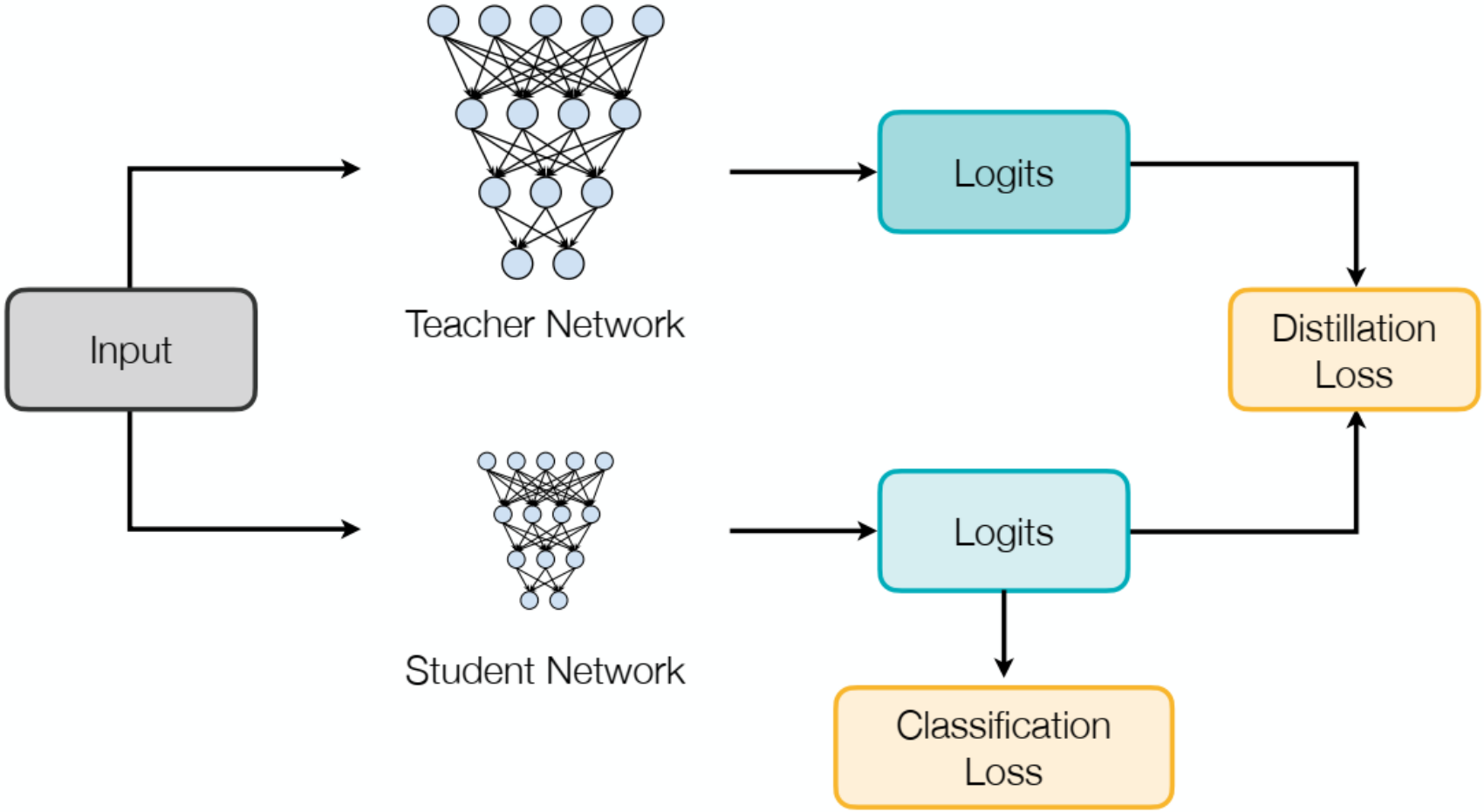
Abstract

A very simple way to improve the performance of almost any machine learning algorithm is to train many different models on the same data and then to average their predictions [3]. Unfortunately, making predictions using a whole ensemble of models is cumbersome and may be too computationally expensive to allow deployment to a large number of users, especially if the individual models are large neural nets. Caruana and his collaborators [1] have shown that it is possible to compress the knowledge in an ensemble into a single model which is much easier to deploy and we develop this approach further using a different compression technique. We achieve some surprising results on MNIST and we show that we can significantly improve the acoustic model of a heavily used commercial system by distilling the knowledge in an ensemble of models into a single model. We also introduce a new type of ensemble composed of one or more full models and many specialist models which learn to distinguish fine-grained classes that the full models confuse. Unlike a mixture of experts, these specialist models can be trained rapidly and in parallel.

Distilling the knowledge in a neural network
G Hinton, O Vinyals, J Dean
arXiv preprint arXiv:1503.02531 2 (7)
10910 2015

Distilling the Knowledge in a Neural Network (Geoffrey Hinton, Oriol Vinyals and Jeff Dean)

Illustration of knowledge distillation



From https://efficientml.ai

Formal Definition of KD

- Neural networks typically use a softmax function to generate the logits z_i to class probabilities $p(z_i, T) = \frac{\exp(\frac{z_i}{T})}{\sum_j \exp(\frac{z_j}{T})}$
  - i,j=0,1,2,…C-1, where C is the number of classes.
  - T is the temperature, which is normally set to 1.
- The goal of knowledge distillation is to align the class probability distributions from teacher and student networks.

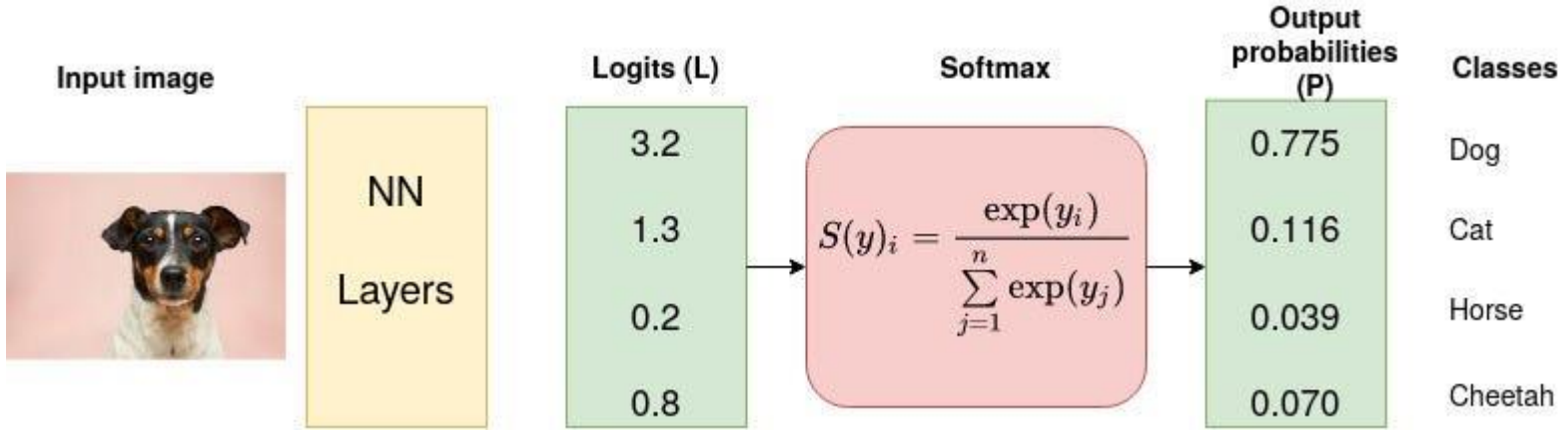
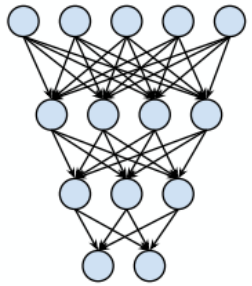


Image from https://medium.com/towards-data-science/softmax-activation-function-how-it-actually-works-d292d335bd78

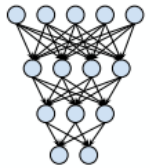
Intuition of knowledge distillation

- Matching prediction probabilities between teacher and student





Teacher Network



Student Network

	Logits	Probabilities
Cat	5	0.982
Dog	1	0.017

$$\frac{\exp(5)}{\exp(5)+\exp(1)}$$

$$\frac{\exp(1)}{\exp(5)+\exp(1)}$$

	Logits	Probabilities
Cat	3	0.731
Dog	2	0.269

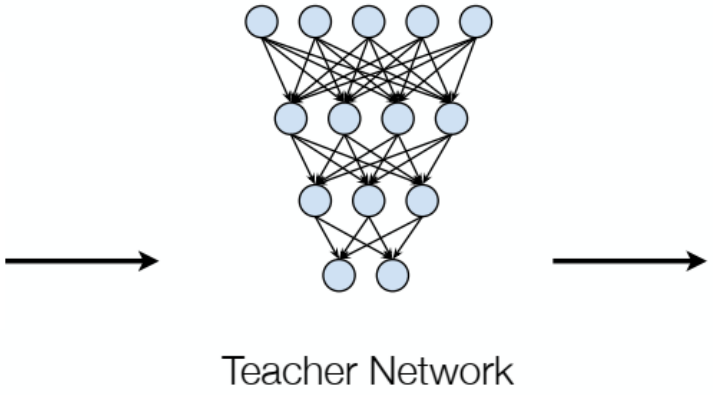
The student model is less confident

From https://efficientml.ai

Intuition of knowledge distillation

- Concept of temperature





Teacher Network

$$\frac{\exp(5/1)}{\exp(5/1)+\exp(1/1)}$$

Soft Labels

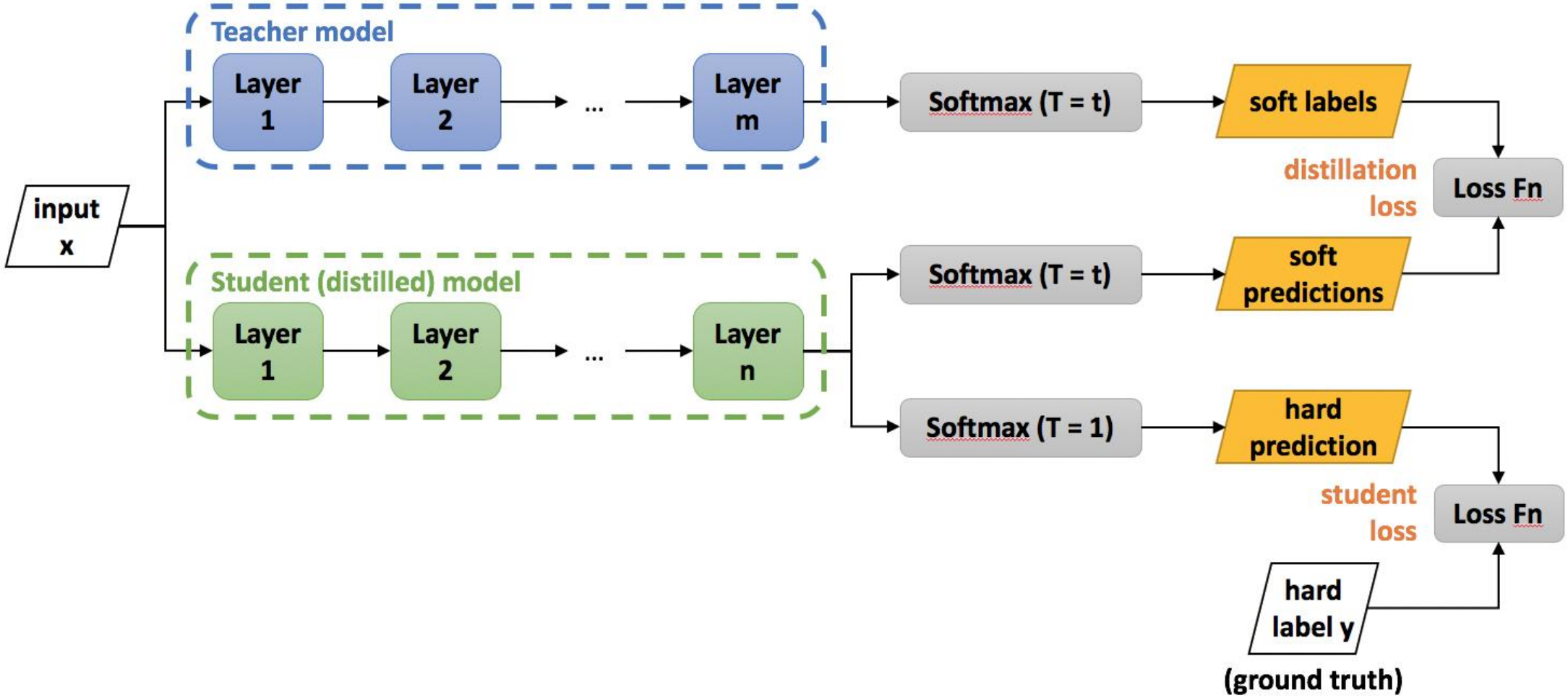
	Logits	Probabilities (T=1)	Probabilities (T=10)
Cat	5	0.982	0.599
Dog	1	0.017	0.401

$$\frac{\exp(5/10)}{\exp(5/10)+\exp(1/10)}$$

A larger temperature smooths the output probability distribution.

From https://efficientml.ai

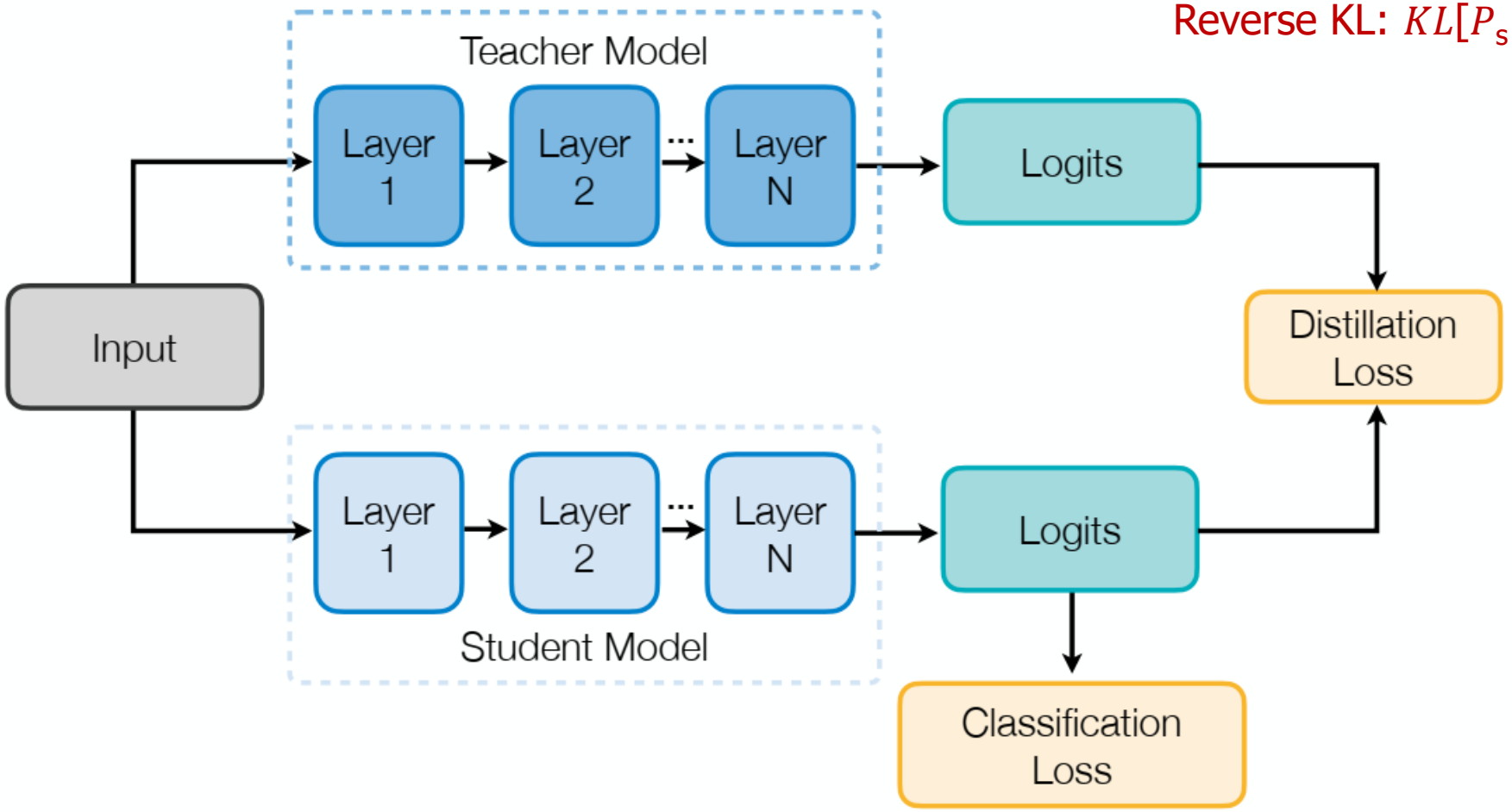
Soft Label vs Hard Label



Knowledge Distillation: A Survey [International Journal of Computer Vision, 2021]

Response-based KD

Forward KL: $KL[P_t||P_s]$, mode-covering
Reverse KL: $KL[P_s||P_t]$, mode-seeking

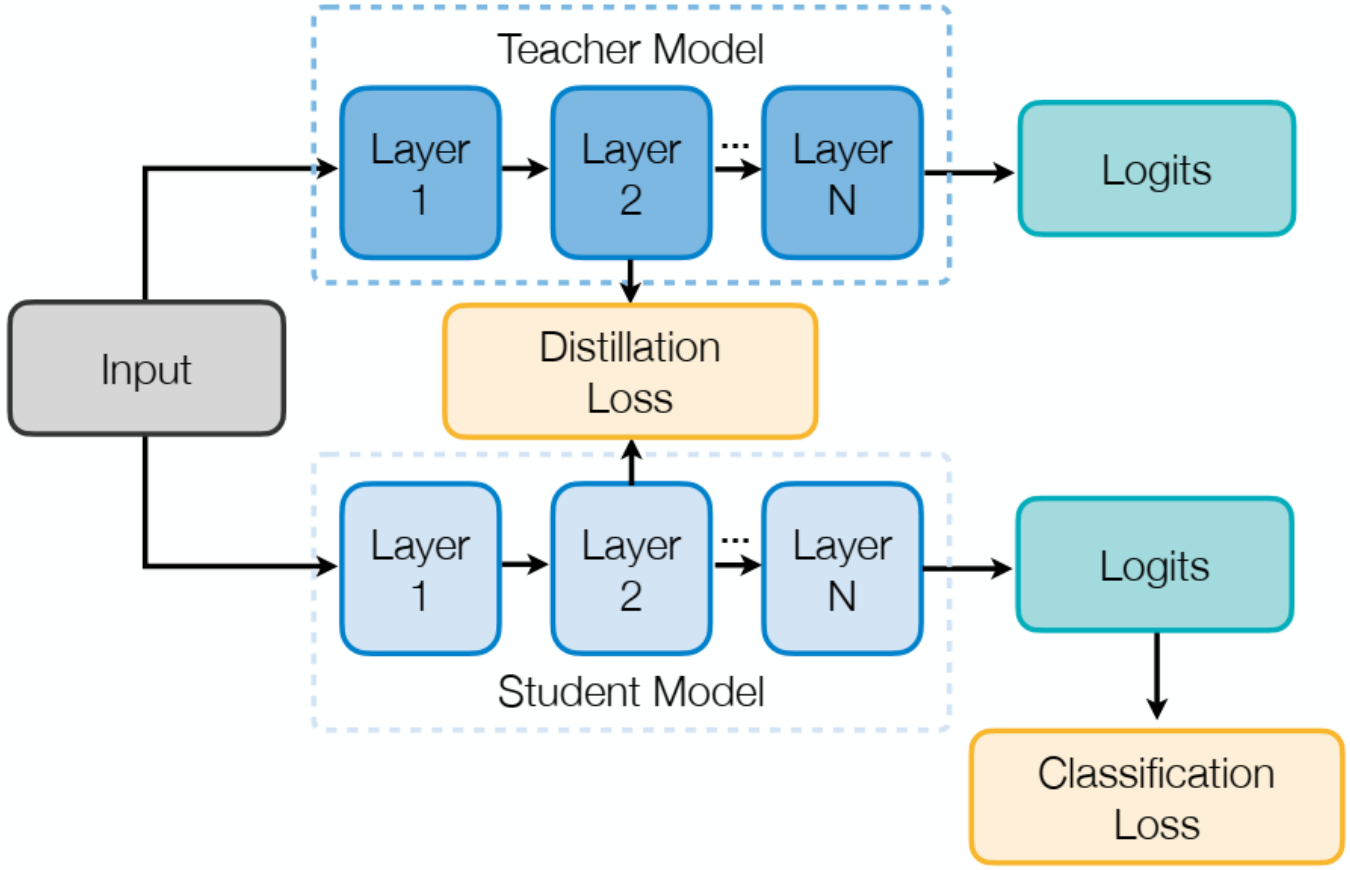


or $KL(p_t||p_s)$

Cross entropy loss:
$\mathbb{E}(-p_t \log p_s)$;
L2 loss:
$E(\|p_t - p_s\|_2^2)$

From https://efficientml.ai

Feature-based KD



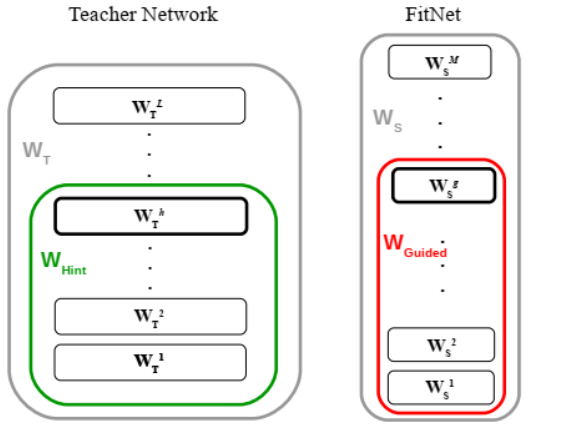
From https://efficientml.ai

Intermediate Representation KD

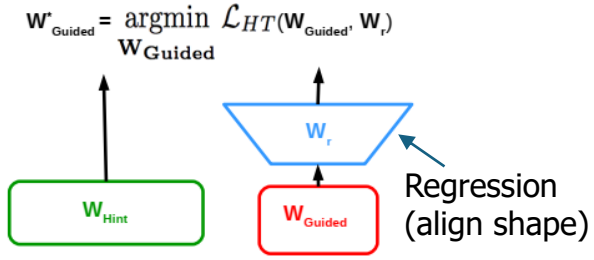
- Student explicitly mimics Teacher's feature maps or activation maps.
- (a) Feature Regression- FitNets
  - Regresses the teacher's feature maps directly
- (b) Attention Map Distillation - AT
  - Transfers the teacher's spatial attention maps to the student, effectively addressing the channel mismatch problem.
- (c) Distribution Matching (MMD-based) - NST
  - Distills the activation distribution (neuron selectivity) from teacher to student, treating feature maps as distributions.
- (d) Factor-level Representation - Factor Transfer
  - Uses an autoencoder to paraphrase or summarize the teacher's feature maps, transferring a factorized representation that is more student-friendly.

Matching intermediate weights

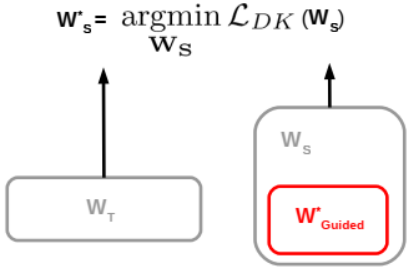
- Hint layer: teacher's hidden layer responsible for guiding the student's learning process.
- Guided layer: student's hidden layer to learn from the teacher's hint layer.
  - The guided layer should be trained to predict the output of the hint layer.



(a) Teacher and Student Networks



(b) Hints Training



(c) Knowledge Distillation

Input: $\mathbf{W_S}, \mathbf{W_T}, g, h$
Output: $\mathbf{W_S^*}$

1: $\mathbf{W_{Hint}} \leftarrow \{\mathbf{W_T}^1, \ldots, \mathbf{W_T}^h\}$
2: $\mathbf{W_{Guided}} \leftarrow \{\mathbf{W_S}^1, \ldots, \mathbf{W_S}^g\}$
3: Intialize $\mathbf{W_r}$ to small random values
4: $\mathbf{W^*_{Guided}} \leftarrow \underset{\mathbf{W_{Guided}}}{\text{argmin}}\ \mathcal{L}_{HT}(\mathbf{W_{Guided}}, \mathbf{W_r})$
5: $\{\mathbf{W_S}^1, \ldots, \mathbf{W_S}^g\} \leftarrow \{\mathbf{W_{Guided}}^{*1}, \ldots, \mathbf{W_{Guided}}^{*g}\}$
6: $\mathbf{W_S^*} \leftarrow \underset{\mathbf{W_S}}{\text{argmin}}\ \mathcal{L}_{KD}(\mathbf{W_S})$

$$\mathcal{L}_{HT}(\mathbf{W_{Guided}}, \mathbf{W_r}) = \frac{1}{2}||u_h(\mathbf{x}; \mathbf{W_{Hint}}) - r(v_g(\mathbf{x}; \mathbf{W_{Guided}}); \mathbf{W_r})||^2$$

regression

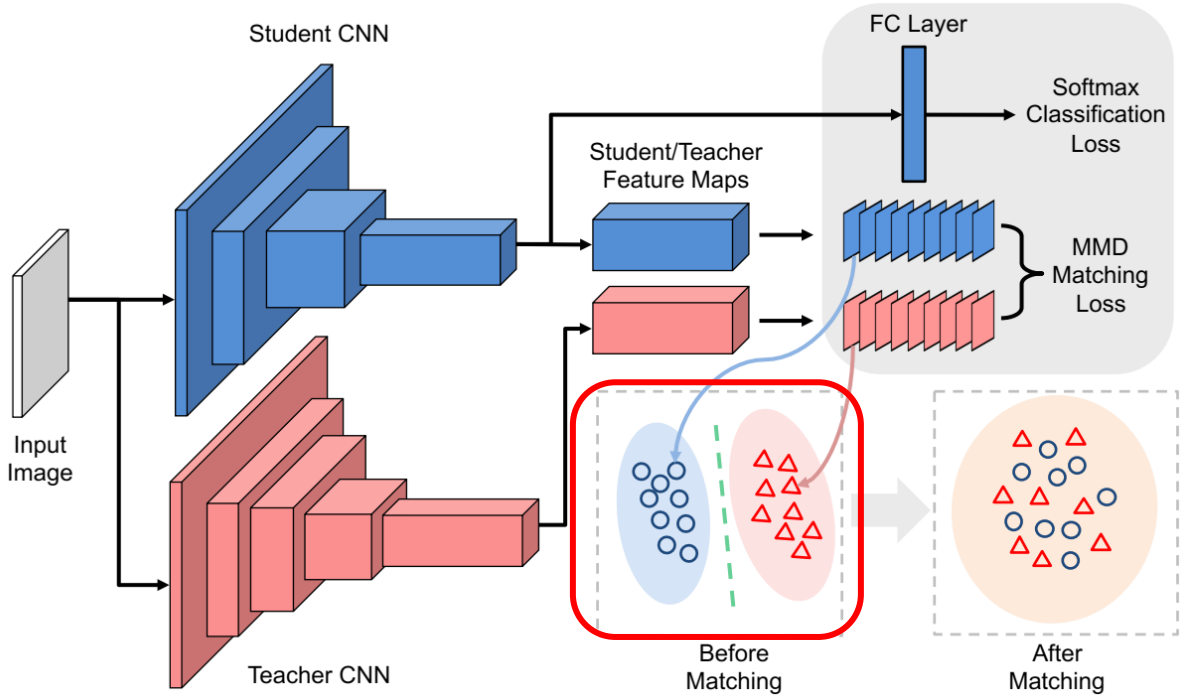
$$\mathcal{L}_{KD}(\mathbf{W_S}) = \mathcal{H}(\mathbf{y_{true}}, \mathrm{P_S}) + \lambda \mathcal{H}(\mathrm{P_T^\tau}, \mathrm{P_S^\tau})$$

Cross entropy

FitNets: Hints for Thin Deep Nets [Romero *et al.*, ICLR 2015]

Neuron Selectivity Transfer (NST)

- Minimizing maximum mean discrepancy between feature maps
- Intuition: teacher and student networks should have similar feature distributions, not just output probability distributions.

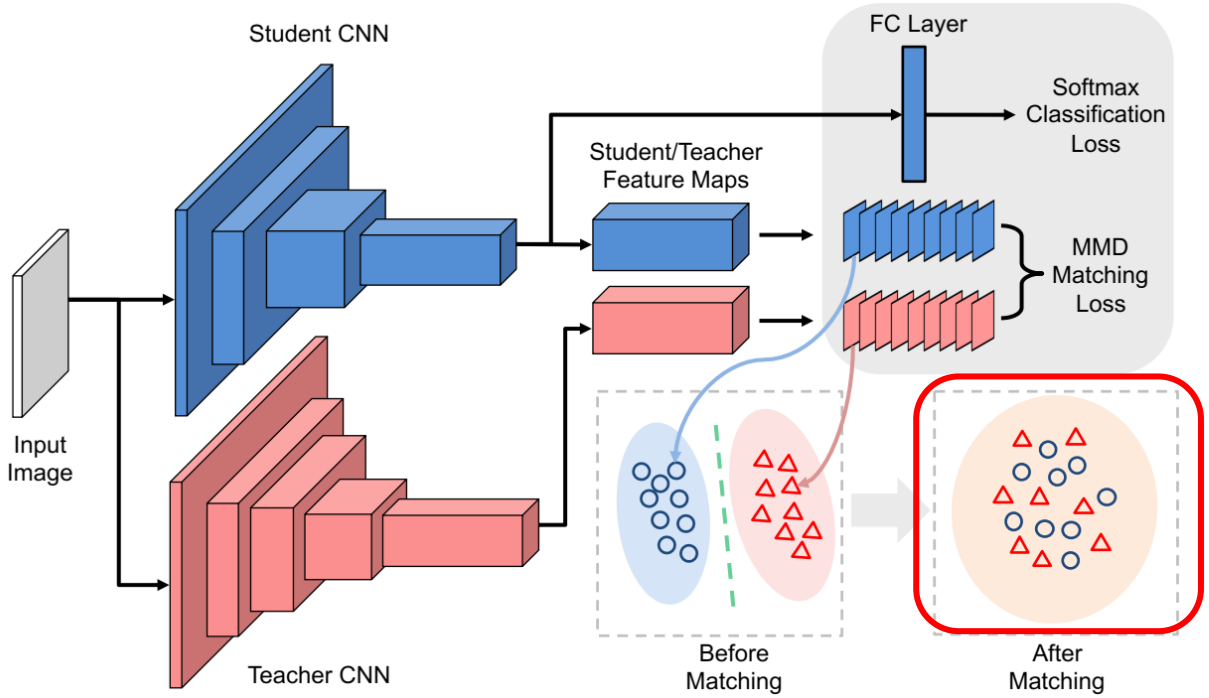


Teacher and student have very different feature distributions without distillation

Like What You Like: Knowledge Distill via Neuron Selectivity Transfer [Huang and Wang, arXiv 2017]

Neuron Selectivity Transfer (NST)

- Minimizing maximum mean discrepancy between feature maps
- Intuition: teacher and student networks should have similar feature distributions, not just output probability distributions.



With the distillation objective, teacher and student feature distributions are similar

$$\mathcal{L}_{\text{MMD}^2}(\mathbf{F}_T, \mathbf{F}_S) = \frac{1}{{C_T}^2}\sum_{i=1}^{C_T}\sum_{i'=1}^{C_T} k\left(\frac{\mathbf{f}_T^{i\cdot}}{\|\mathbf{f}_T^{i\cdot}\|_2}, \frac{\mathbf{f}_T^{i'\cdot}}{\|\mathbf{f}_T^{i'\cdot}\|_2}\right) + \frac{1}{{C_S}^2}\sum_{j=1}^{C_S}\sum_{j'=1}^{C_S} k\left(\frac{\mathbf{f}_S^{j\cdot}}{\|\mathbf{f}_S^{j\cdot}\|_2}, \frac{\mathbf{f}_S^{j'\cdot}}{\|\mathbf{f}_S^{j'\cdot}\|_2}\right) - \frac{2}{C_T C_S}\sum_{i=1}^{C_T}\sum_{j=1}^{C_S} k\left(\frac{\mathbf{f}_T^{i\cdot}}{\|\mathbf{f}_T^{i\cdot}\|_2}, \frac{\mathbf{f}_S^{j\cdot}}{\|\mathbf{f}_S^{j\cdot}\|_2}\right).$$

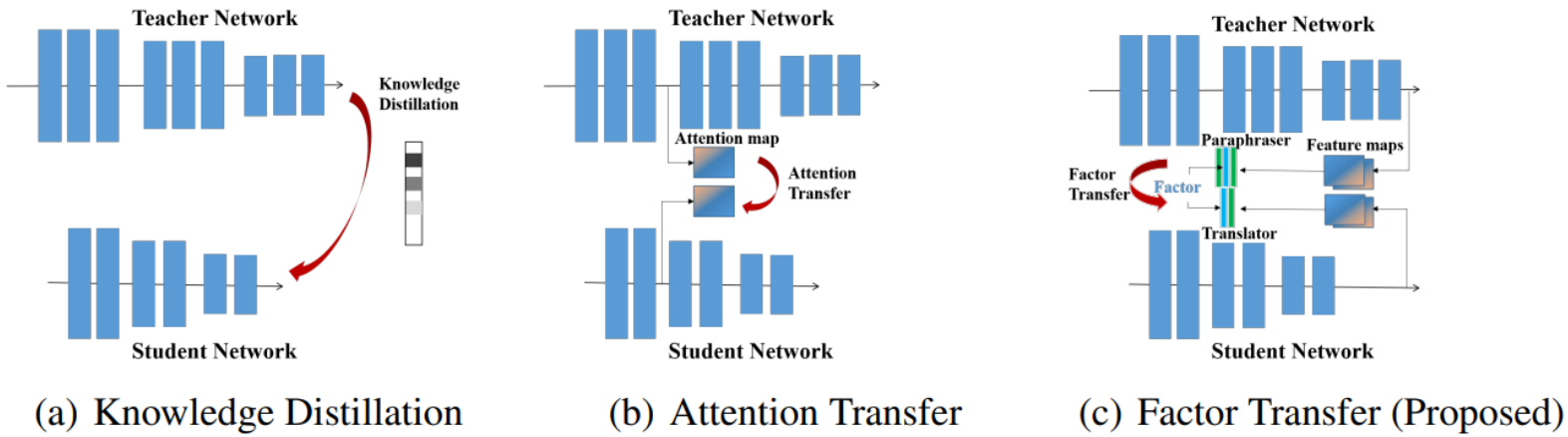
Cosine of angle between teacher/student feature vectors

Use maximum mean discrepancy (MMD) as an objective. k is the kernel function, and its simplest form is the dot product.

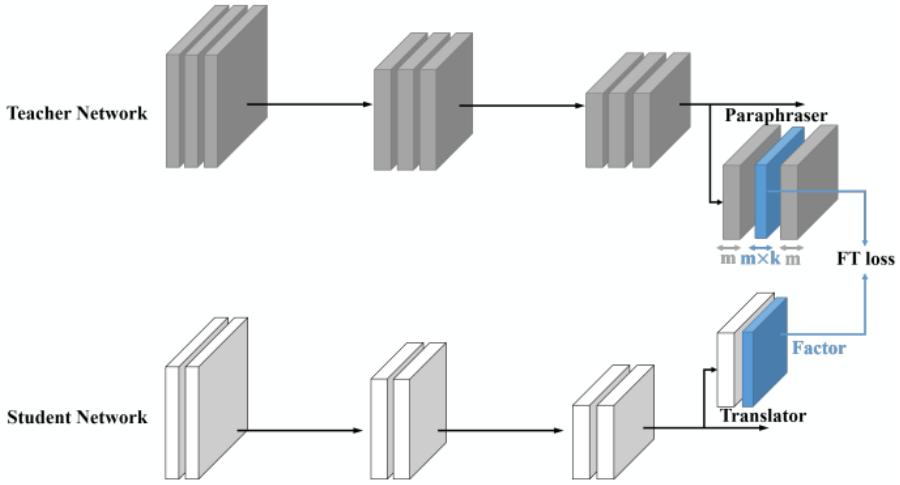
[Like What You Like: Knowledge Distill via Neuron Selectivity Transfer [Huang and Wang, arXiv 2017]]

Factor Transfer

- Directly transferring the teacher's outputs overlooks the inherent differences between the teacher network and the student network
- Need to re-interpret the output of the teacher network to resolve these differences
- Factor Transfer trains neural networks that can extract good factors and to match these factors.
- Paraphraser (Translator) extracts factors from a teacher (student) network.



(a) Knowledge Distillation (b) Attention Transfer (c) Factor Transfer (Proposed)



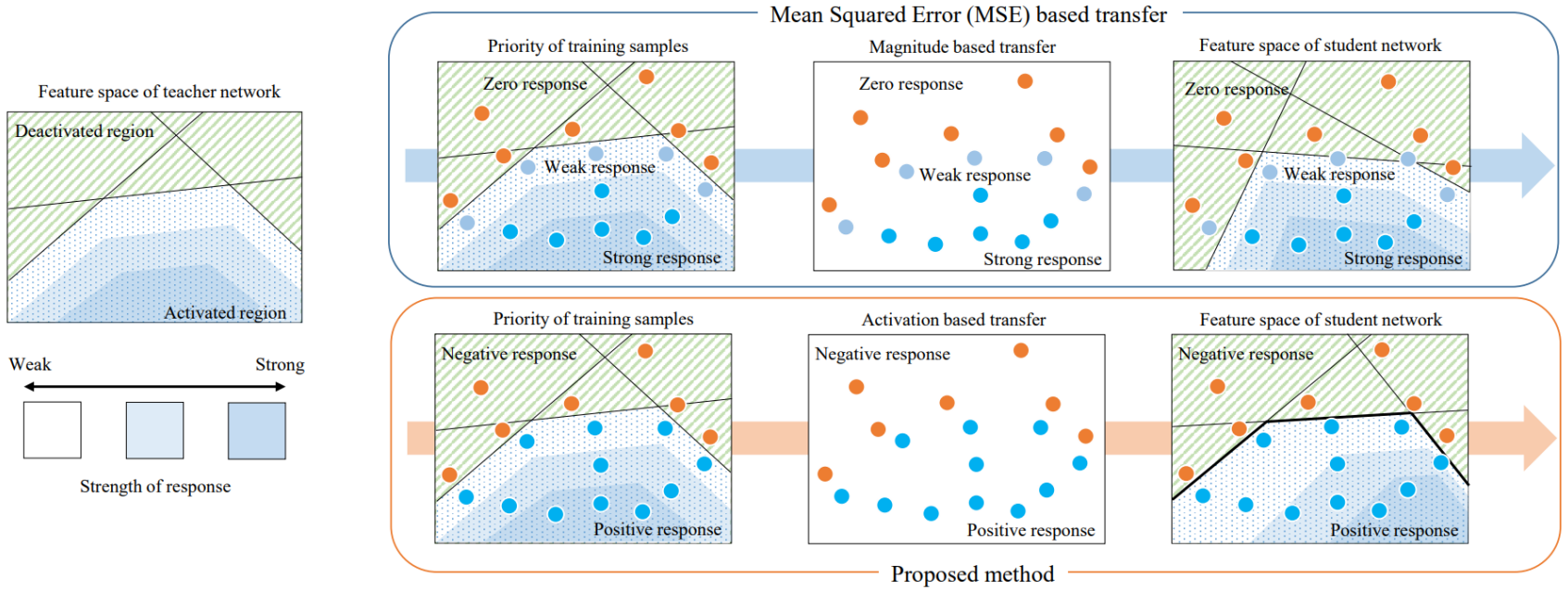
Paraphrasing Complex Network: Network Compression via Factor Transfer [Kim *et al.*, NeurIPS 2018]

Structural / Functional KD

- Instead of matching raw feature values, transfer the rules and structure embedded in the relationships between features.
- Distillation of structure, not value.
- Robust to architecture mismatch.
- **(a) Process-aware KD**
  - Distills the teacher's layer-to-layer feature transformation matrix (FSP Matrix)
- **(b) Activation Boundary KD**
  - Forces the student to replicate the teacher's ReLU activation boundaries.
- **(c) Relational KD**
  - Distills distances and angles between samples to preserve the geometry of the feature space

Matching sparsity patterns

- Intuition: the teacher and student networks should have similar sparsity patterns after the ReLU activation.
- a constant penalty when the activations of teacher and student are different.
- Transfer of activation boundaries



$$\sigma(x) = max(0, x)$$

$$\mathcal{L}(\boldsymbol{I}) = \|\sigma(\mathcal{T}(\boldsymbol{I})) - \sigma(\mathcal{S}(\boldsymbol{I}))\|_2^2$$

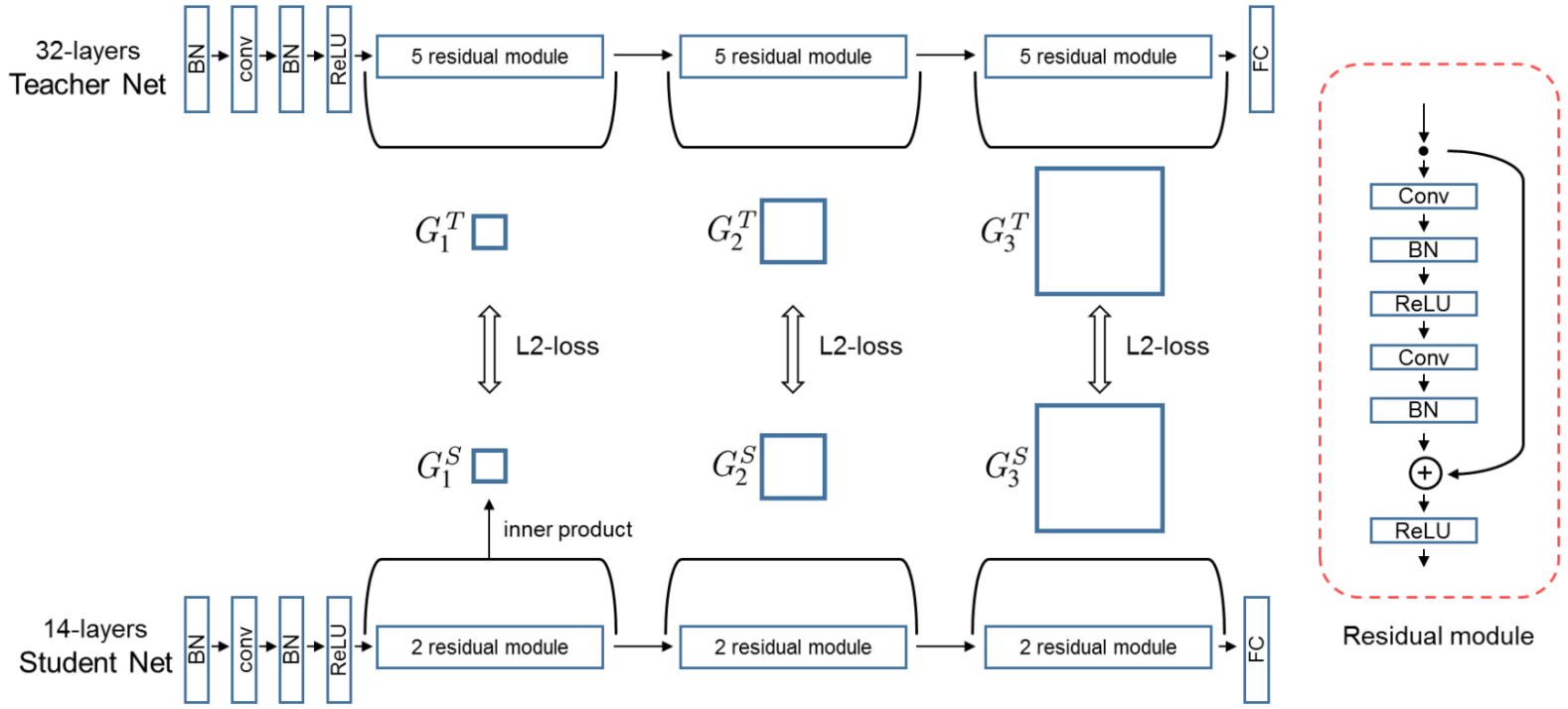
$$\rho(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise.} \end{cases}$$

$$\mathcal{L}(\boldsymbol{I}) = \|\rho(\mathcal{T}(\boldsymbol{I})) - \rho(\mathcal{S}(\boldsymbol{I}))\|_1$$

Knowledge Transfer via Distillation of Activation Boundaries Formed by Hidden Neurons [Heo *et al.*, AAAI 2019

Matching relational information

- Relations between different layers



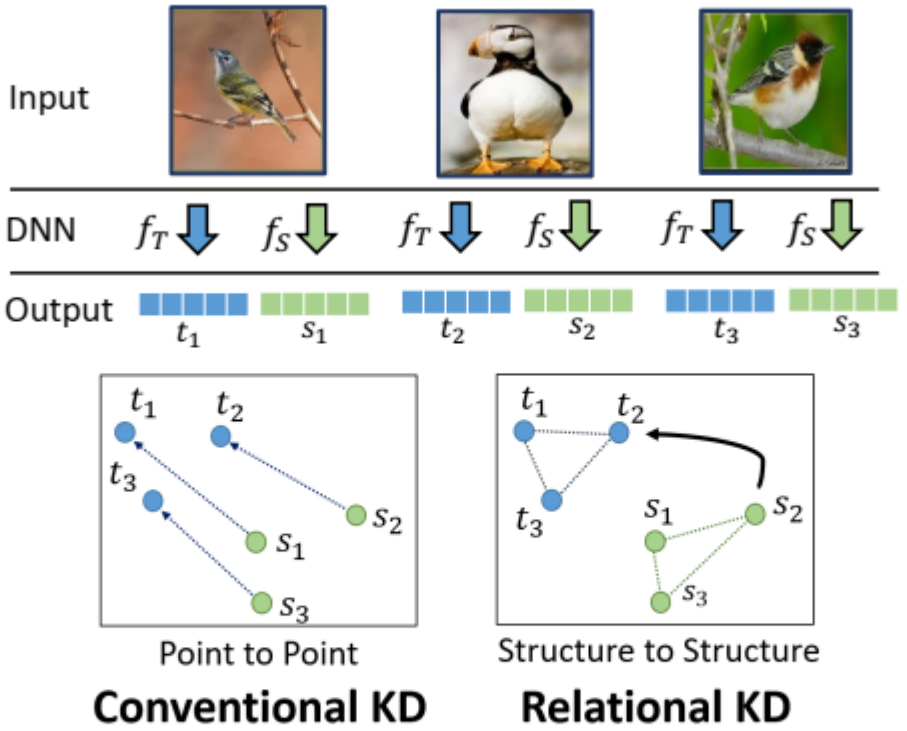
$$G_{i,j}(x;W) = \sum_{s=1}^{h}\sum_{t=1}^{w} \frac{F^1_{s,t,i}(x;W) \times F^2_{s,t,j}(x;W)}{h \times w}$$

- FSP (flow of solution procedure) matrices of teacher and student networks (G_1^T, G_1^S).
- Inner product of the features from two layers.

A Gift from Knowledge Distillation: Fast Optimization, Network Minimization and Transfer Learning [Yim *et al.*, CVPR 2017]

Matching relational information

- Relations between different samples



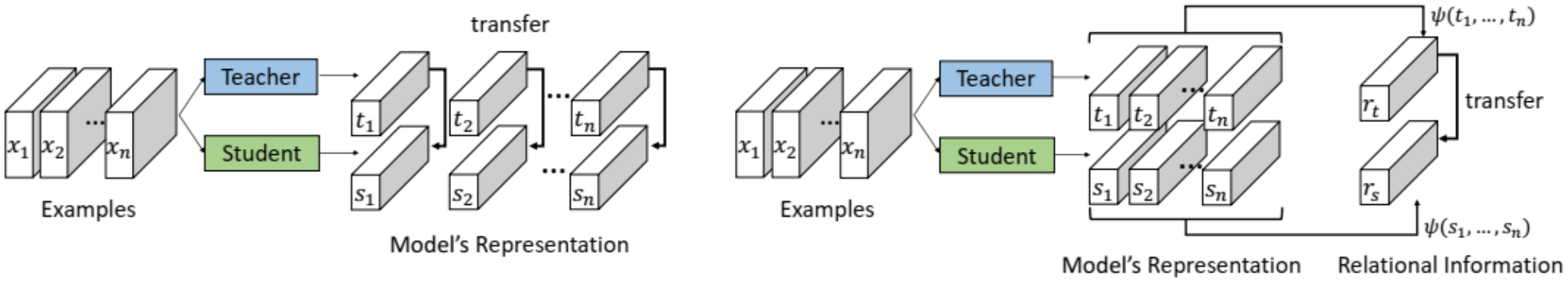
Conventional KD focuses on matching features / logits for **one input**.
Relational KD looks at the **relations** between intermediate features from **multiple inputs**.

Relational Knowledge Distillation [Park *et al.*, CVPR 2019]

Matching relational information

- Relations between different samples

$\psi(s_1, s_2, \ldots, s_n) = (\|s_1 - s_2\|_2^2, \|s_1 - s_3\|_2^2, \ldots, \|s_1 - s_n\|_2^2, \ldots \|s_{n-1} - s_n\|_2^2)$ is a vector of length $n(n-1)/2$ representing pairwise distances of feature vectors.



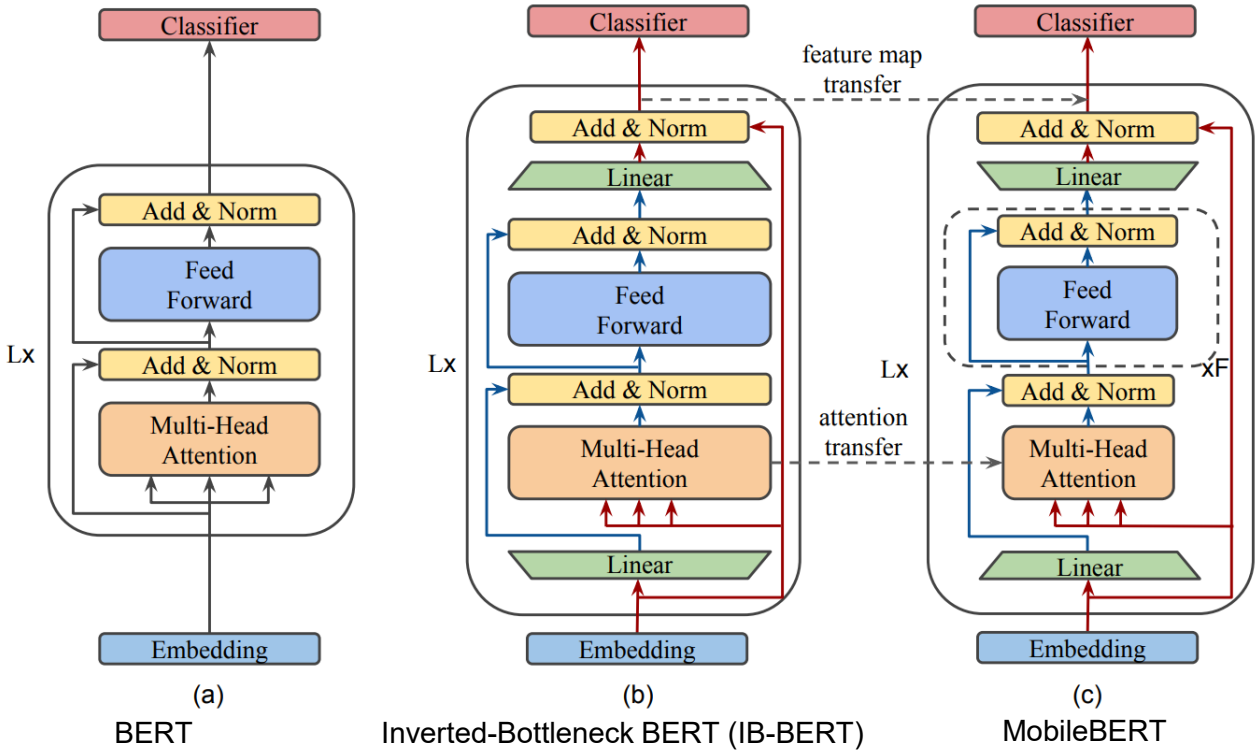
Representative method: FSP (previous slide)
Relation **within** the model, calculated **separately** for each input

Representative method: RKD
Relation **across different samples**

Relational Knowledge Distillation [Park *et al.*, CVPR 2019]

KD for Transformer

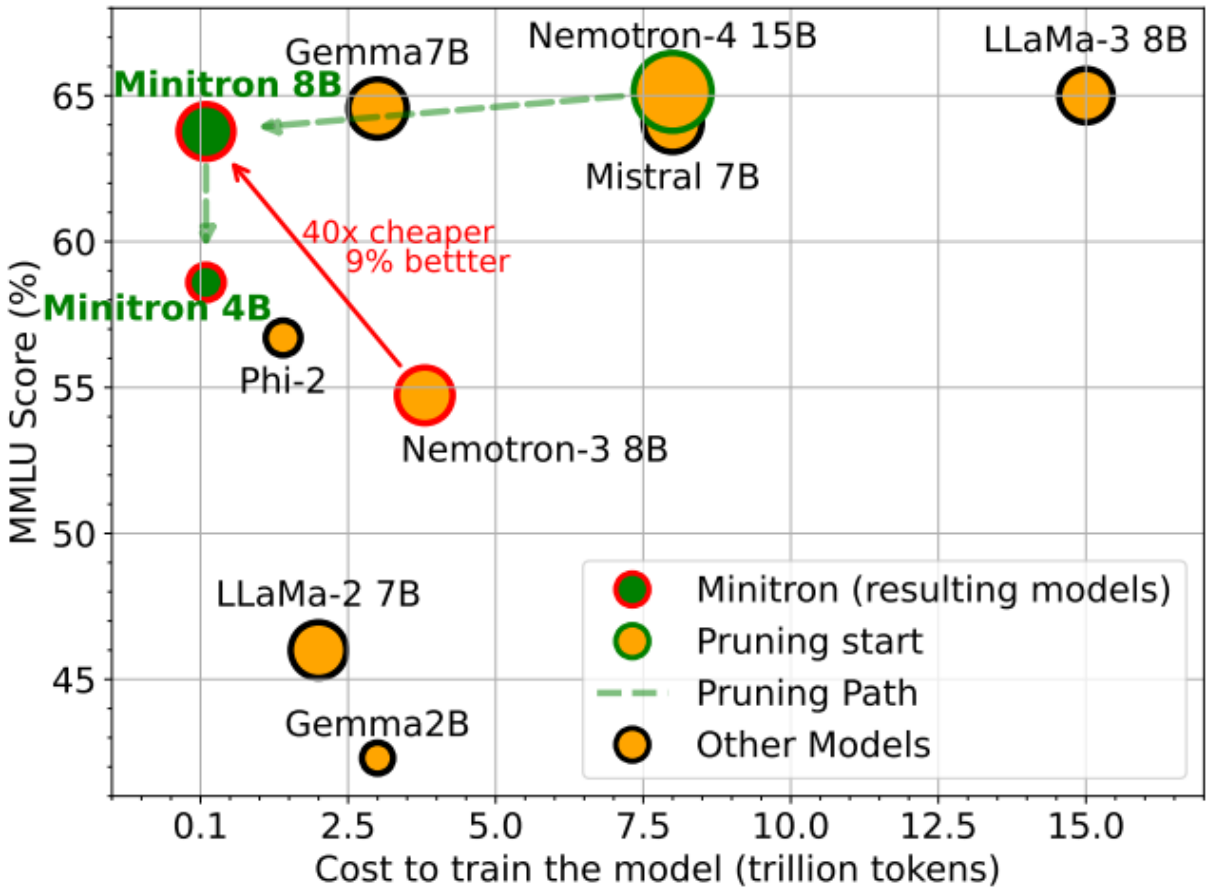
- **MobileBERT**
  - IB-BERT and MobileBERT have the same feature map size
  - Feature map transfer, Attention transfer

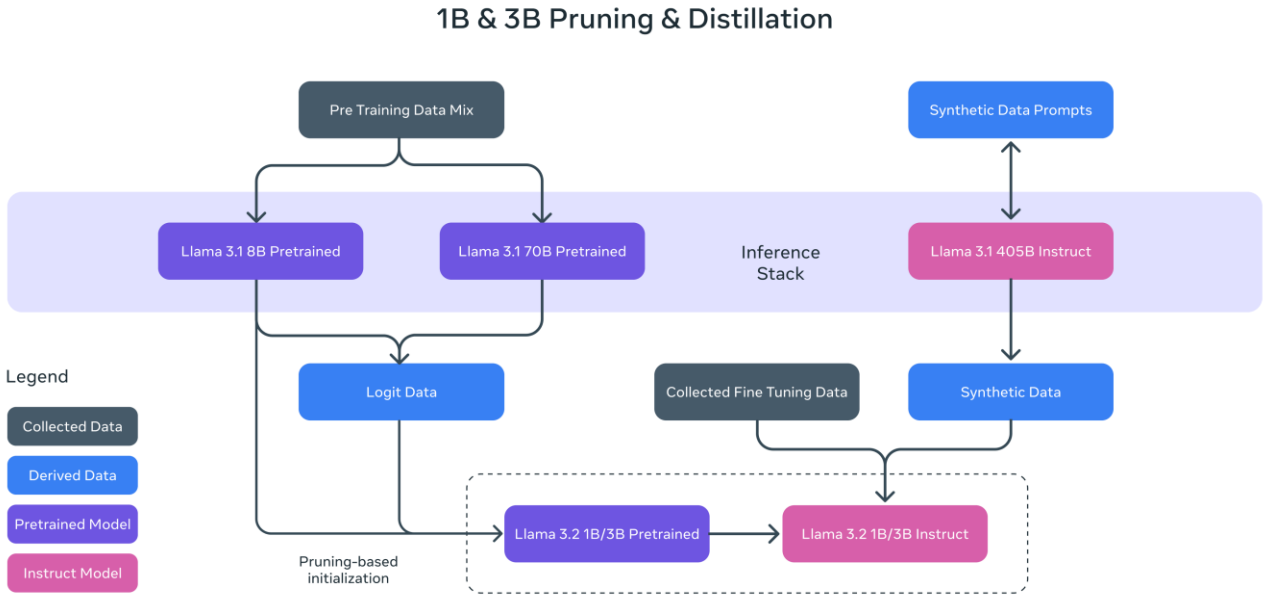


MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices [ACL'20]

KD for Transformer

https://pytorch.org/torchtune/0.5/tutorials/llama_kd_tutorial.html



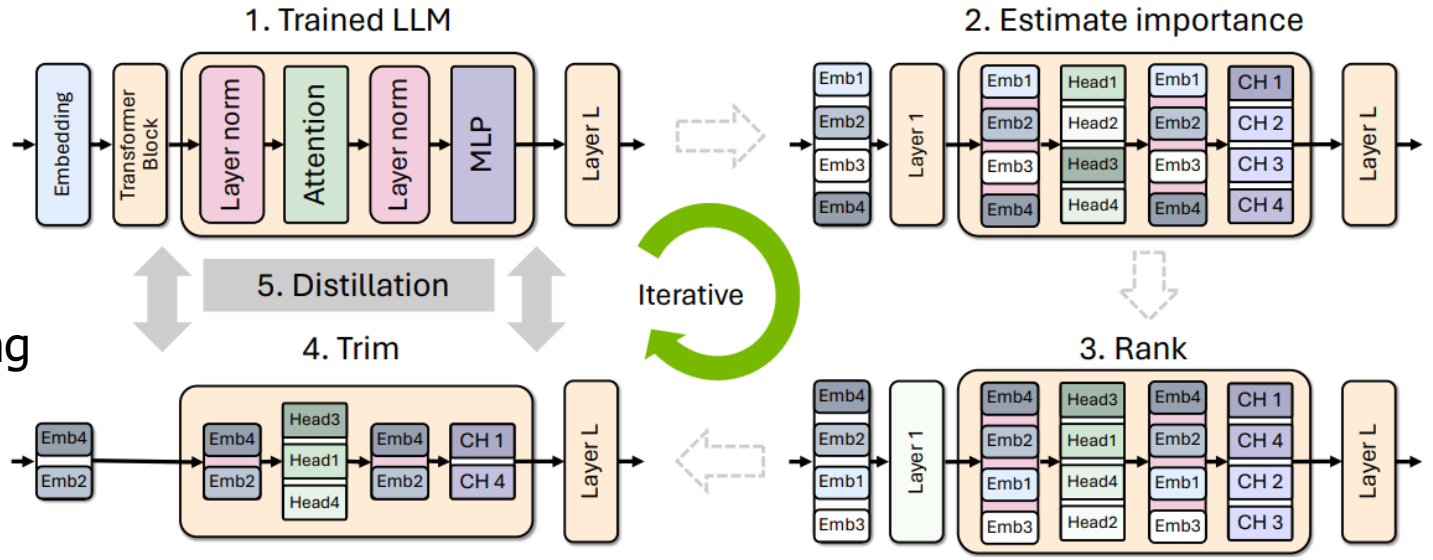


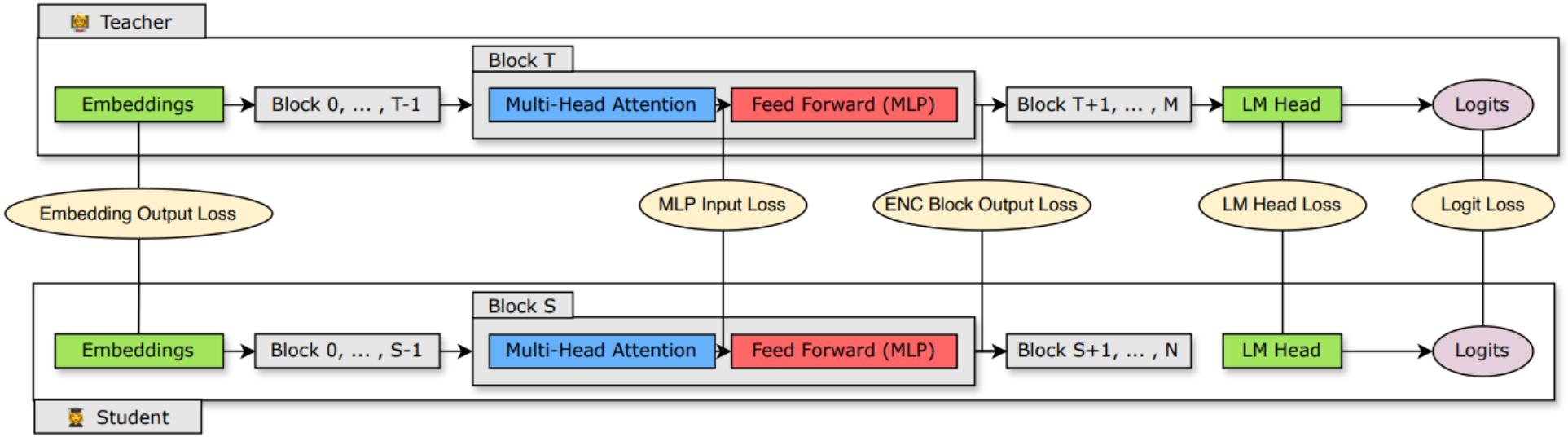
Llama 3.2: Revolutionizing edge AI and vision with open, customizable models

Compact Language Models via Pruning and Knowledge Distillation [NeurIPS'24]

KD for Transformer

applies KD during retraining after pruning





Compact Language Models via Pruning and Knowledge Distillation [NeurIPS'24]